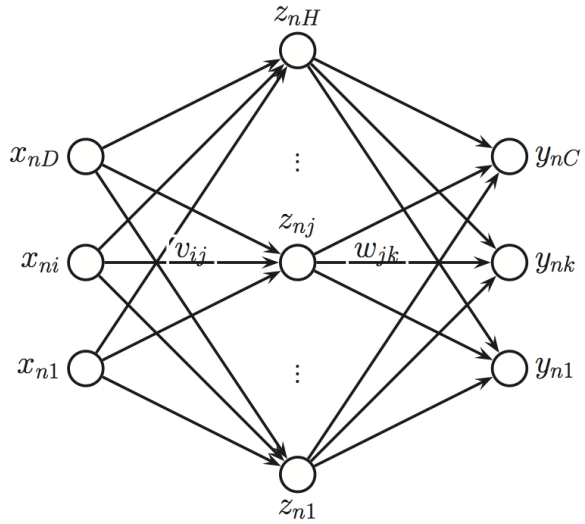
Machine Learning



Lecture 3: representation learning & Neural networks

Iasonas Kokkinos

Imperial College London

i.kokkinos@ic.ac.uk

Meeting hours and course feedback

Monday, 6–7 pm

Office 450

Questions, clarifications and feedback

forms.cloud.microsoft/e/De8UK4392r

Week 1-2: ML foundations

Loss functions

Squared L2 loss for regression, cross-entropy for classification

Regularisation

L2 shrinks coefficients. L1 encourages sparsity. Probabilistic interpretation.

Minimizers

Closed form, second-order methods, gradient descent and SGD

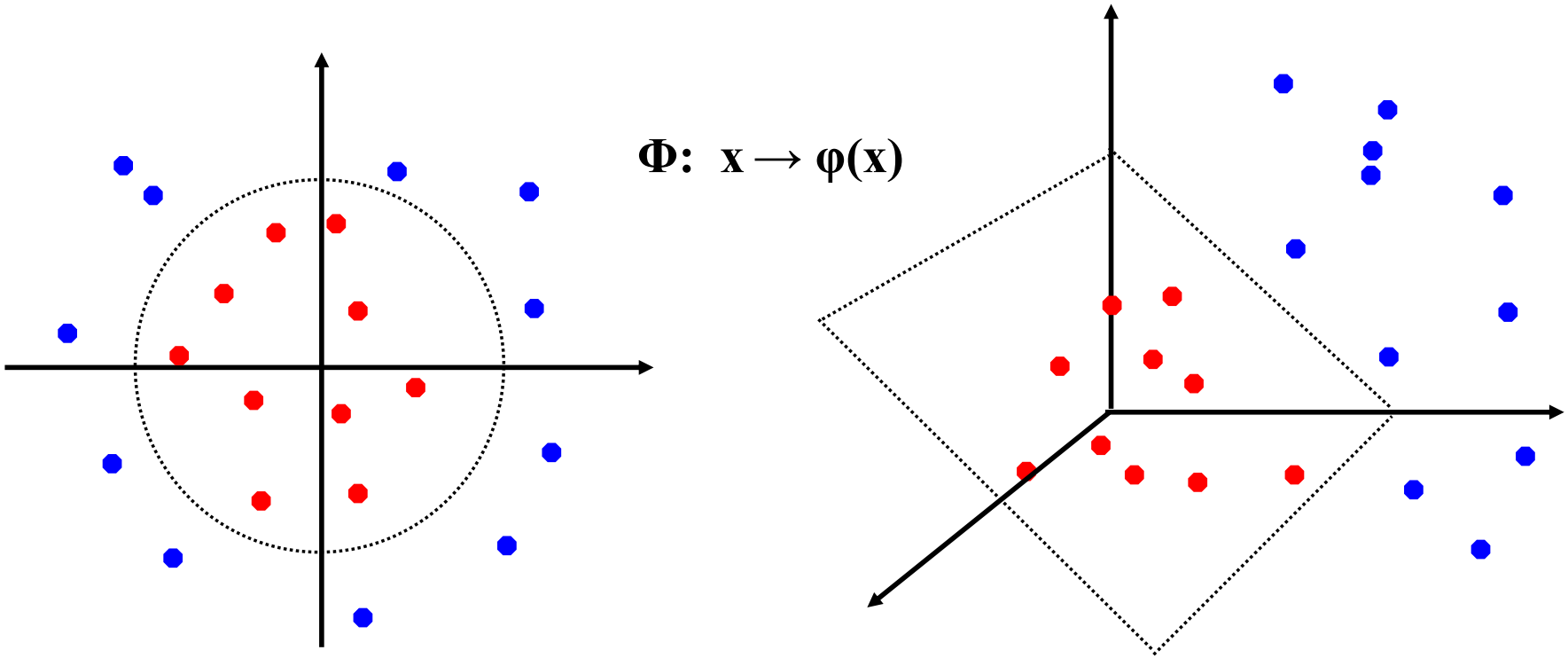
ML fundamentals: rooted in probability, statistics and optimisation.

Historical context: 1763–1970

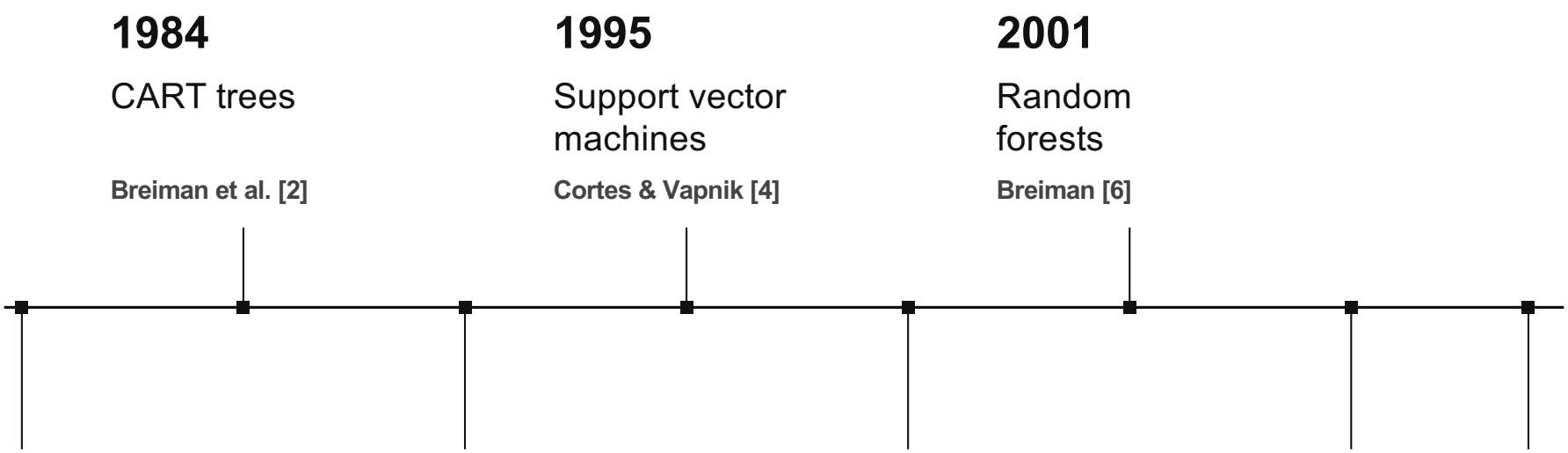
Hastie, Tibshirani & Friedman (2009), The Elements of Statistical Learning.

Beyond linear boundaries

- Weeks 1-2: “More features” & regularization

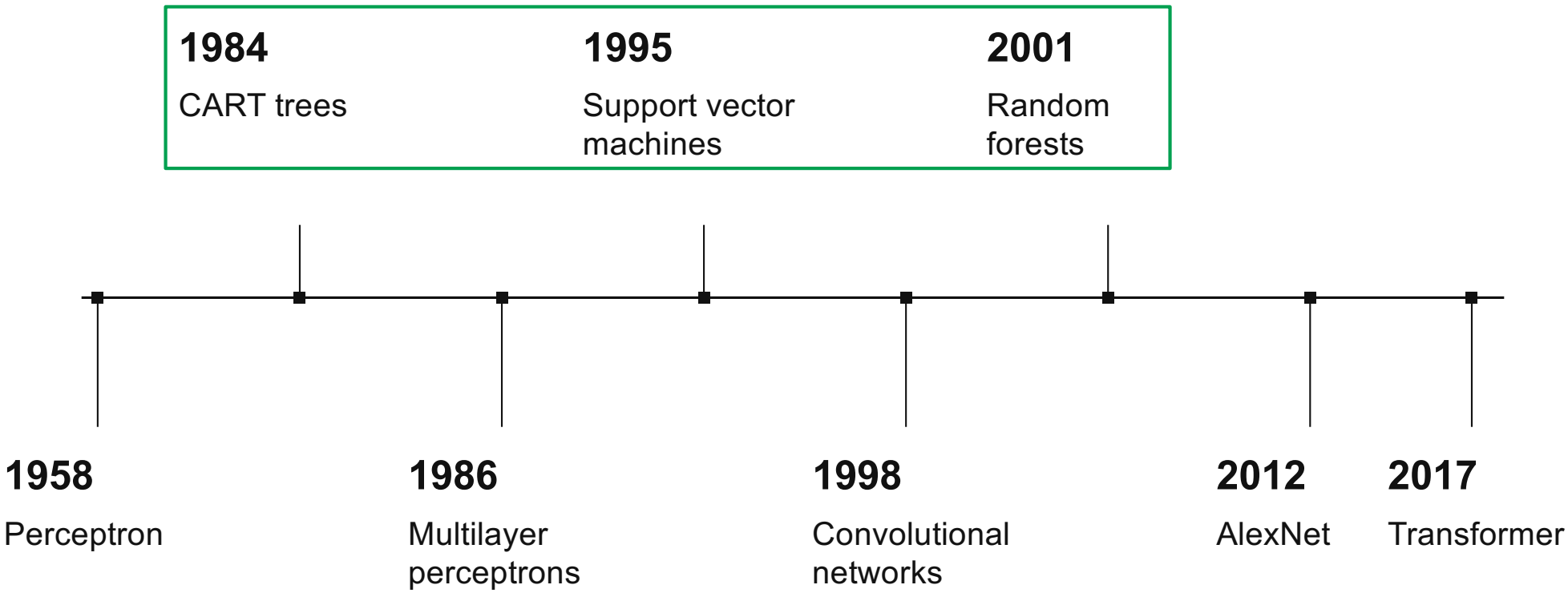


ML milestones



Selected publications. [n] refers to the full references at the end.

ML milestones: good/smart/fast features



Decision trees

Decision trees

Bagging and random forests

Kernels and SVMs

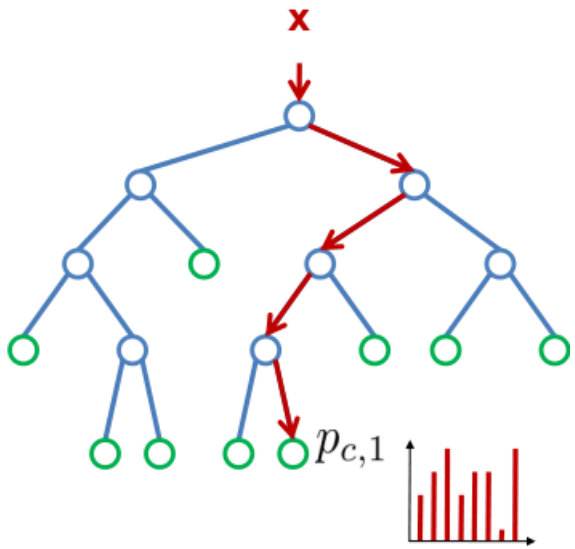
Neural networks

Expressiveness and representations

Backpropagation

1984 CART trees

Breiman, Friedman, Olshen and Stone

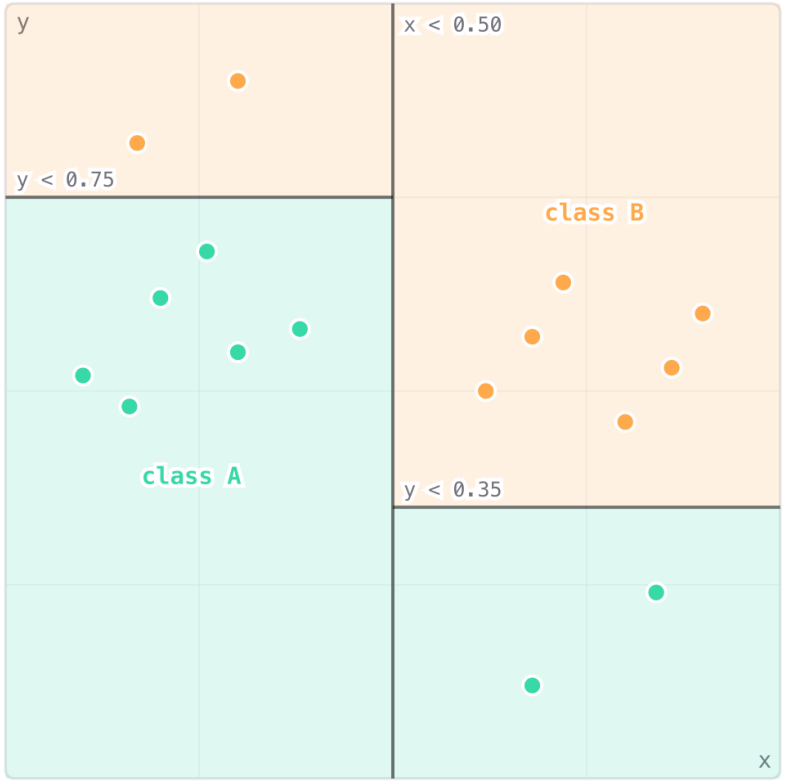


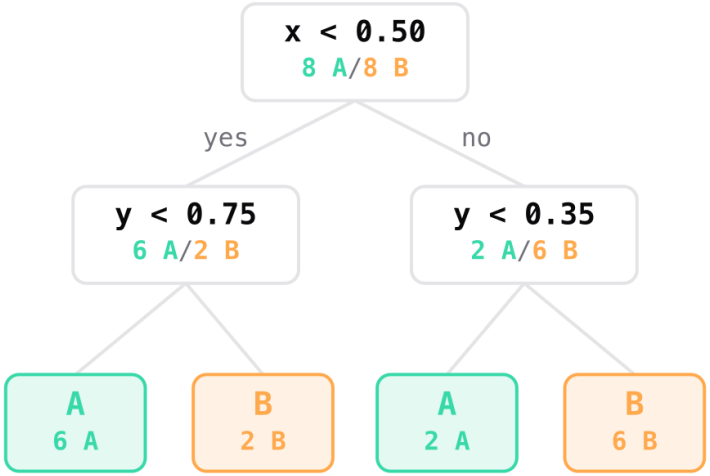
Reading: Decision trees

K. Murphy (2022), §18.1 Classification and regression trees (CART)

A path acts as a learned feature

AlgoMaster: live decision-tree animation: https://algomaster.io/animations/ai-ml/ml-decision-tree





$$\phi_{a,b}(x, y) = \mathbf{1}[x < a,\ y < b]$$

$$\phi_{0.5,0.75}(x, y) = \mathbf{1}[x < 0.5,\ y < 0.75]$$

A path feature is 1 exactly when every test on that path is satisfied.

Tree depth and overfitting

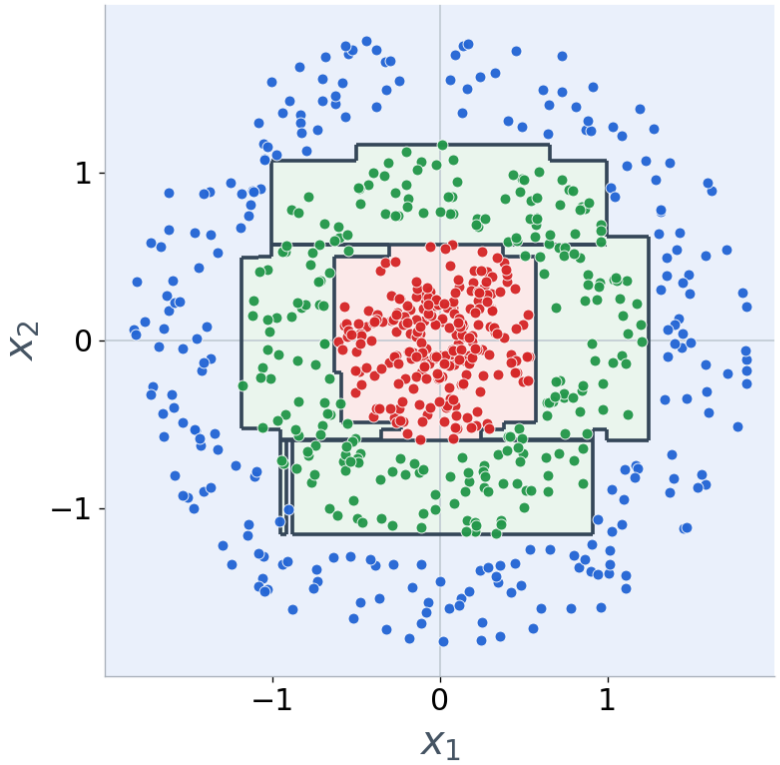
Decision tree

36 leaves

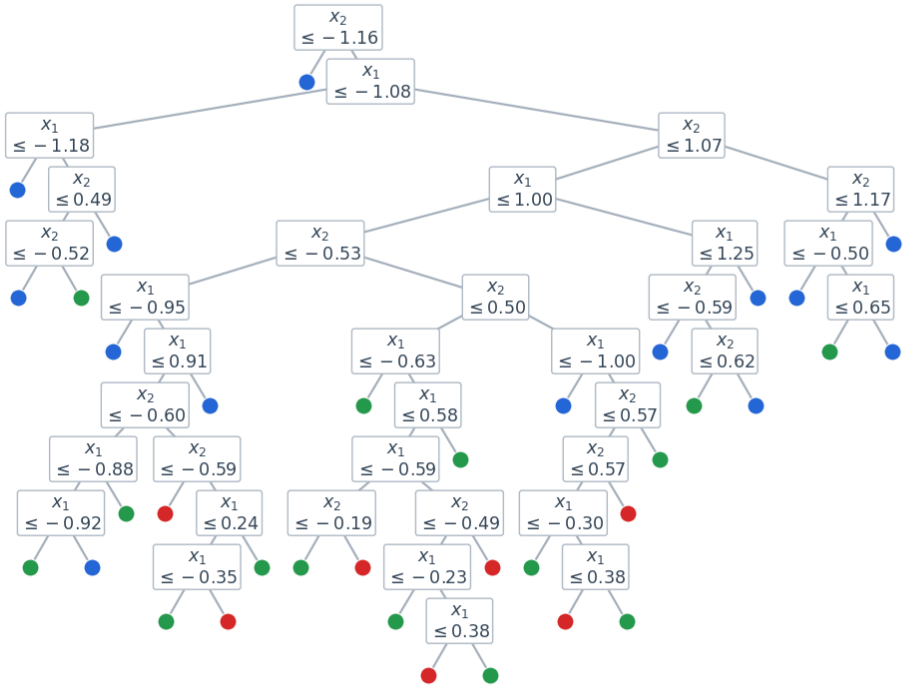
Maximum depth 12

Training error 0.0%

Decision boundary



Tree structure



Left: ≤ Right: >

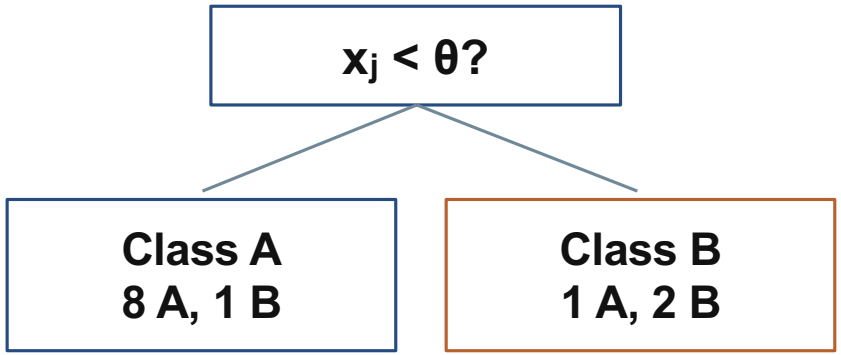
Inner ring Middle ring Outer ring Decision boundary

Beyond a useful depth, fitting training detail can hurt unseen performance.

Pruning a decision tree

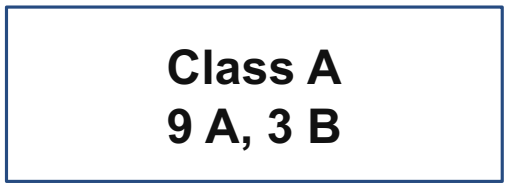
Grow the tree, then remove splits that do not help on validation data.

Before pruning



Validation errors: 4 / 20

After pruning



Validation errors: 3 / 20

1. Visit a node whose children are leaves.
2. Replace its split with a leaf fitted on its training cases.
3. Keep the change if validation performance improves.
4. Work upwards and repeat.

Training error and validation error

Tree construction	Training	Validation	Diagnosis
Early stopping	0.2975	0.4588	Underfit
Exhaustive growth	0.0142	0.3763	Overfit
Pruning	0.1025	0.1997	Best of these

Decision trees: strengths and limits

What makes trees useful

Greedy training is fast. Prediction follows a short path of tests.
Tests can use different feature types, such as colour, size or shape.

How we control complexity

Choose depth, minimum leaf size and pruning using validation data.
Compare held-out performance with the training error.

Bagging and random forests

Decision trees

Bagging and random forests

Kernels and SVMs

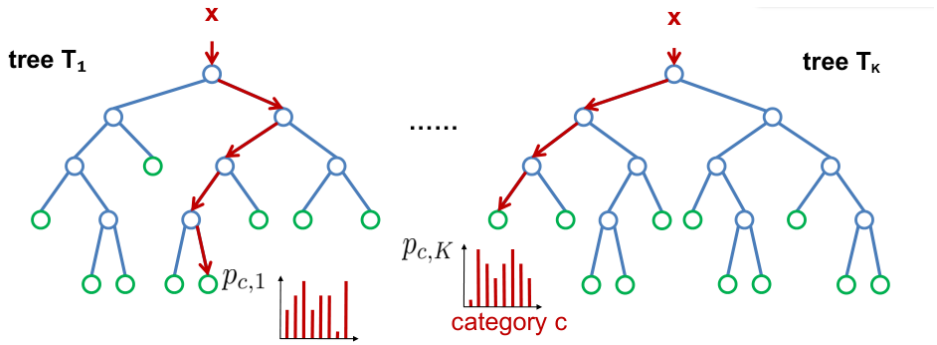
Neural networks

Expressiveness and representations

Backpropagation

2001 Random forests

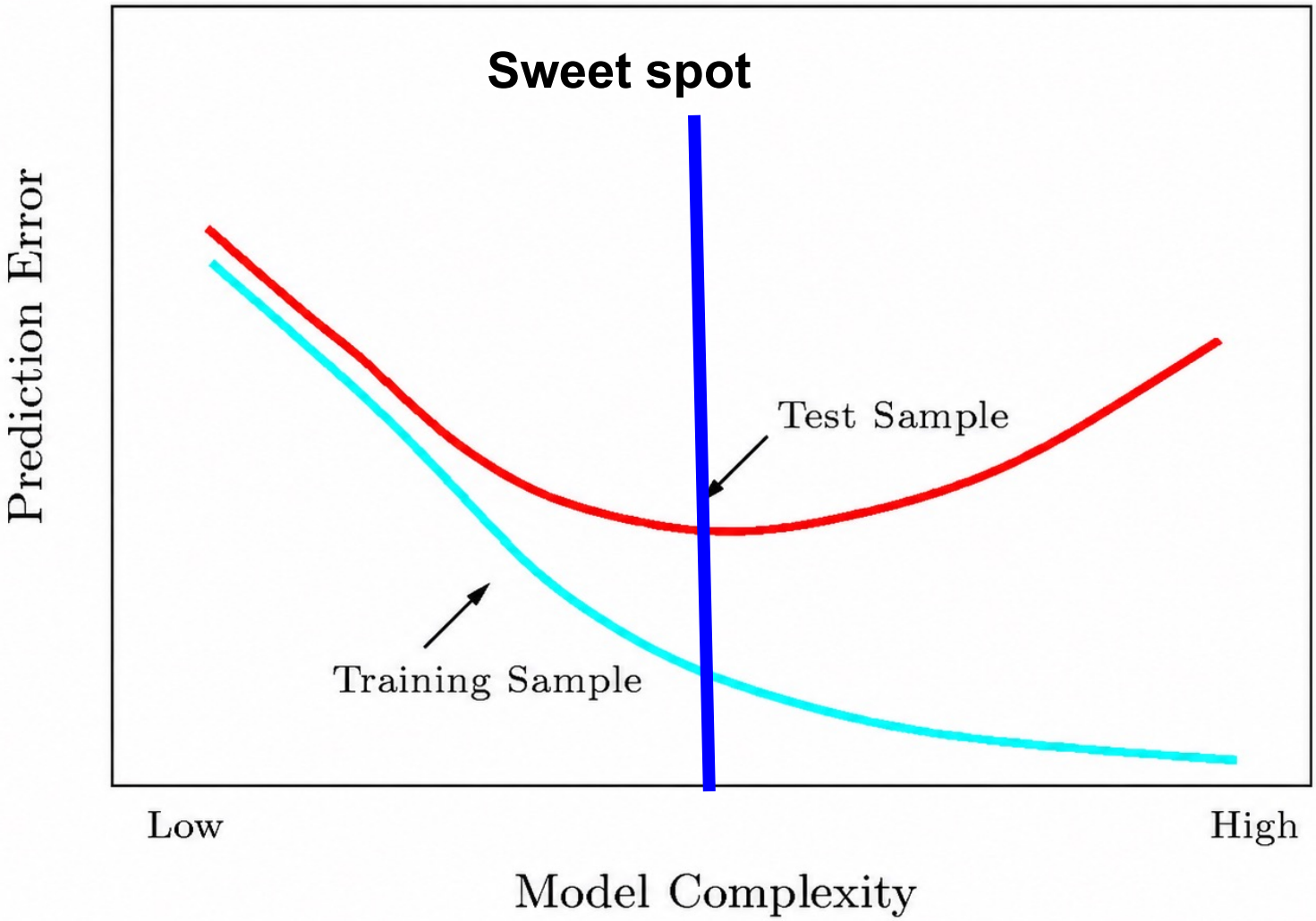
Breiman



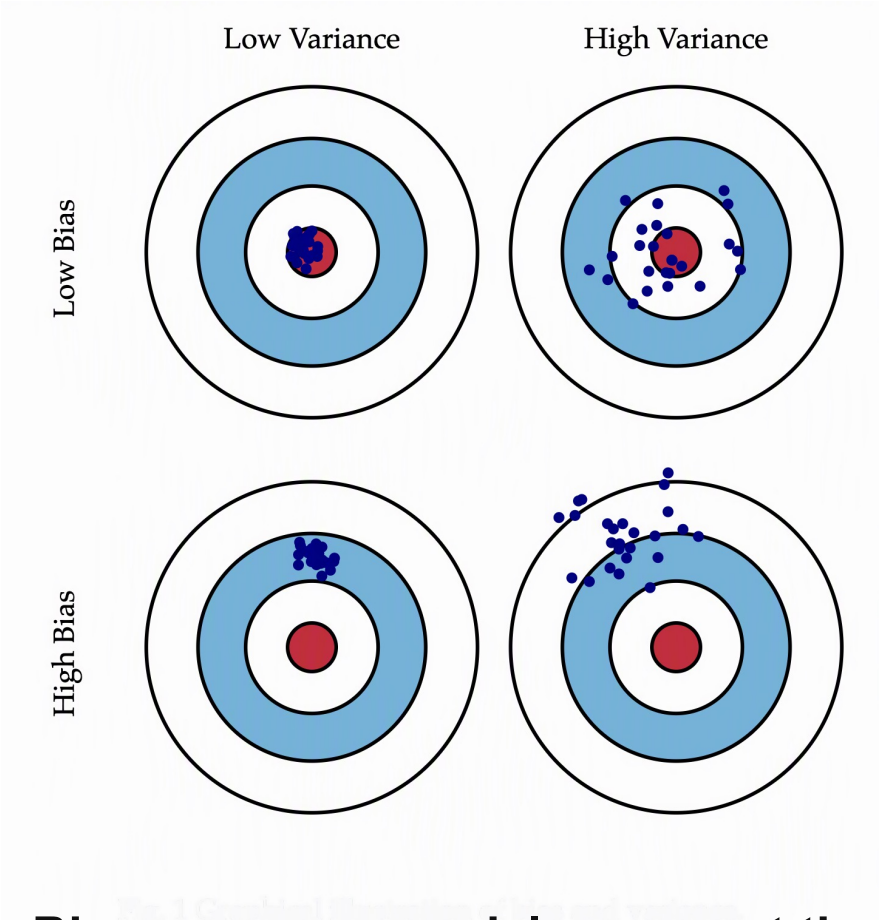
Reading: Bagging and random forests

K. Murphy (2022), §§18.3–18.4 Bagging and random forests

Model complexity and generalization error



Bias and variance - intuition



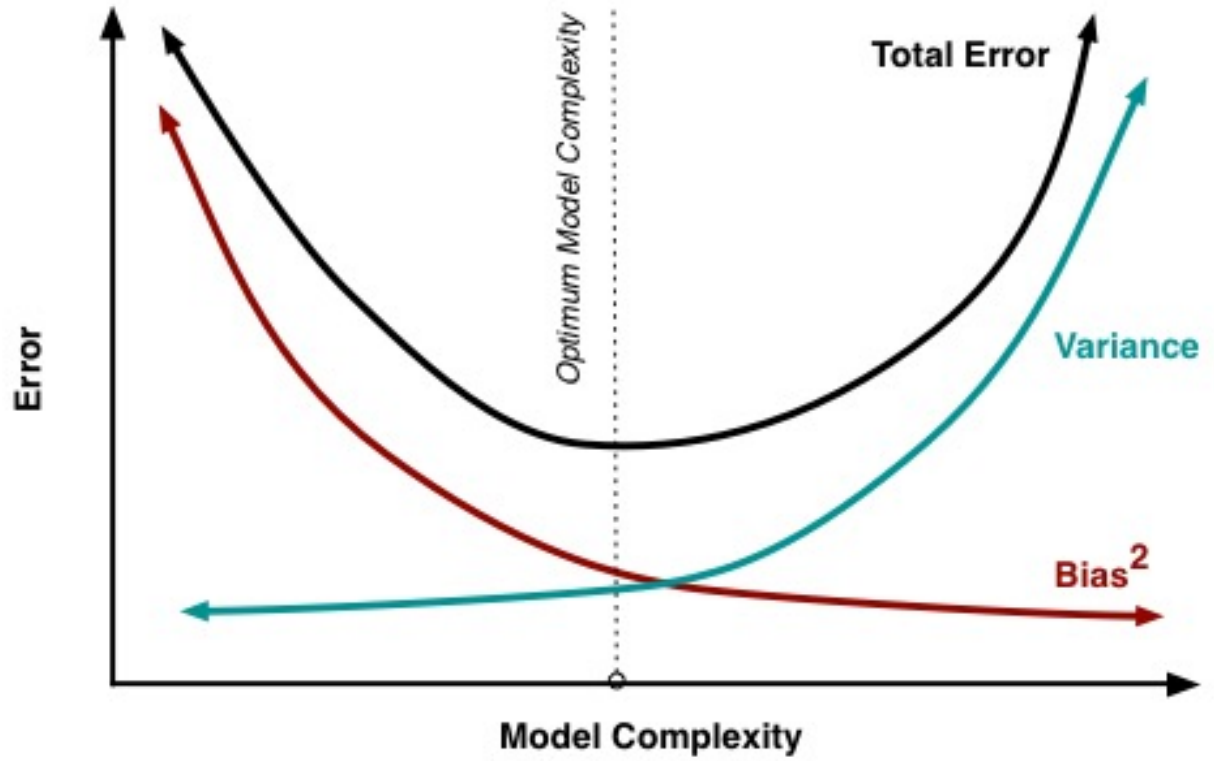
Bias: can your model represent the underlying function?

Variance: will your learned solution depend on the particular training set? (e.g. influence of noise)

https://scott.fortmann-roe.com/docs/BiasVariance.html

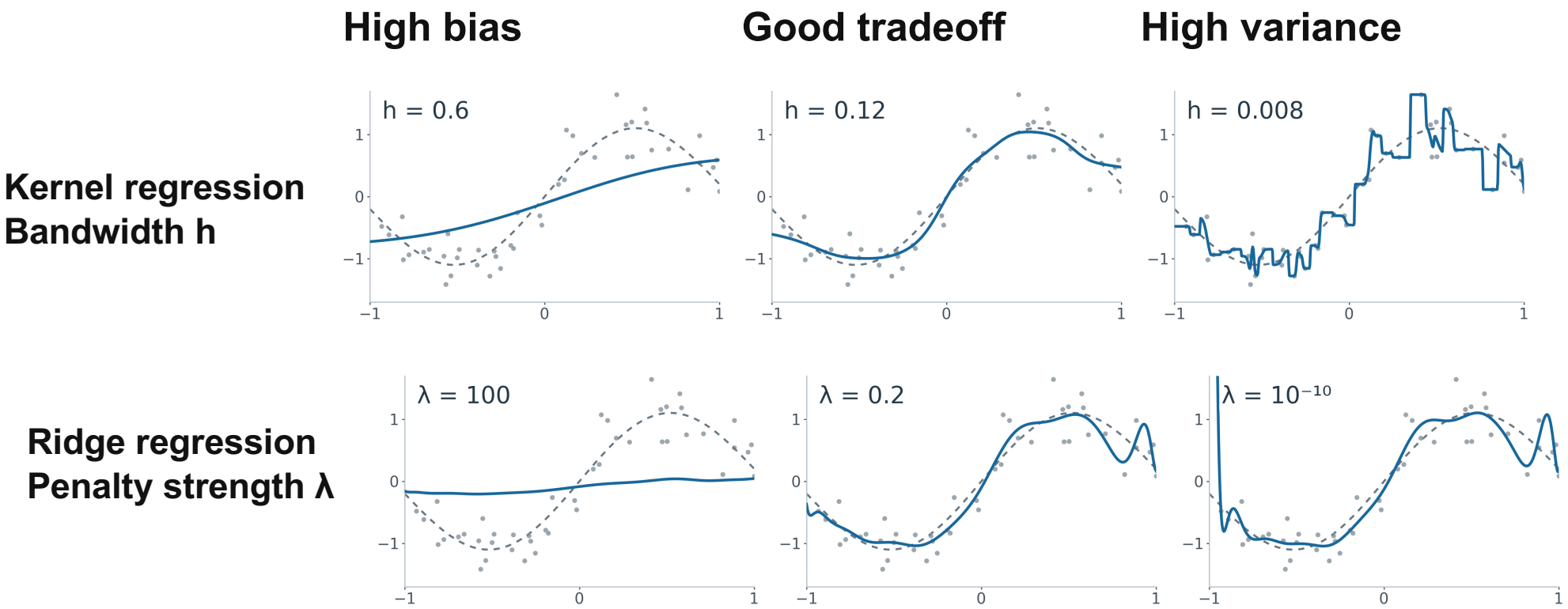
Bias-variance decomposition

$$\mathbb{E}_{\mathcal{D}}[R(\hat{f}_{\mathcal{D}})] = \mathbb{E}_{\mathbf{X}}[\text{bias}^2(\mathbf{X}) + \text{variance}(\mathbf{X}) + \text{noise}(\mathbf{X})]$$



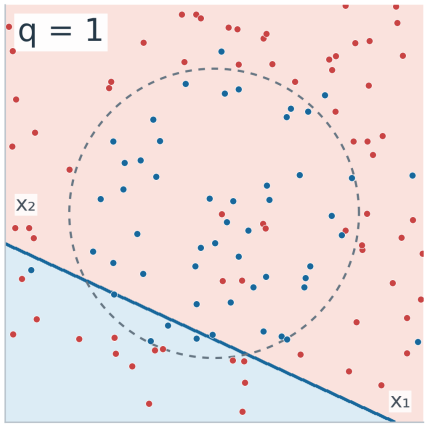
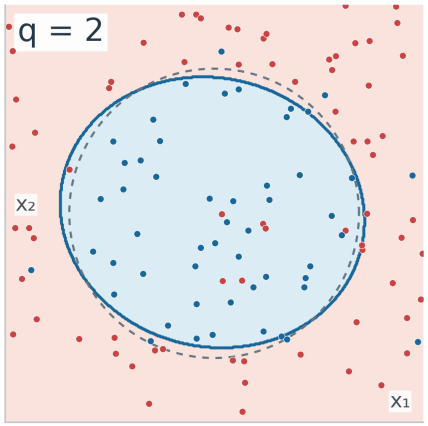
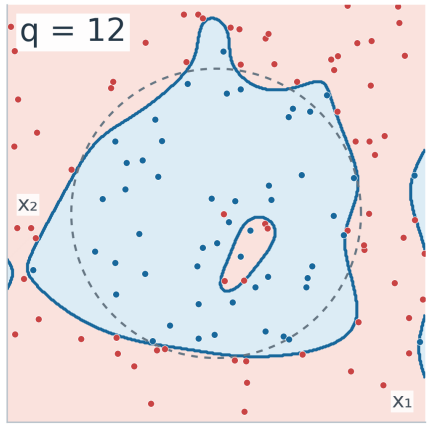
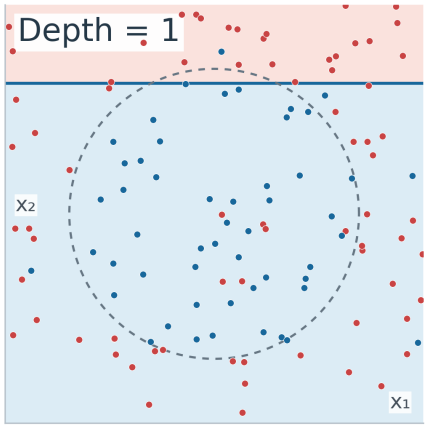
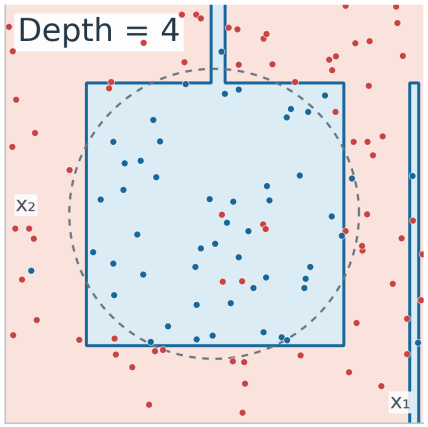
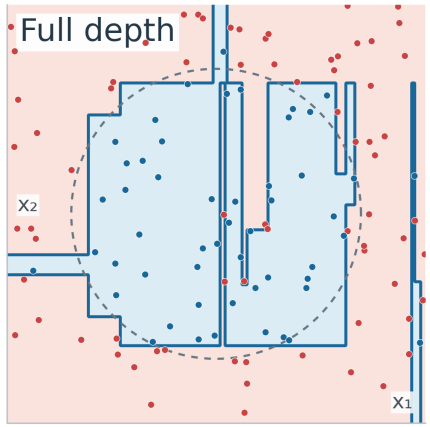
Schematic; noise is not shown.

Bias and variance: regression examples



Dots: training data. Dashed: true model. Blue: fitted predictor.

Bias and variance: classification examples

	High bias	Good tradeoff	High variance
Lecture 2 **Logistic regression** **Feature degree q**			
Lecture 3 **Decision trees** **Maximum depth**			

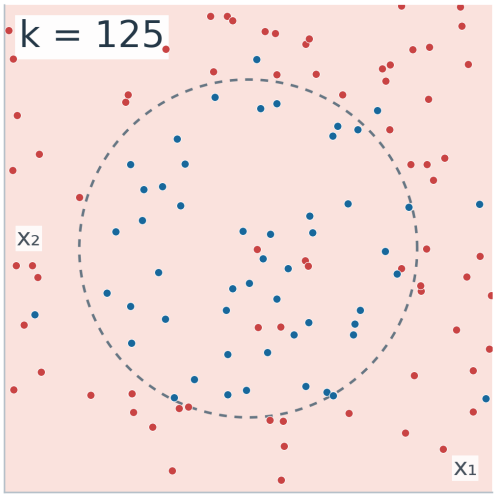
Colour: predicted class. Dots: training labels. Dashed: true boundary.

Illustrative fits · Increasing complexity can fit training noise

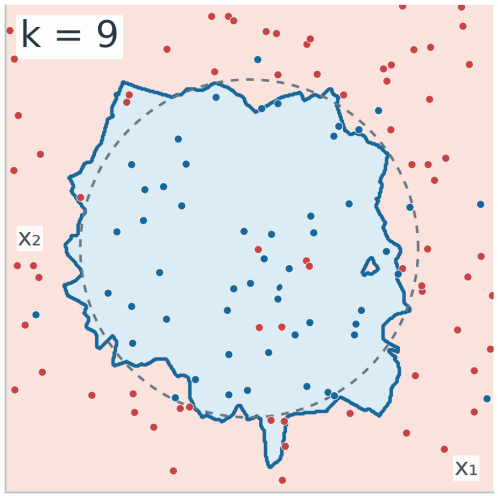
Bias and variance: kNN classification

The same training data, with different neighbourhood sizes.

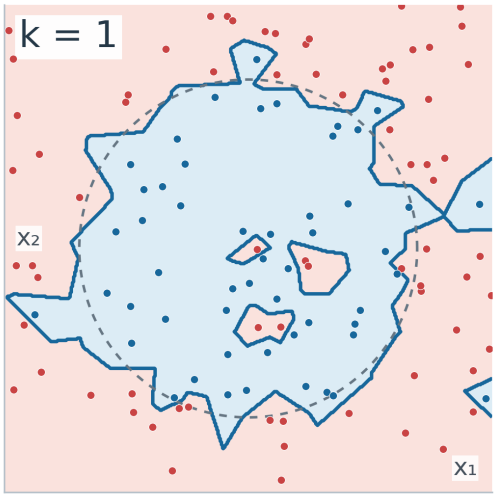
High bias



Good tradeoff



High variance

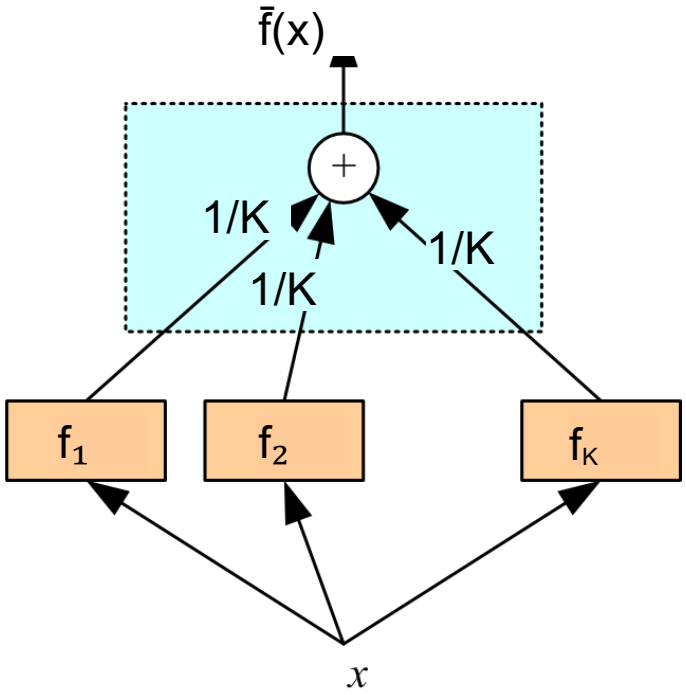


A large k smooths the boundary. A small k can fit individual labels.

Choose k using validation data.

Colour: prediction. Dots: training labels. Dashed: true boundary.

A committee can reduce variance



At input x, average K learners.
$f_k(x)$ is learner k's prediction.

$$\bar{f}(x) = \frac{1}{K}\sum_{k=1}^{K} f_k(x)$$

Averaging models with the same expected prediction leaves their bias unchanged.

For uncorrelated fluctuations with equal variance:

$$\mathrm{Var}[\bar{f}(x)] = \frac{\mathrm{Var}[f_k(x)]}{K}$$

Positive correlation reduces the gain. We need diverse learners.

Error of an average predictor

K predictors $f_k(x)$ form a committee.
$f^*(x)$ is the true function and $\varepsilon_k(x)$ the error of predictor k.

$$\overline{f}(\mathbf{x}) = \frac{1}{K}\sum_{k=1}^{K} f_k(\mathbf{x}), \qquad f_k(\mathbf{x}) = f^*(\mathbf{x}) + \epsilon_k(\mathbf{x})$$

$$E_k = \mathbb{E}_{\mathbf{x}}[\epsilon_k(\mathbf{x})^2], \qquad E_{\mathrm{AV}} = \frac{1}{K}\sum_{k=1}^{K} E_k$$

E_k is expected squared error. E_{AV} averages it over the K predictors.

$$E_{\mathrm{COM}} = \mathbb{E}_{\mathbf{x}}[(\overline{f}(\mathbf{x}) - f^*(\mathbf{x}))^2]$$

$$= \mathbb{E}_{\mathbf{x}}\left[\left(\tfrac{1}{K}\sum_{k=1}^{K}\left(f_k(\mathbf{x}) - f^*(\mathbf{x})\right)\right)^2\right]$$

$$= \mathbb{E}_{\mathbf{x}}\left[\left(\tfrac{1}{K}\sum_{k=1}^{K}\epsilon_k(\mathbf{x})\right)^2\right]$$

Uncorrelated errors can average out

Assume zero-mean errors, uncorrelated across different predictors.

$$\mathbb{E}_{\mathbf{x}}[\epsilon_k(\mathbf{x})\epsilon_{k'}(\mathbf{x})] = 0 \quad \text{for } k \neq k'$$

Expand the square. Cross terms vanish for k ≠ k′.

$$E_{\mathrm{COM}} = \frac{1}{K^2}\left(\sum_k \mathbb{E}_{\mathbf{x}}[\epsilon_k^2] + \sum_k \sum_{k' \neq k} \mathbb{E}_{\mathbf{x}}[\epsilon_k \epsilon_{k'}]\right)$$

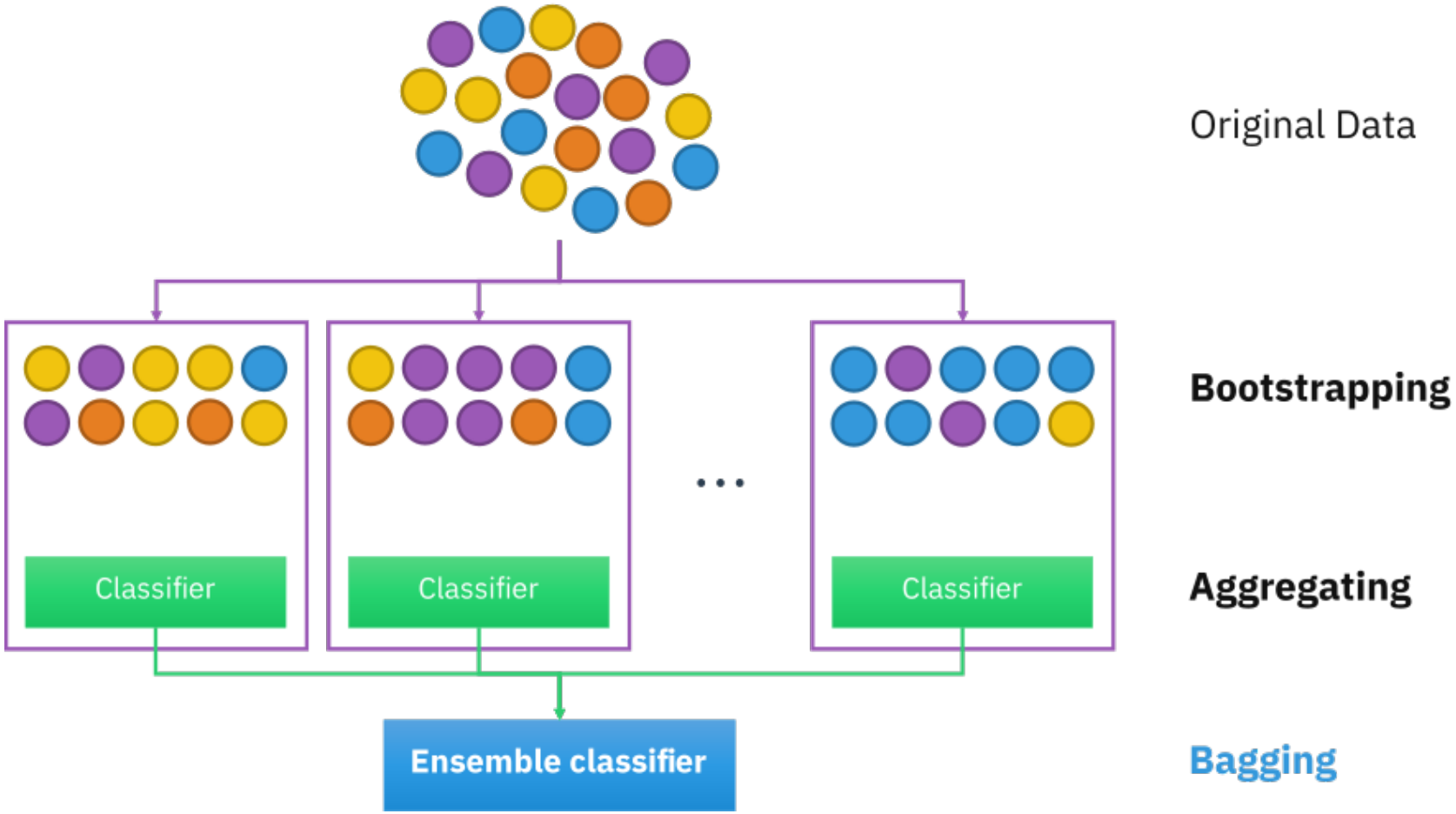
$$= \frac{1}{K^2}\sum_k E_k = \frac{1}{K} E_{\mathrm{AV}}$$

Averaging reduces independent variation. Shared bias remains.

Bagging uses bootstrap samples

Draw N training examples with replacement for each resample.

Fit one base learner per resample, then combine votes or probabilities.



A bootstrap sample

Six training examples:

1 2 3 4 5 6

Six draws with replacement:

1 1 2 4 4 6 1 and 4 repeat. 3 and 5 are omitted.

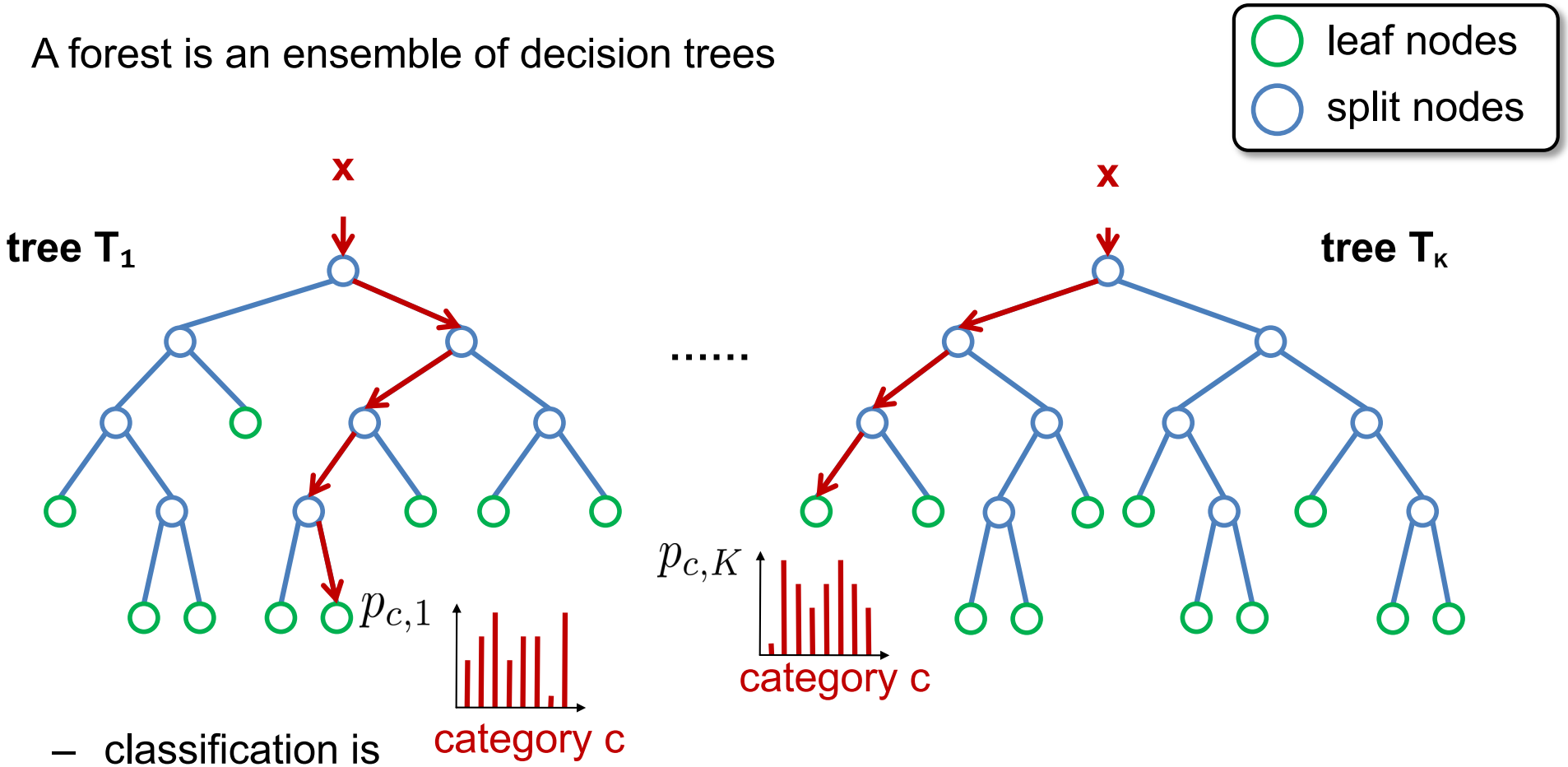
Out-of-bag examples: 3 and 5.
Evaluate each using only models that omitted it during fitting.

Resamples share observations, so their model errors can be correlated.

Random forests also vary the candidate features to reduce correlation.

A forest combines trees

- A forest is an ensemble of decision trees



  - classification is

$$p_c(\mathbf{x}) = \frac{1}{K}\sum_{k=1}^{K} p_{c,k}(\mathbf{x})$$

Training a random forest

- Draw a bootstrap sample to train each tree
- Train each tree:
  - At each node, search a random subset of measured features.

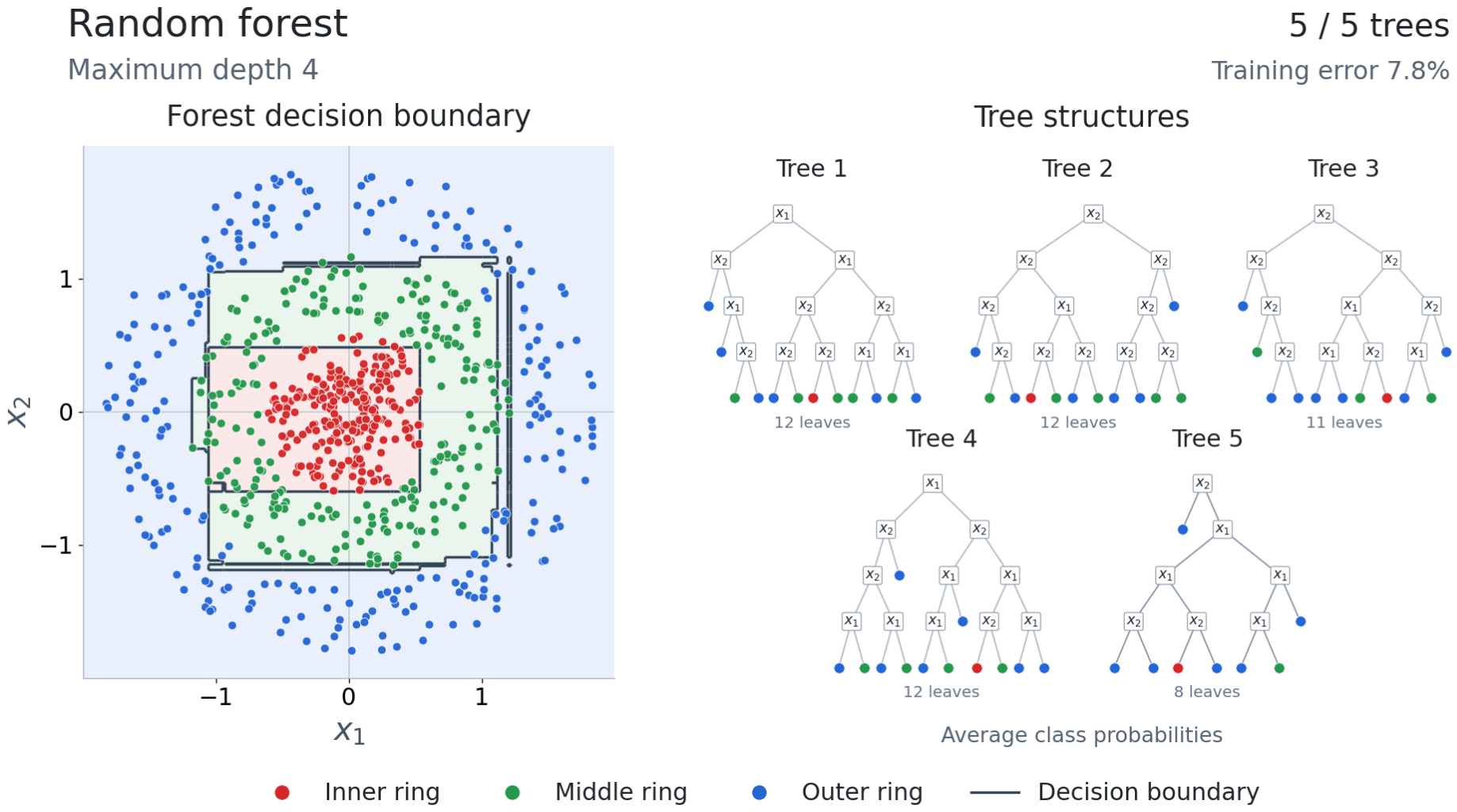
$$\mathcal{F} = \{h_j(\mathbf{x}) = x_j : j = 1, \ldots, d\}, \qquad \mathcal{F}_n \subset \mathcal{F}, \quad |\mathcal{F}_n| = m < d$$

  - Choose the best split within that subset, then recurse.

$$(h_n^*, \theta_n^*) = \underset{h \in \mathcal{F}_n,\, \theta}{\arg\max}\, \mathrm{IG}(I_n; h, \theta)$$

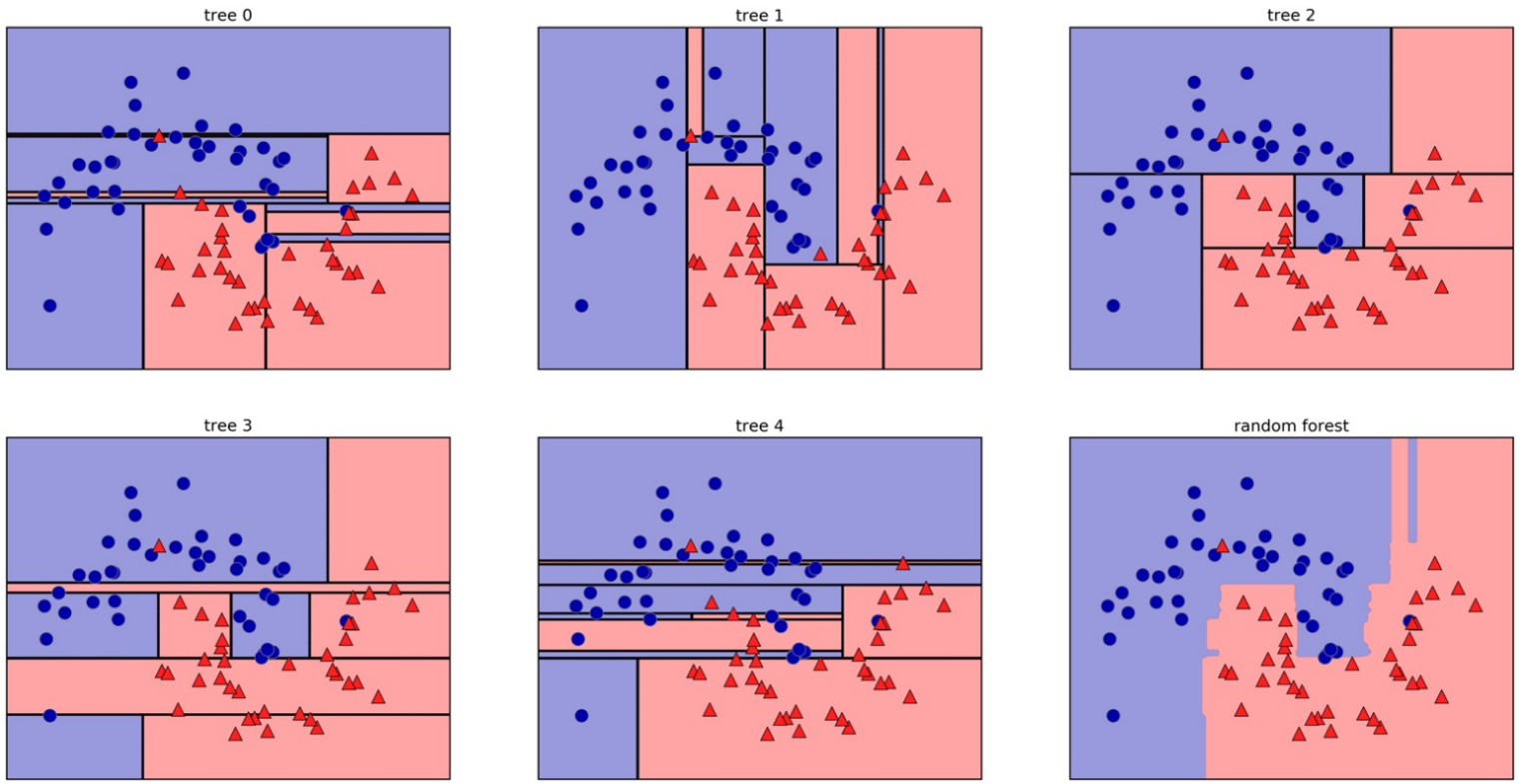
- Feature variation reduces correlation between tree predictions.
- Combine probabilities or votes.
- Choose complexity using validation or out-of-bag predictions.

A forest composed of five trees



Five trees, maximum depth 4. Average their class probabilities.

A forest composed of 5 trees



Limits of greedy feature selection

Extremely flexible classifiers (power of composition)

Low-cost (simple comparisons, no kernel evaluations)

But: learned greedily

Later splits do not jointly retune the earlier tests.

An alternative: use a vast feature space through a kernel.

Kernels and SVMs

Decision trees

Bagging and random forests

Kernels and SVMs

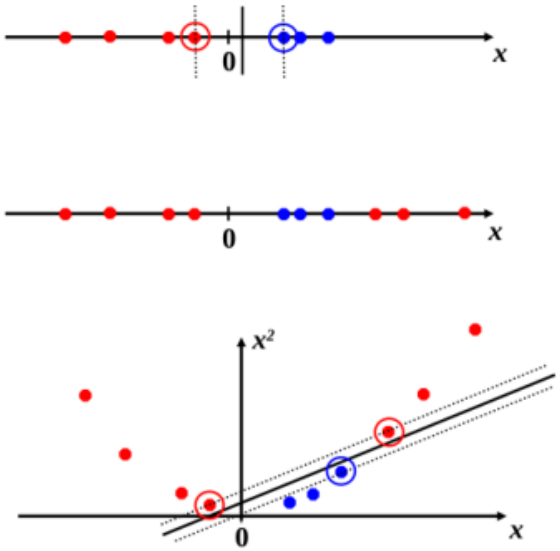
Neural networks

Expressiveness and representations

Backpropagation

1995 Support vector machines

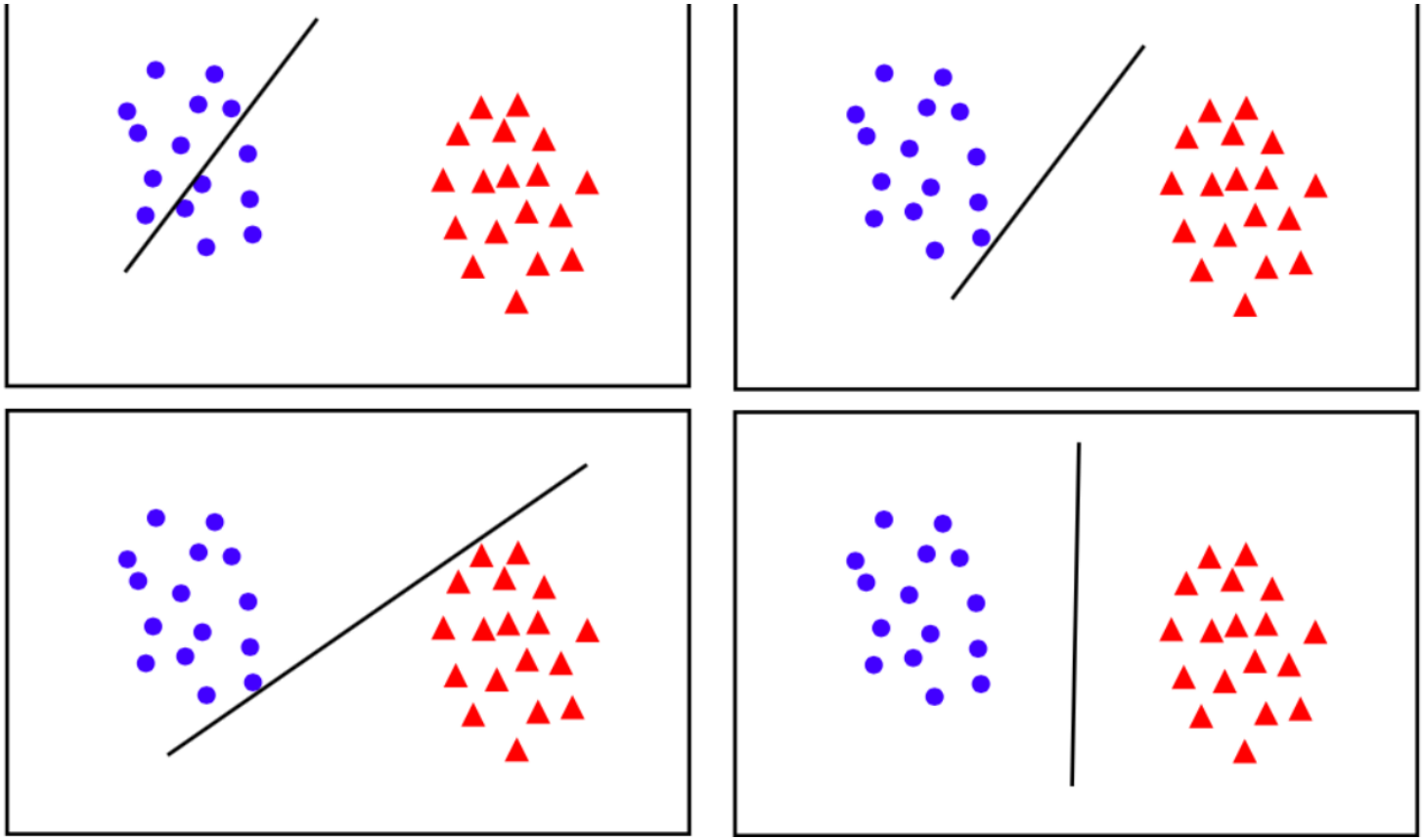
Cortes and Vapnik



Reading: Kernels and SVMs

K. Murphy (2022), §§17.1, 17.3 Kernels and support vector machines

Choosing a separating hyperplane



Which boundary has greatest clearance from the training data?

A support vector machine (SVM) chooses a large-margin separator.
First use the input space; then introduce a kernel.

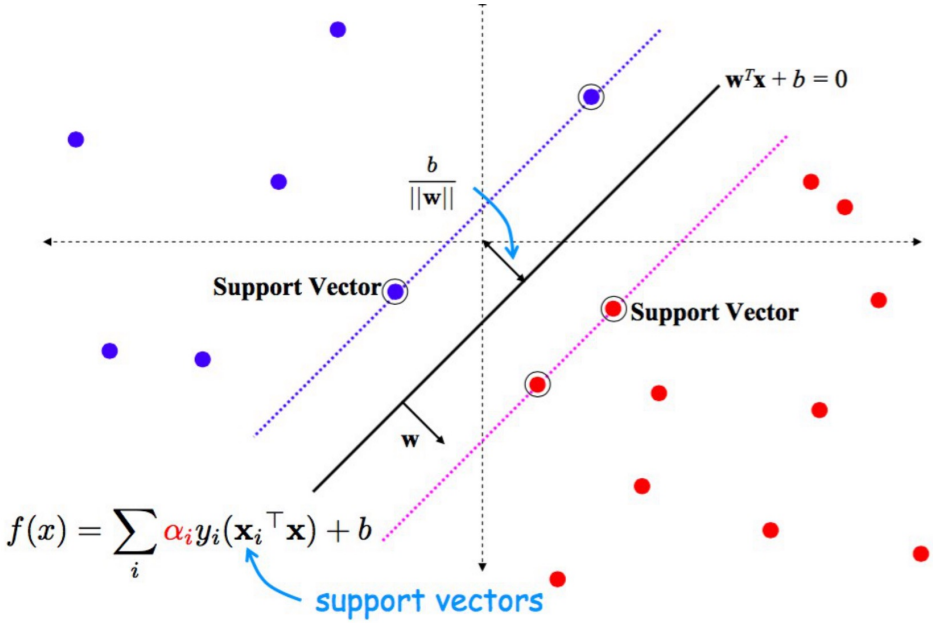
Support vectors determine the classifier

$$\mathbf{w} = \sum_{i=1}^{N} \alpha_i y_i \mathbf{x}_i$$

$$y_i \in \{-1, +1\}$$

$\alpha_i \geq 0$ is learned.
Only $\alpha_i > 0$ contribute:
the **support vectors**.

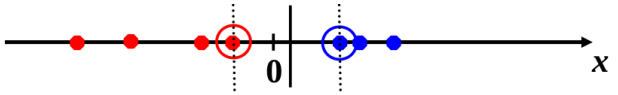
$$\hat{y} = \text{sign}(\mathbf{w}^T \mathbf{x} + b)$$



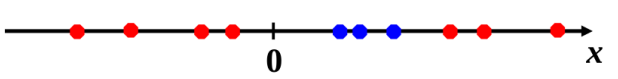
Optimisation chooses the signed coefficients $\alpha_i y_i$ and the bias b.
We will not derive the optimisation problem in this course.

Nonlinear separation through features

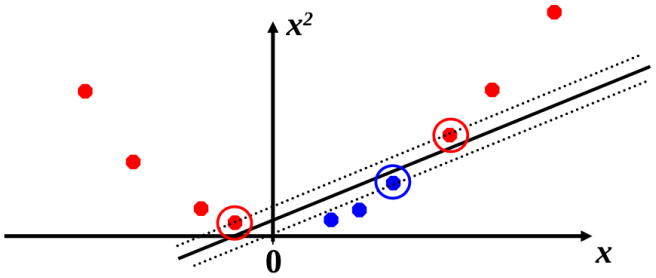
A linear boundary in the original input may be insufficient.



$$\mathbf{x} \mapsto \phi(\mathbf{x})$$



A linear classifier in the feature space can be nonlinear in the original input.



A classifier as a kernel expansion

The learned weight vector is a combination of support-vector features.

$$\mathbf{w} = \sum_{i \in \mathrm{SV}} \alpha_i y_i \, \boldsymbol{\phi}(\mathbf{x}_i)$$

$$f(\mathbf{x}; \mathbf{w}) = \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}) + b$$

$$= \sum_{i \in \mathrm{SV}} \alpha_i y_i \underbrace{\boldsymbol{\phi}(\mathbf{x}_i)^T \boldsymbol{\phi}(\mathbf{x})}_{k(\mathbf{x}_i, \mathbf{x})} + b$$

SV: support vectors. Labels $y_i = \pm 1$. Coefficients $\alpha_i \geq 0$ are learned.

Prediction only needs kernel evaluations at the support vectors.

Polynomial kernels and explicit features

$$k(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^T \mathbf{z})^3$$

For two input coordinates, a cubic feature map has four terms:

$$\mathbf{x} \in \mathbb{R}^2 : \quad \phi(\mathbf{x}) = (x_1^3, \sqrt{3}x_1^2 x_2, \sqrt{3}x_1 x_2^2, x_2^3)^T$$

$$\phi(\mathbf{x})^T \phi(\mathbf{z}) = (x_1 z_1 + x_2 z_2)^3$$

In d dimensions, the number of cubic monomials grows rapidly:

$$\mathbf{x} \in \mathbb{R}^d : \qquad \dim \phi(\mathbf{x}) = \binom{d+2}{3} \quad (d = 100 : \ 171{,}700)$$

Explicit features: evaluate each input's monomials, then take their inner product.

Kernel: take one d-term inner product and cube the result.

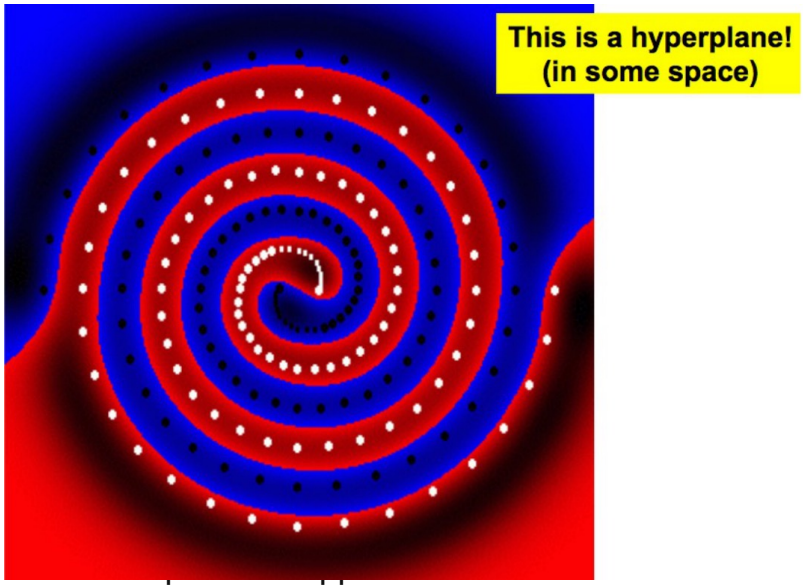
Gaussian kernels and flexible boundaries

As in kernel regression, a Gaussian gives nearby inputs more influence.

$$k(\mathbf{x}, \mathbf{z}) = \exp\left[-\frac{\|\mathbf{x} - \mathbf{z}\|^2}{2\sigma^2}\right], \qquad \sigma > 0$$

The SVM combines Gaussians centred on its support vectors.

The feature space is infinite-dimensional (see appendix), but each kernel evaluation is finite.



A linear decision in feature space can produce a complicated boundary in input space.

Learning the features and the predictor

A rich feature space can make a simple final decision powerful.

Trees and forests

Select paths of tests from many possible combinations.
Choose splits greedily, without jointly retuning earlier tests.

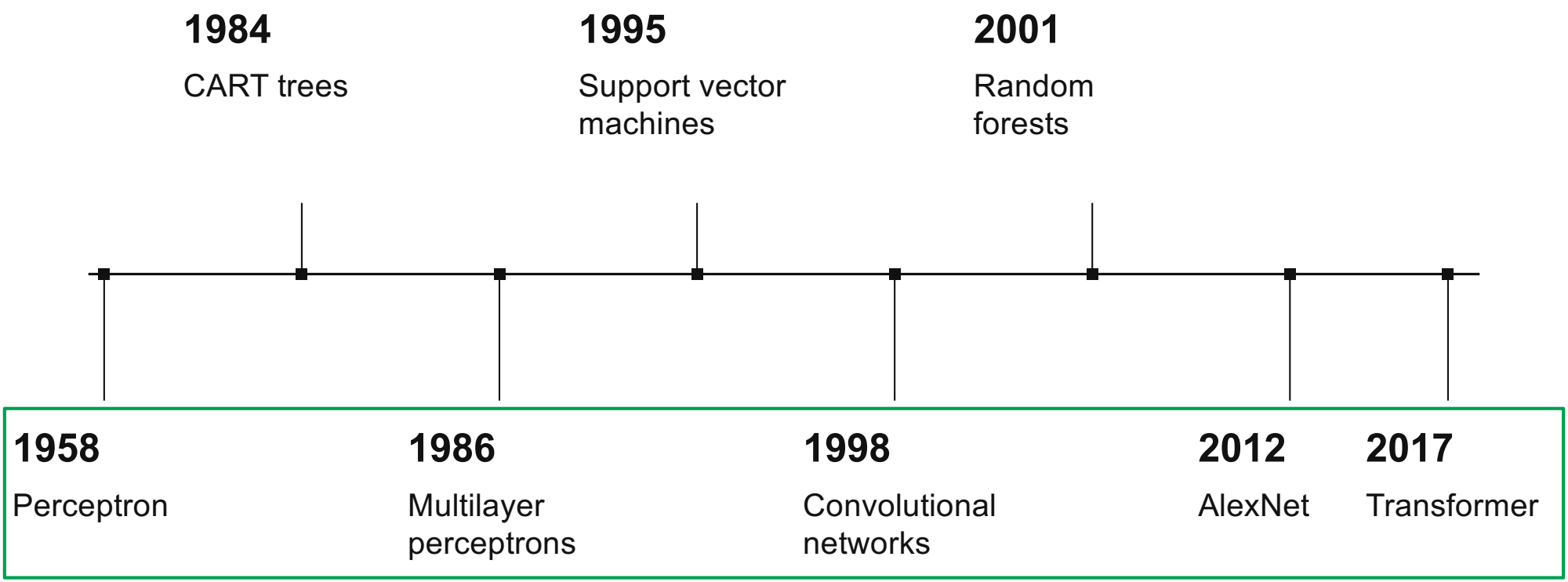
Fixed-kernel methods

Use implicit features through kernel evaluations.
Fit classifier coefficients while the chosen feature map stays fixed.

Neural networks

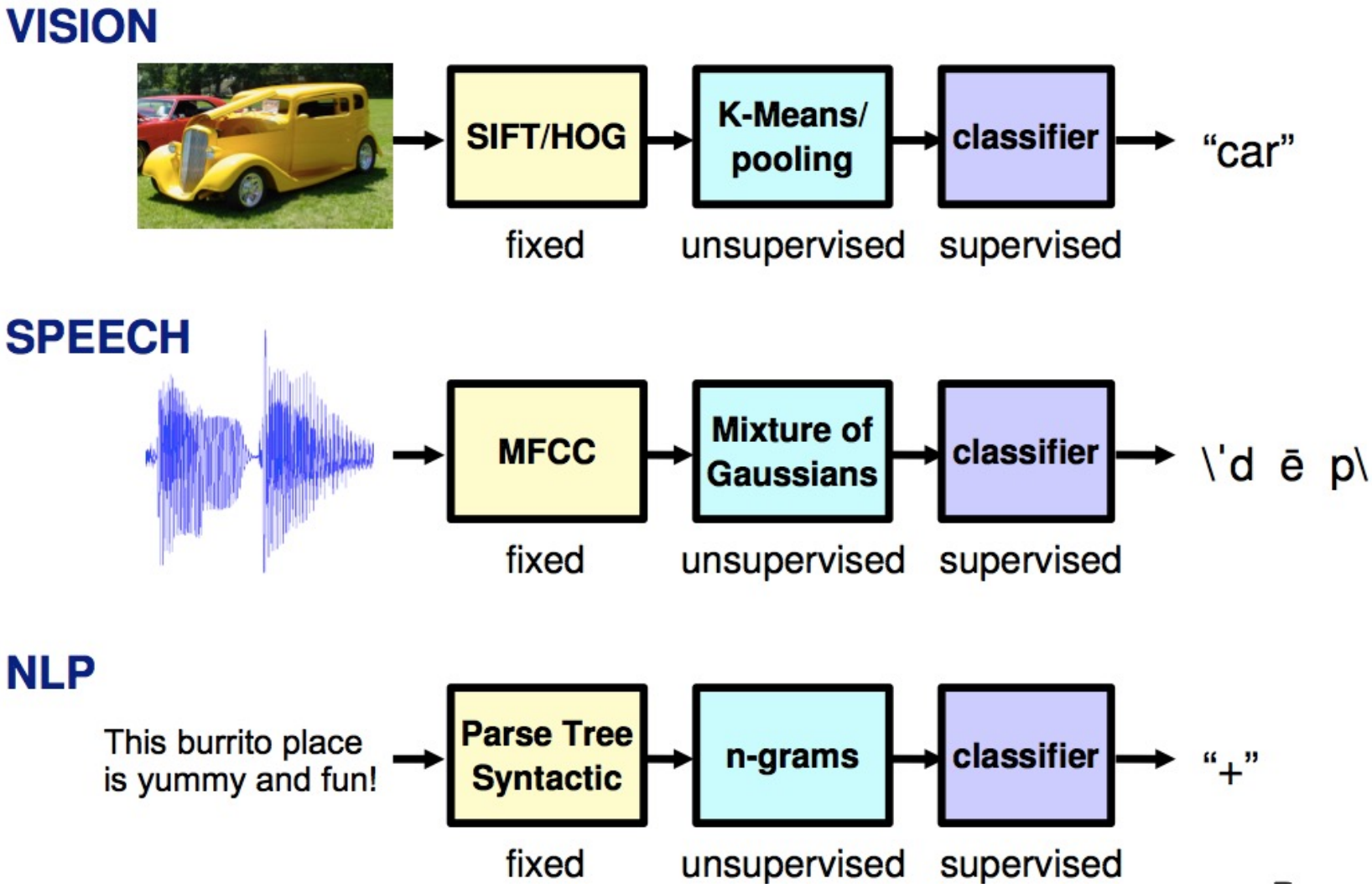
Optimise intermediate features and the final predictor together, using the final prediction loss.

Selected milestones in machine learning



Deep Learning

Machine Learning pre Deep Learning



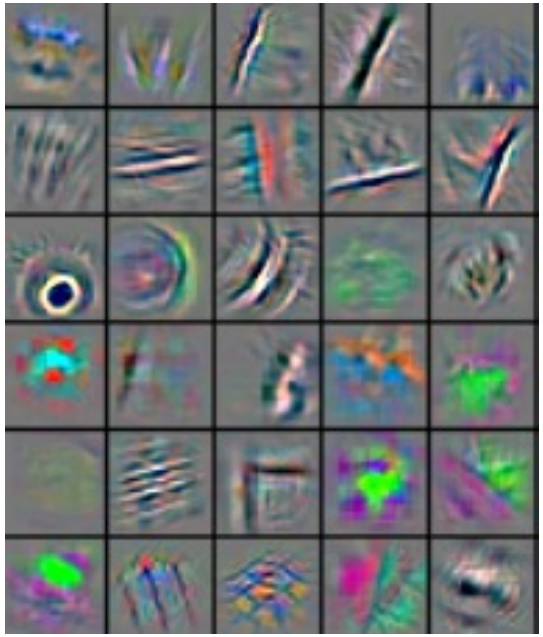
Slide Credit: Marc'Aurelio Ranzato, Yann LeCun

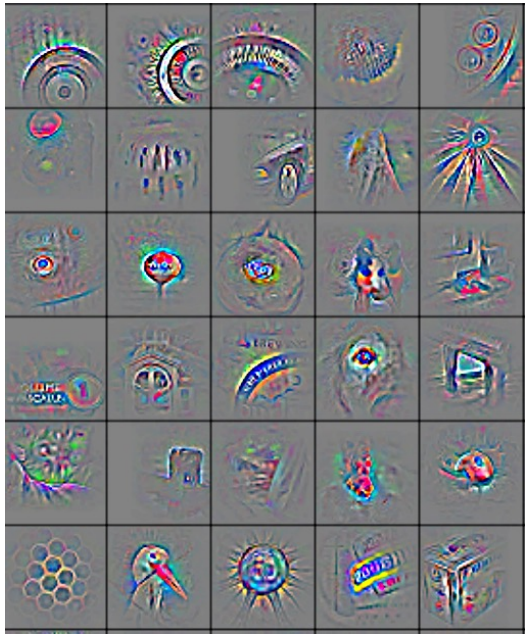
Deep Learning = learning feature compositions



Low-Level Feature → Mid-Level Feature → High-Level Feature → Trainable Classifier → “car”



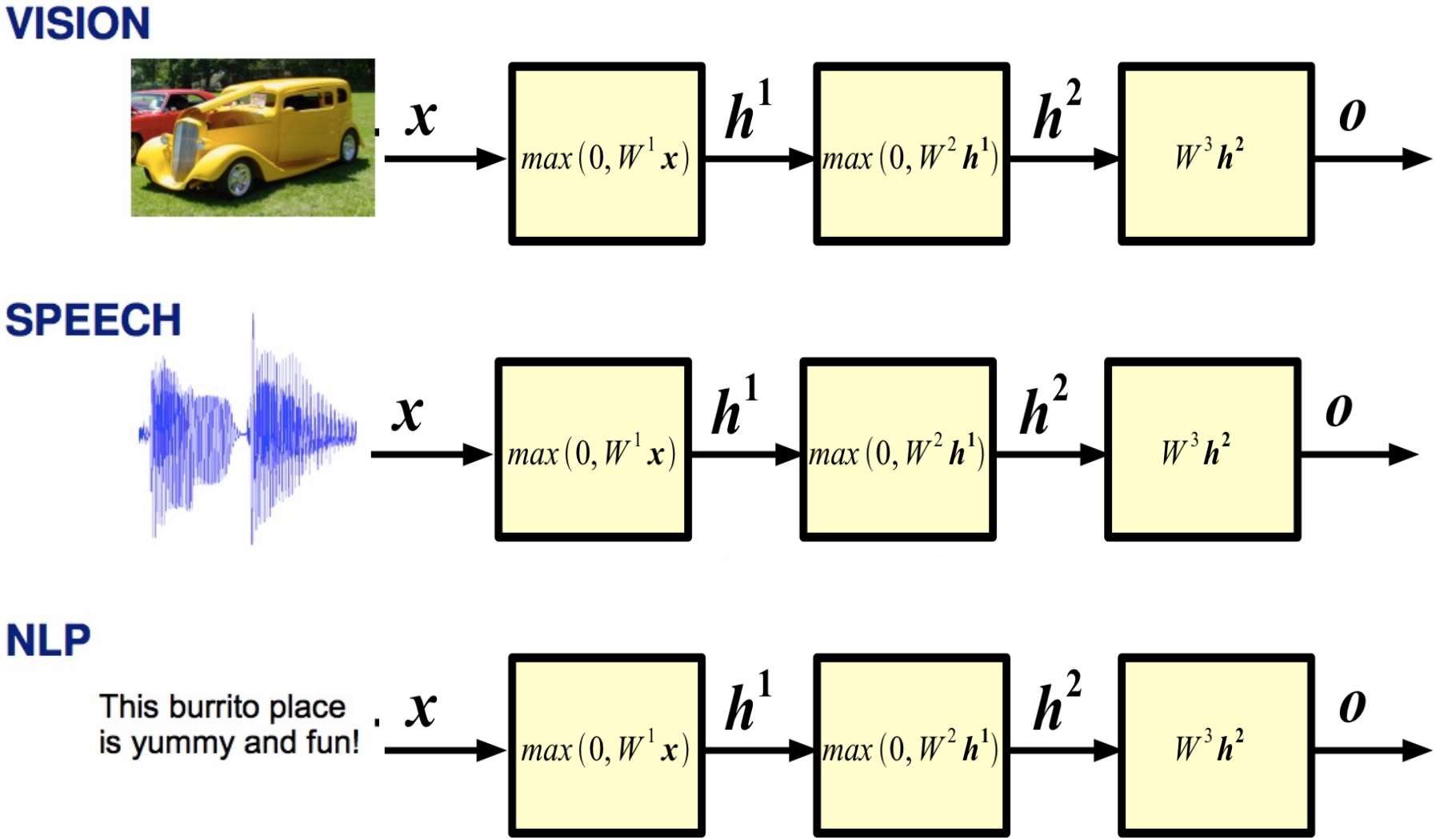




Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

Slide Credit: Marc'Aurelio Ranzato, Yann LeCun

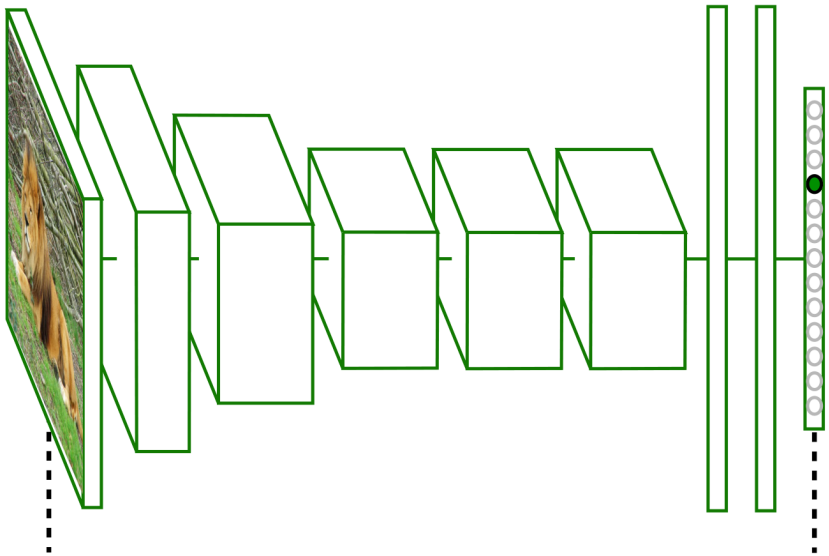
Deep Learning: learn the features!



Slide Credit: Marc'Aurelio Ranzato, Yann LeCun

2012: AlexNet

AlexNet used a deep convolutional network for ImageNet classification.



ImageNet top-5 error

AlexNet: 16.4%
Best shallow competitor: 26%

Next week: convolutional networks and ImageNet

Krizhevsky, Sutskever and Hinton (2012). Comparison reused from Lecture 4.

Course schedule

- 1st lecture: Intro & Regression
- 2nd lecture: Classification
- 3rd lecture: Neural Networks
- 4th lecture: Convolutional Neural Networks
- 5th lecture: Transformers
- 6th lecture: Unsupervised learning
- 7th lecture: Generative modelling
- 8th lecture: Revision & seminar

Lectures 1–2: supervised problems. Lectures 3–5: deep learning.
Buildup to unsupervised learning in 6 and generative models in 7.

Neural networks

Decision trees

Bagging and random forests

Kernels and SVMs

Neural networks

Expressiveness and representations

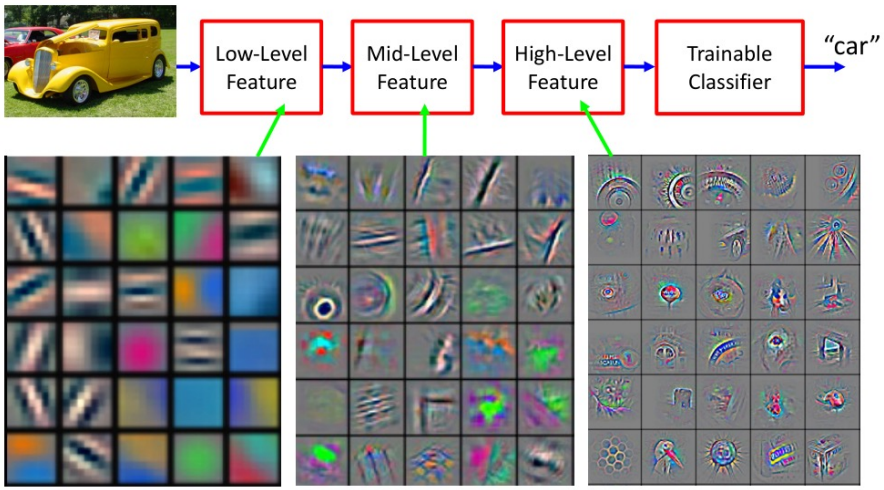
Backpropagation

1958 Perceptron

Rosenblatt

1986 Multilayer perceptrons

Rumelhart, Hinton and Williams



Ranzato and LeCun; Zeiler and Fergus (2013)

Reading: Multilayer perceptrons

K. Murphy (2022), §§10.2.5, 13.2

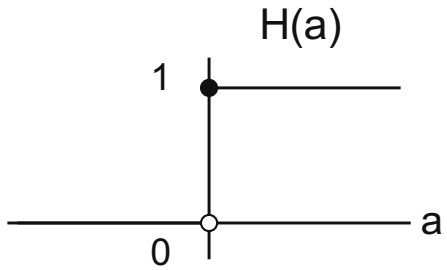
The perceptron: a hard decision

Lecture 2: logistic regression outputs a probability

$$p(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b)$$

A perceptron uses a hard threshold (Heaviside unit)

$$\hat{y} = H(\mathbf{w}^T \mathbf{x} + b)$$



For the same weights, threshold the probability at 0.5

$$\sigma(\mathbf{w}^T \mathbf{x} + b) \geq 0.5 \iff \mathbf{w}^T \mathbf{x} + b \geq 0$$

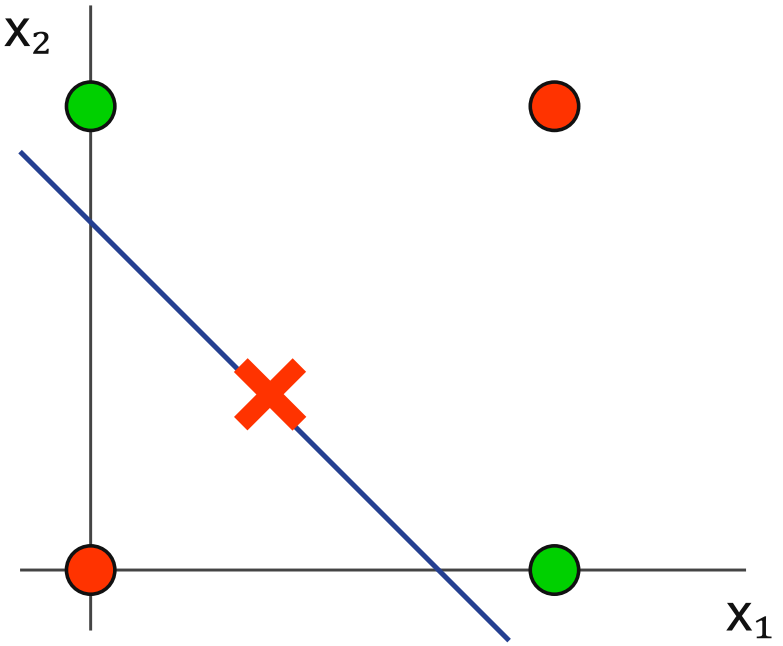
The perceptron is a deterministic classifier.
Its learning rule differs from logistic regression (not covered).

The XOR problem

XOR is 1 when the two binary inputs differ, and 0 otherwise.

$$\hat{y} = H(b + w_1 x_1 + w_2 x_2)$$

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0



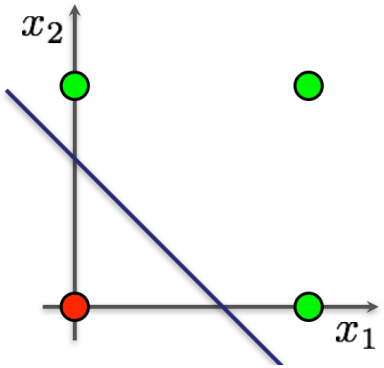
No straight line separates the two classes.
A single perceptron cannot represent XOR.

XOR: two useful features

Compute two useful features: OR and AND.

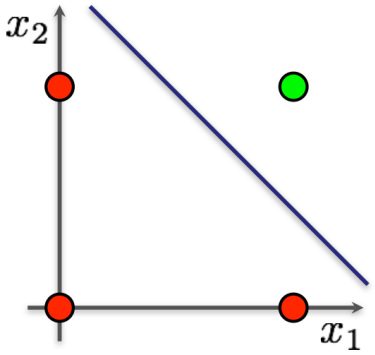
OR feature

x_1	x_2	z_1
0	0	0
0	1	1
1	0	1
1	1	1



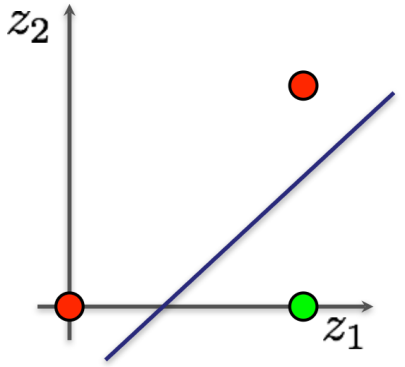
AND feature

x_1	x_2	z_2
0	0	0
0	1	0
1	0	0
1	1	1



OR, but not AND

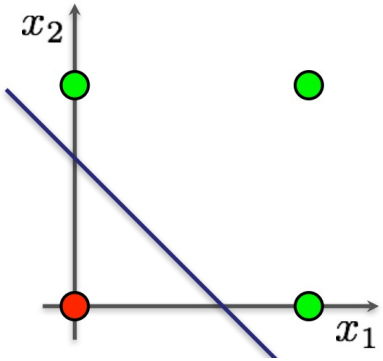
z_1	z_2	y
0	0	0
1	0	1
1	0	1
1	1	0

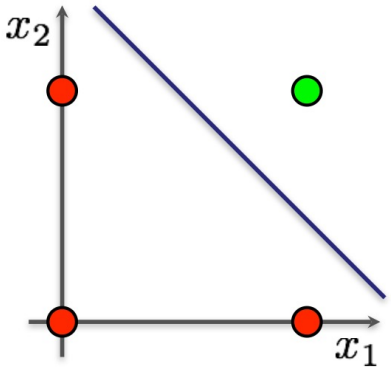


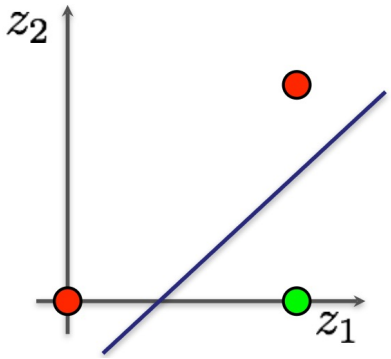
The two positive XOR inputs both become $(z_1, z_2) = (1, 0)$.

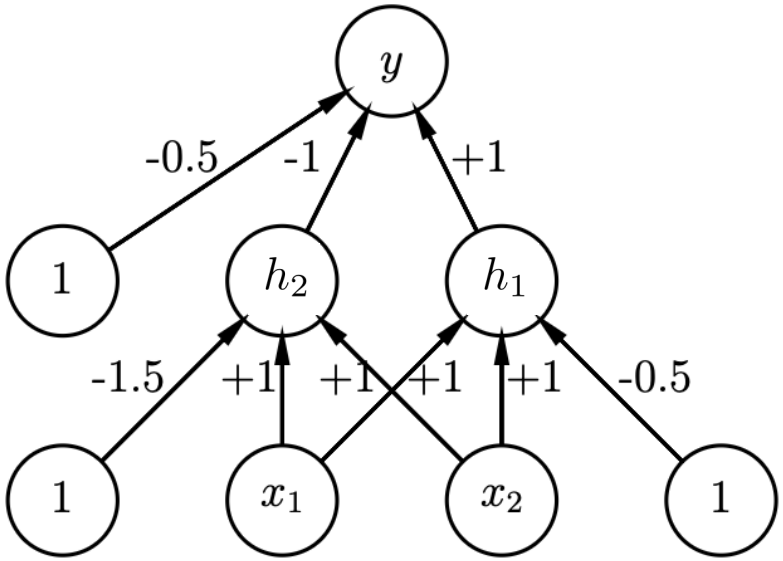
A hand-crafted network for XOR

$$z_1 = H(x_1 + x_2 - 0.5) \quad z_2 = H(x_1 + x_2 - 1.5) \quad y = H(z_1 - z_2 - 0.5)$$







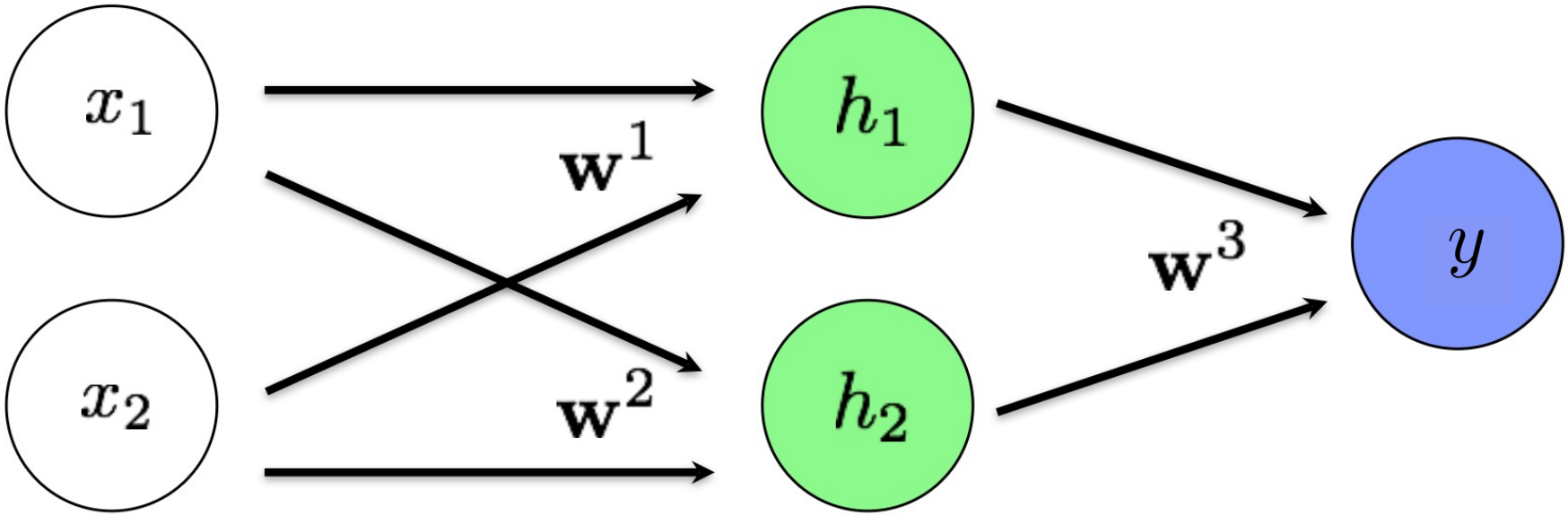


K. Murphy (2022), Fig. 13.1(b). $h_1 = z_1$ (OR), $h_2 = z_2$ (AND).

XOR: the composition is a network

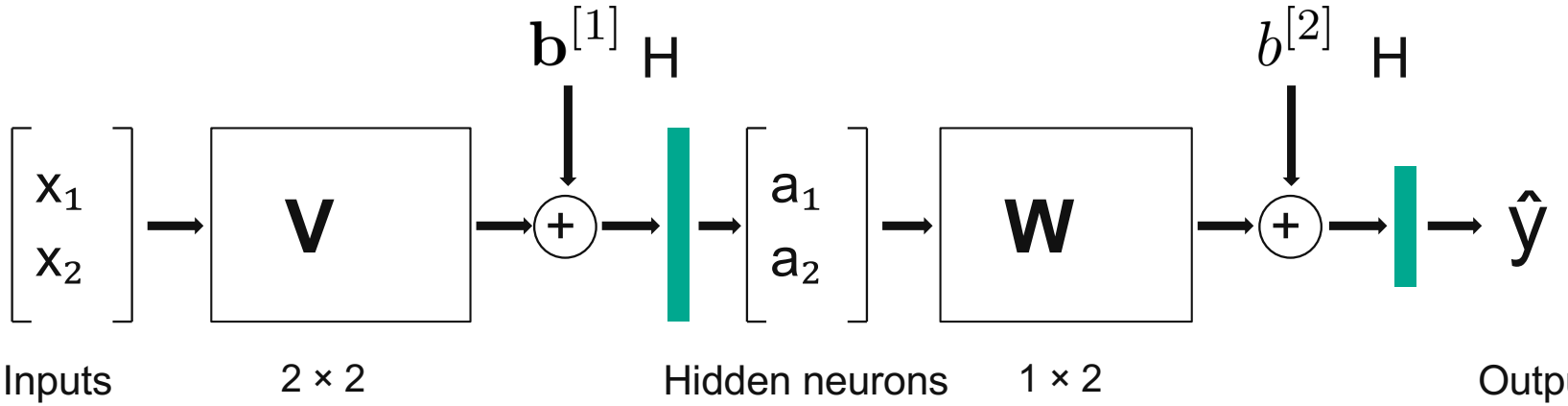
$$y = f\big(g_1(x_1, x_2), g_2(x_1, x_2)\big)$$

The hidden functions g_1 and g_2 compute OR and AND.
One output unit combines these two hidden activations.



Hidden unit 1: $h_1 = z_1$ (OR). Hidden unit 2: $h_2 = z_2$ (AND).
The output y combines these two features to compute XOR.

A two-layer network for XOR



$$a_1 = H(V_{11}x_1 + V_{12}x_2 + b_1^{[1]}) \qquad \hat{y} = H(W_{11}a_1 + W_{12}a_2 + b^{[2]})$$

$$a_2 = H(V_{21}x_1 + V_{22}x_2 + b_2^{[1]})$$

$$\mathbf{V} = \begin{bmatrix} V_{11} & V_{12} \\ V_{21} & V_{22} \end{bmatrix} \qquad \mathbf{b}^{[1]} = \begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \end{bmatrix} \qquad \mathbf{W} = \begin{bmatrix} W_{11} & W_{12} \end{bmatrix}, \qquad b^{[2]} \in \mathbb{R}$$

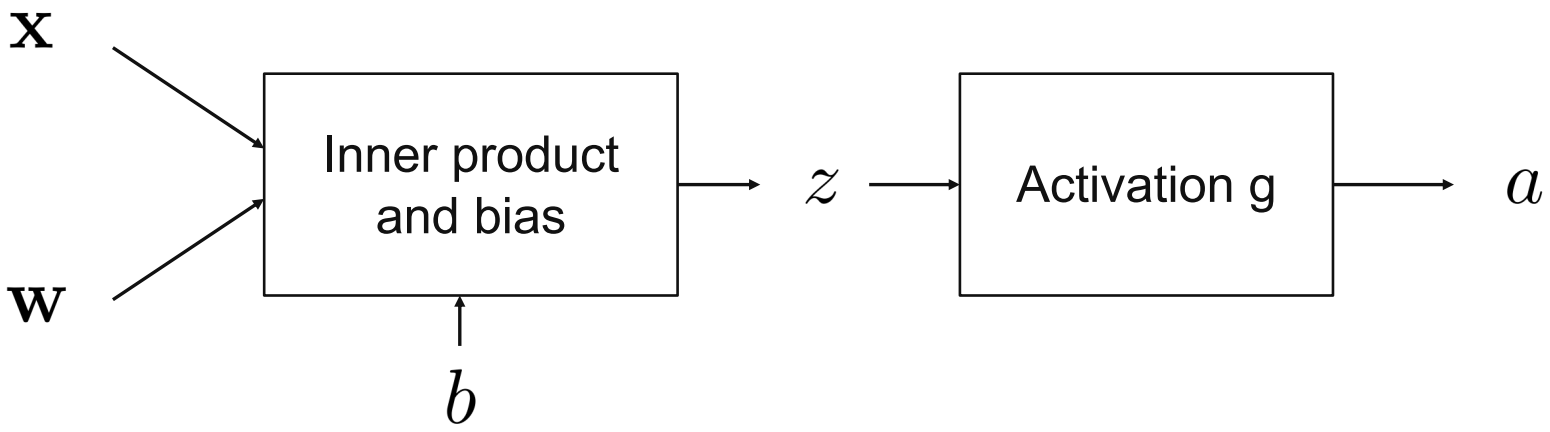
For compact notation, an affine map and activation form one layer.

$$\mathbf{a} = H(\mathbf{V}\mathbf{x} + \mathbf{b}^{[1]}), \qquad \hat{y} = H(\mathbf{W}\mathbf{a} + b^{[2]})$$

With the XOR weights: a_1 = OR, a_2 = AND, and $\hat{y}$ = XOR.

The basic computation unit: a neuron

$$\mathbf{x}, \mathbf{w} \in \mathbb{R}^d, \quad b \in \mathbb{R}$$



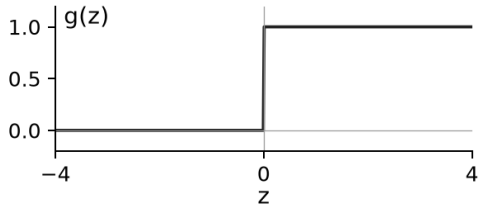
$$z = \mathbf{w}^T \mathbf{x} + b \in \mathbb{R} \qquad a = g(z)$$

A neuron takes a weighted sum of its inputs, adds a bias, then applies a scalar nonlinearity.

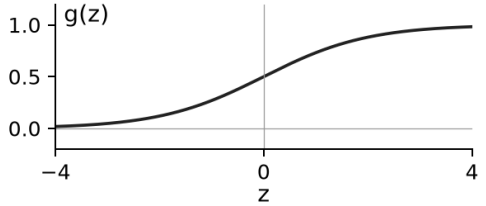
Activation functions

$$z = \mathbf{w}^T \mathbf{x} + b, \qquad a = g(z)$$

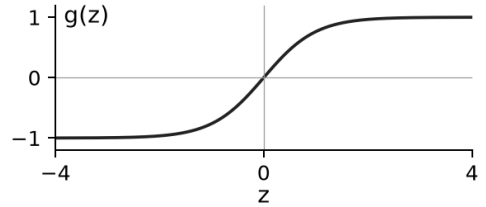
Step (perceptron) $$g(z) = \begin{cases} 0 & z < 0 \\ 1 & z \geq 0 \end{cases}$$



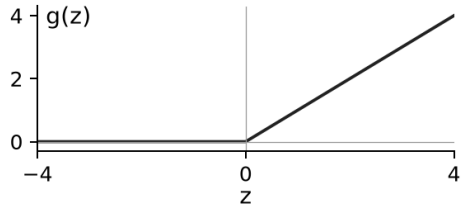
Sigmoid $$g(z) = \frac{1}{1 + \exp(-z)}$$



Tanh $$g(z) = \frac{\exp(z) - \exp(-z)}{\exp(z) + \exp(-z)}$$



ReLU $$g(z) = \max(0, z)$$



Why the nonlinearity matters

Two affine layers collapse into one affine map:

$$\mathbf{W}(\mathbf{V}\mathbf{x} + \mathbf{b}^{[1]}) + \mathbf{b}^{[2]}$$

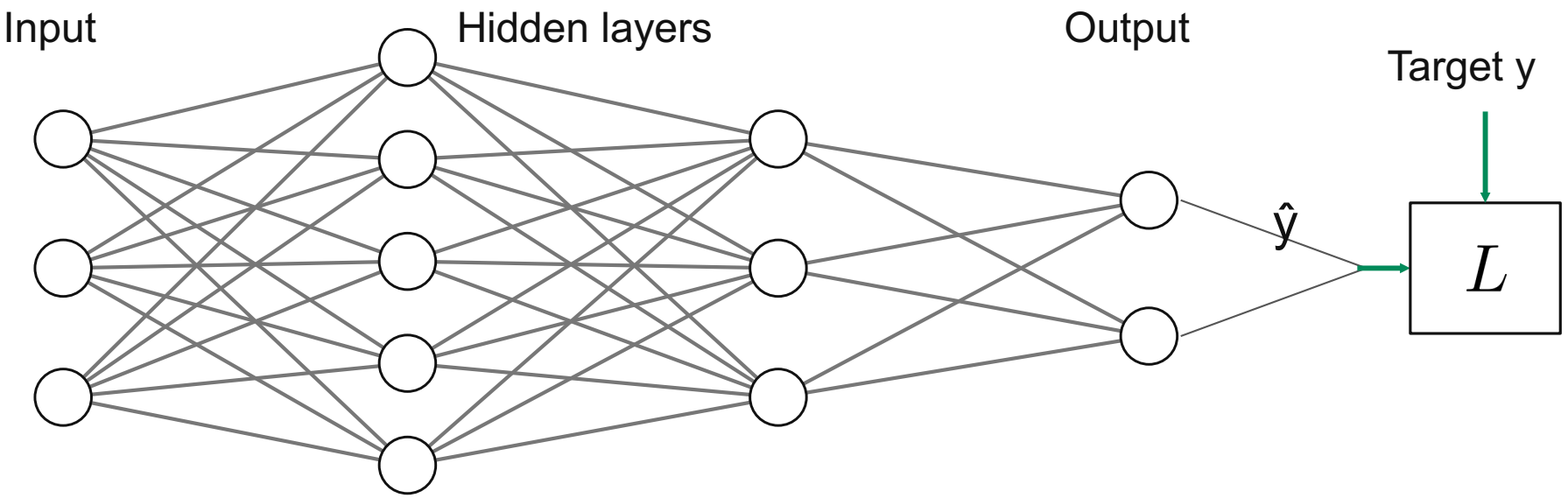
$$= (\mathbf{W}\mathbf{V})\mathbf{x} + (\mathbf{W}\mathbf{b}^{[1]} + \mathbf{b}^{[2]})$$

Here V and W are weights. The two bias terms are shown explicitly; x is the input.

Nonlinear activations make composition more expressive.

Gradients throughout the network

Learn every weight and bias by differentiating the scalar loss L.



Backpropagation

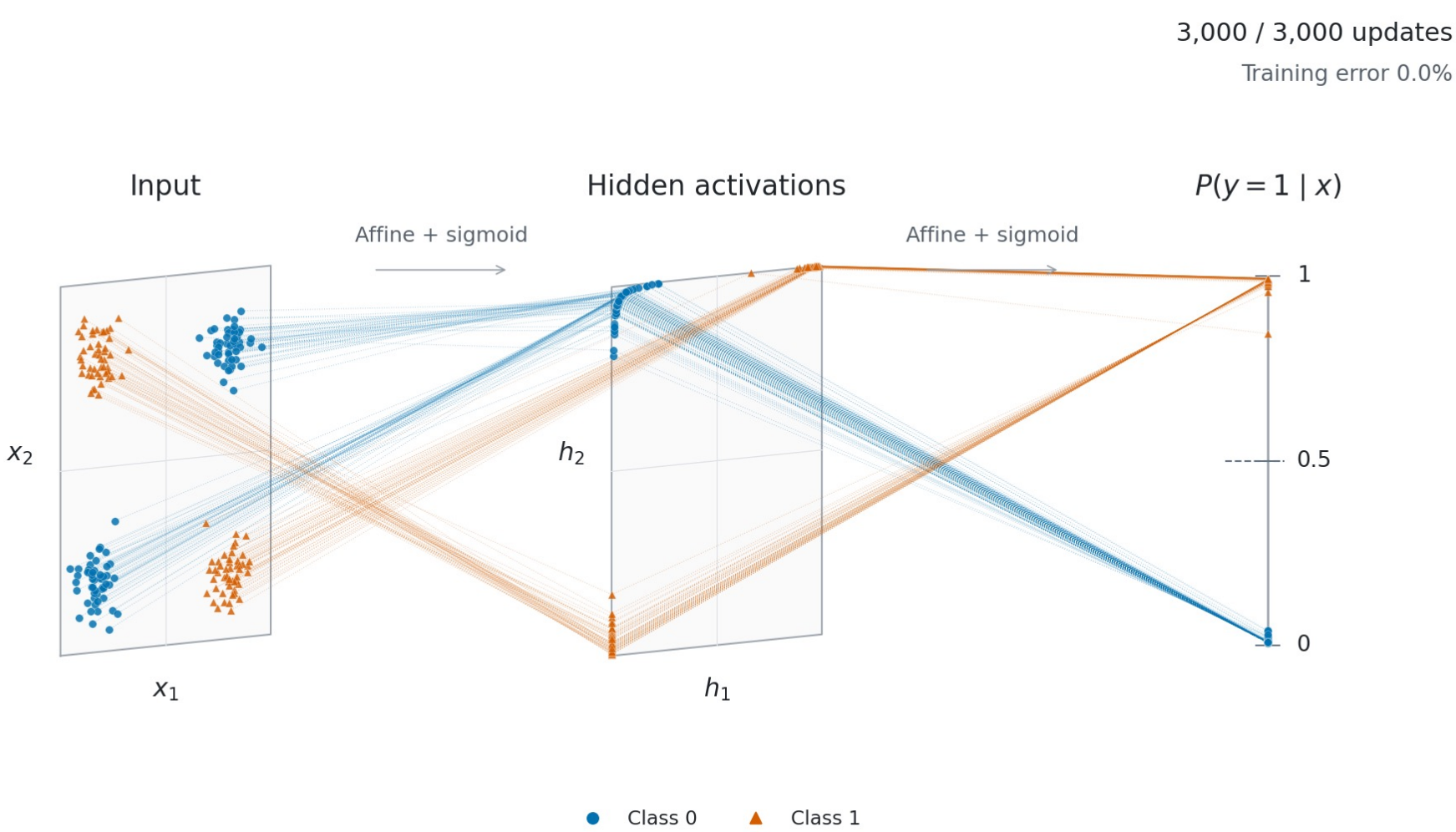
Derivatives for every weight and bias

$$\frac{\partial L}{\partial W_{ij}^{(l)}}, \qquad \frac{\partial L}{\partial b_i^{(l)}}$$

Adapted from G. Hinton

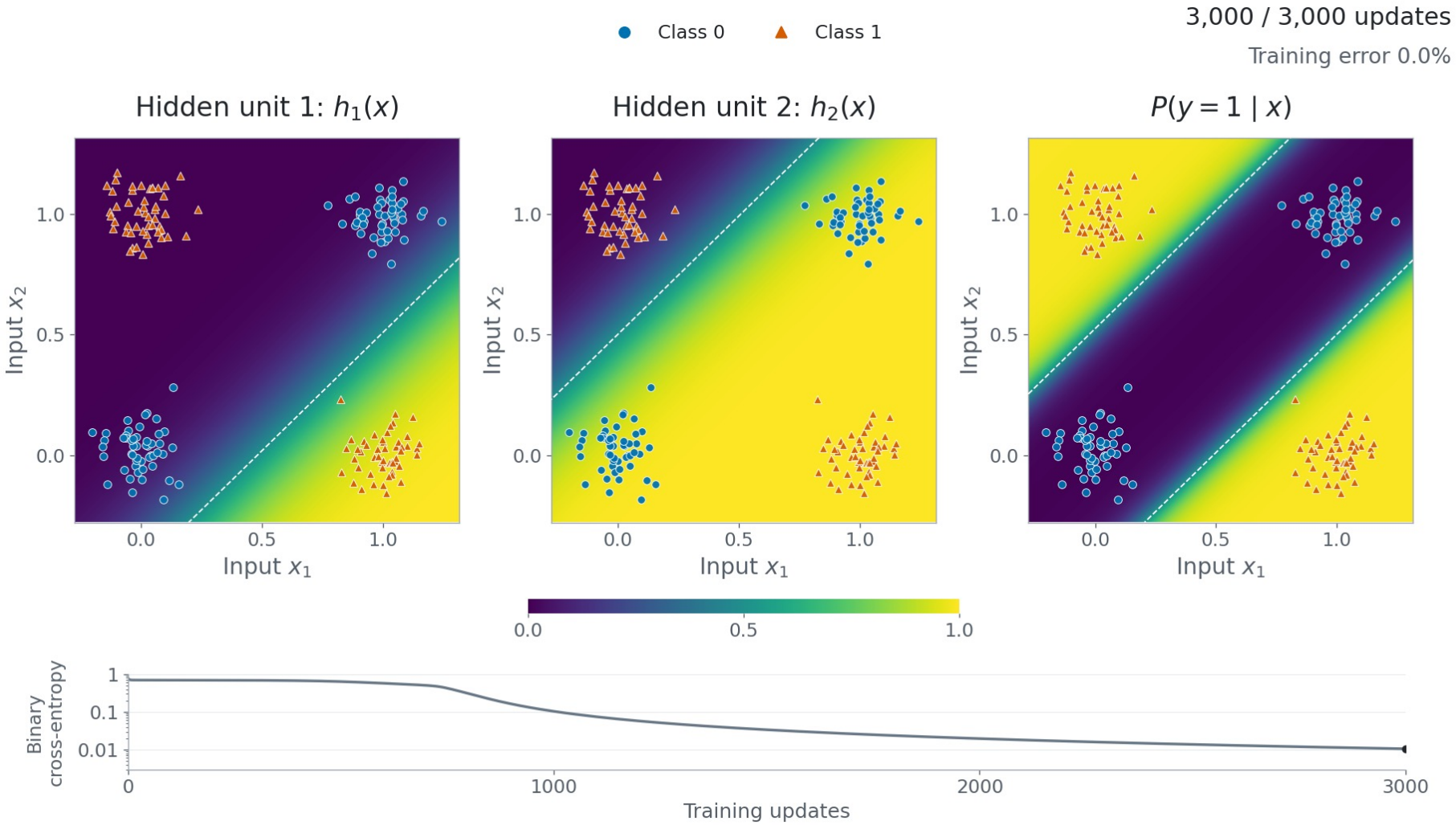
Learning XOR: representations during training

Two sigmoid hidden units feed a single sigmoid output.



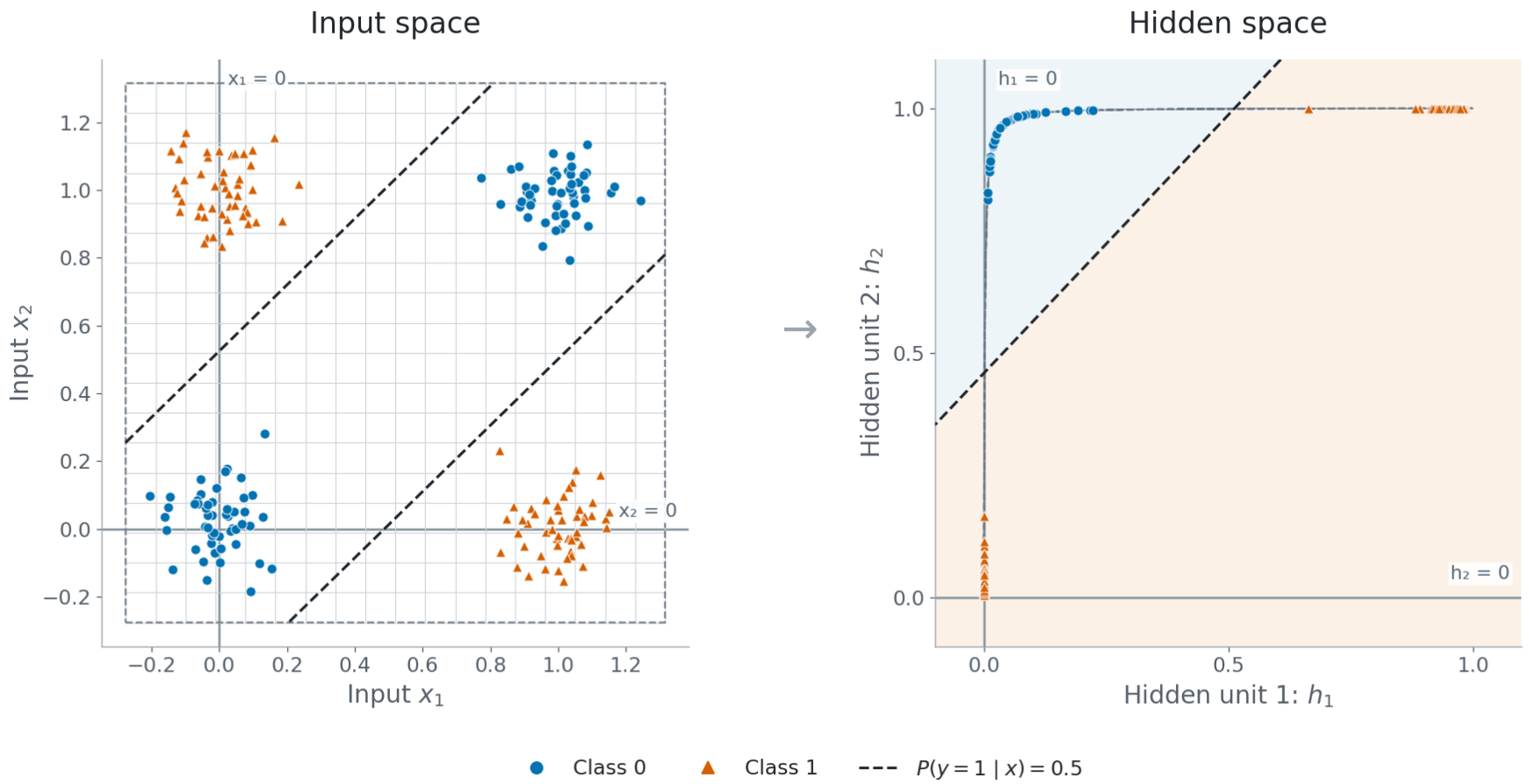
Hidden activations and output probability

The same network evaluated densely over the original input space.



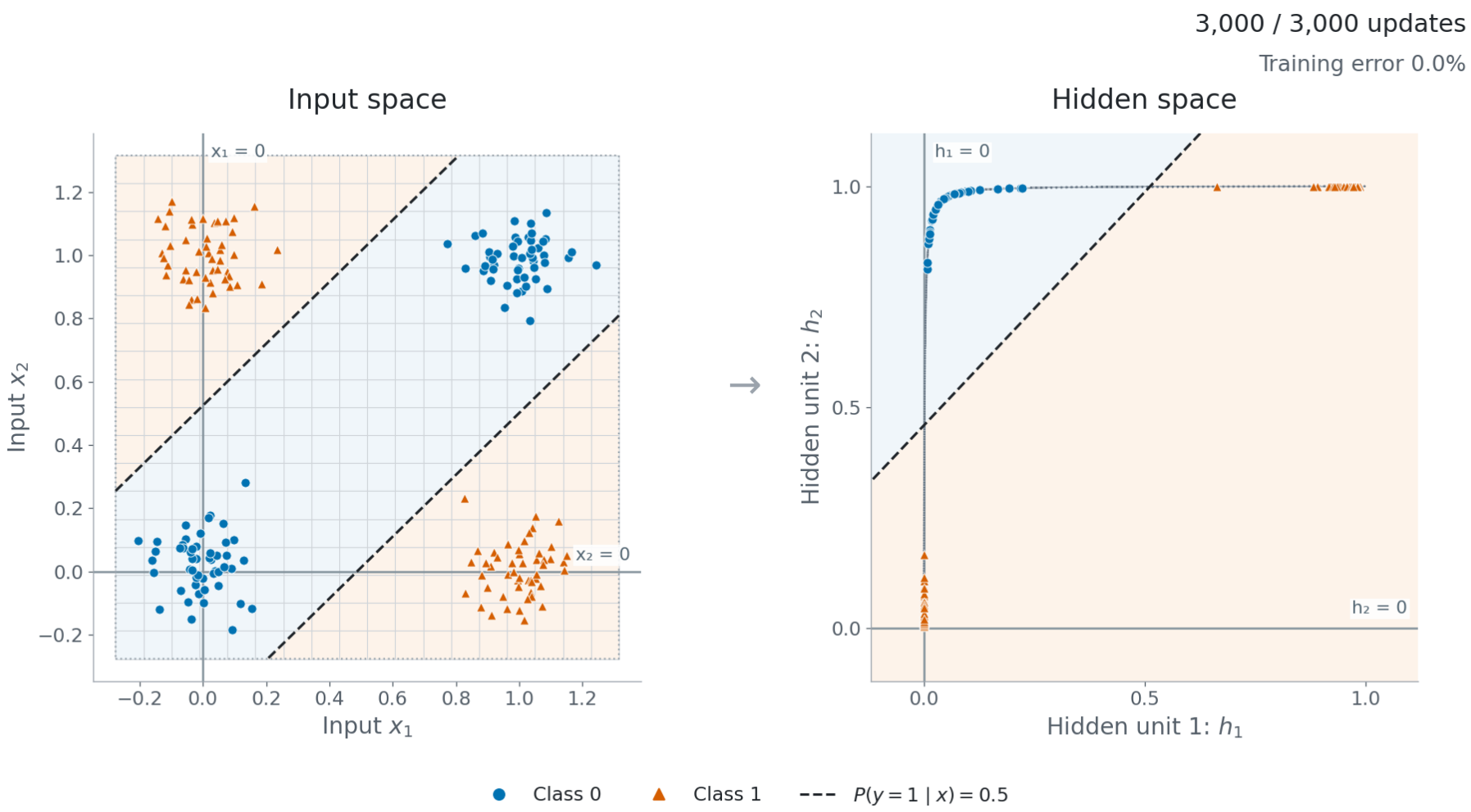
How a trained network deforms space

Fixed weights transform the grid through an affine map and sigmoid.



Learning the deformation and separating line

The hidden grid and decision boundary evolve over 3,000 updates.



Neural networks: expressiveness

Decision trees

Bagging and random forests

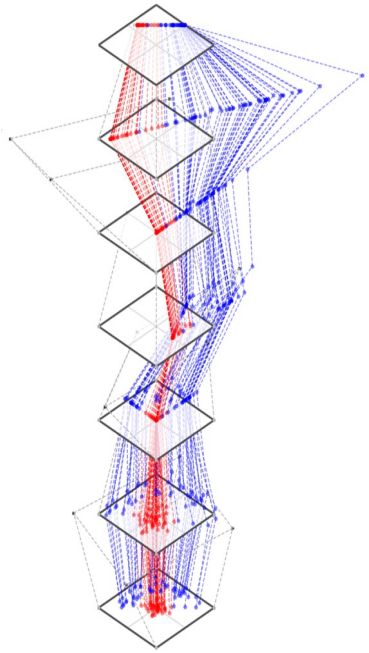
Kernels and SVMs

Neural networks

Expressiveness and representations

Backpropagation

Selected publication milestones



1998 Convolutional networks

2012 AlexNet

2017 Transformer

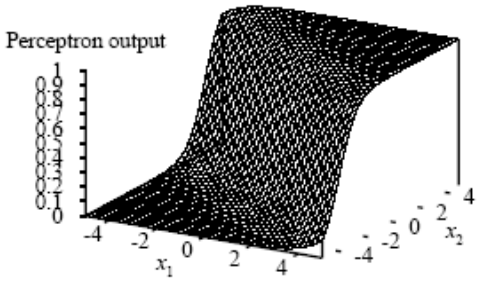
Later lectures

Reading: Expressiveness and representations

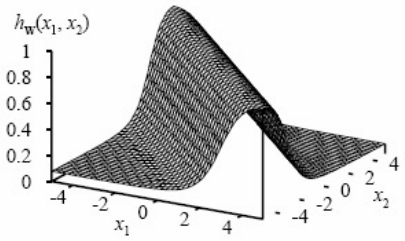
K. Murphy (2022) §§13.2.4–13.2.5 Example models and the importance of depth

Expressiveness of perceptrons

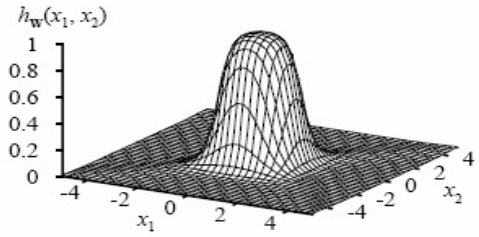
Single sigmoid neuron: ‘soft threshold’ function



Two opposite `soft threshold’ functions: a ridge



Two ridges: a bump



Universal function approximation

For every continuous f on compact K and every ε > 0,
one sufficiently wide hidden layer can achieve:

$$\widehat{f}(\mathbf{x}) = b_0 + \sum_{j=1}^{m} \alpha_j \, \sigma(\mathbf{w}_j^T \mathbf{x} + b_j)$$

$$\max_{\mathbf{x} \in K} |f(\mathbf{x}) - \widehat{f}(\mathbf{x})| < \epsilon$$

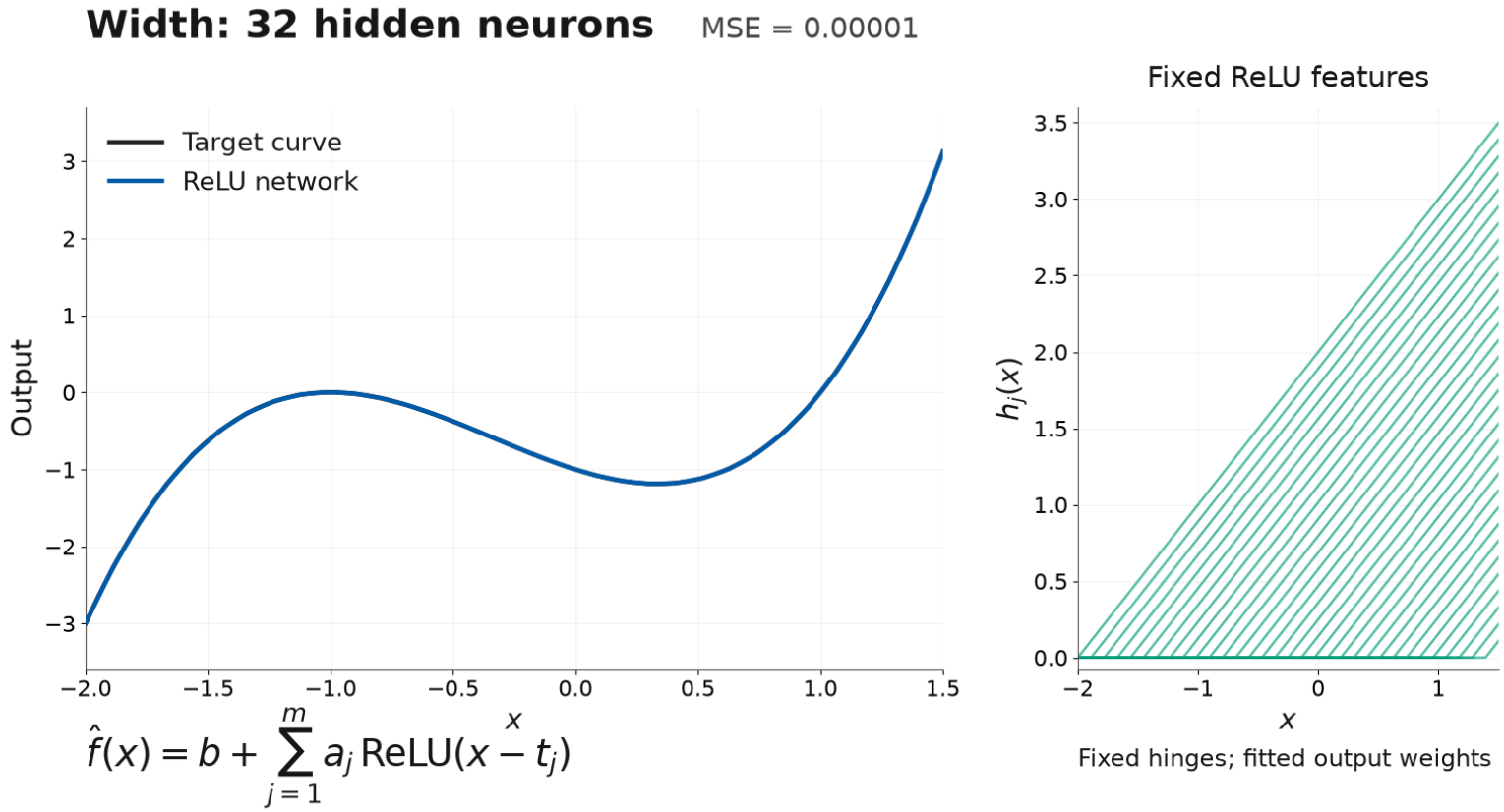
Use biases, a linear output and a continuous non-polynomial activation, such as sigmoid or ReLU.

m is the number of hidden neurons: the width.

This does not guarantee successful learning or generalization!

G. Cybenko (1989); M. Leshno, V. Lin, A. Pinkus and S. Schocken (1993)

Width builds a learned feature space



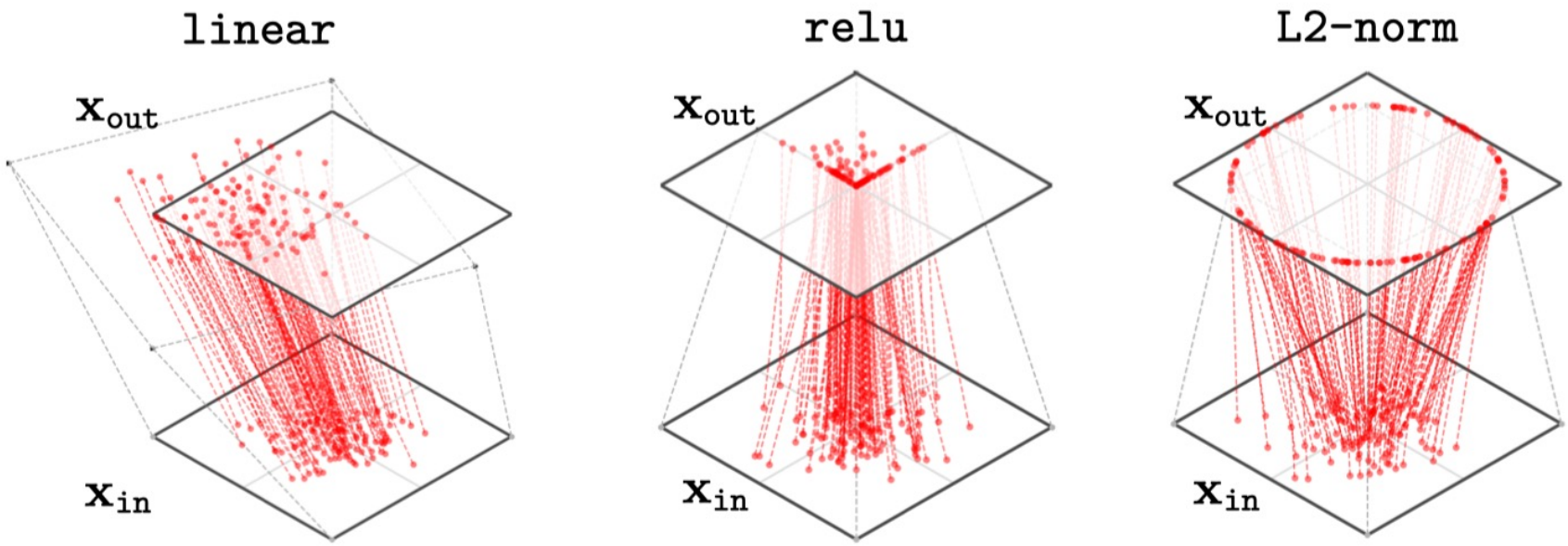
$$\phi(x) = (h_1(x), \ldots, h_m(x))^T \in \mathbb{R}^m$$

For some functions, depth uses far fewer units than width alone.

ReLU construction: B. Fortuner (2017), Can neural networks solve any problem?

Depth: M. Telgarsky (2016), Benefits of depth in neural networks.

Layers transform a cloud of data



Both input and output are 2D, so we can visualise each as a plane.

Lower plane: input. Upper plane: transformed output.

$$\mathbf{x}_{out} = \mathbf{W}\,\mathbf{x}_{in} + \mathbf{b}$$

Stretch and shift.
W: 2 × 2; b: 2 entries.

$$\mathbf{x}_{out,j} = \max(0, \mathbf{x}_{in,j})$$

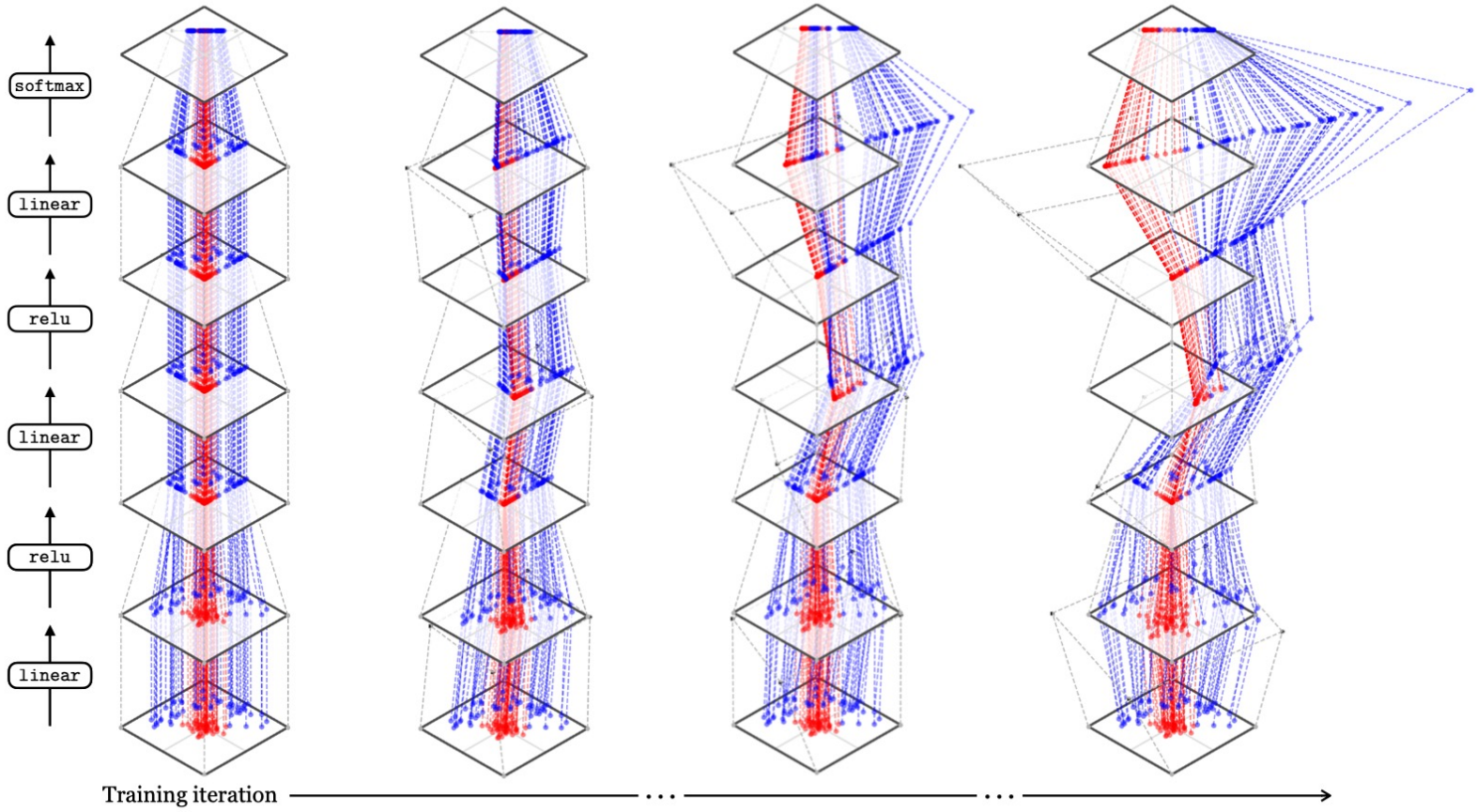
Set negatives to zero.
j = 1, 2 (each coordinate)

$$\mathbf{x}_{out} = \mathbf{x}_{in} \,/\, \|\mathbf{x}_{in}\|_2$$

$$\|\mathbf{x}\|_2 = \sqrt{(\mathbf{x}_1^2 + \mathbf{x}_2^2)}$$

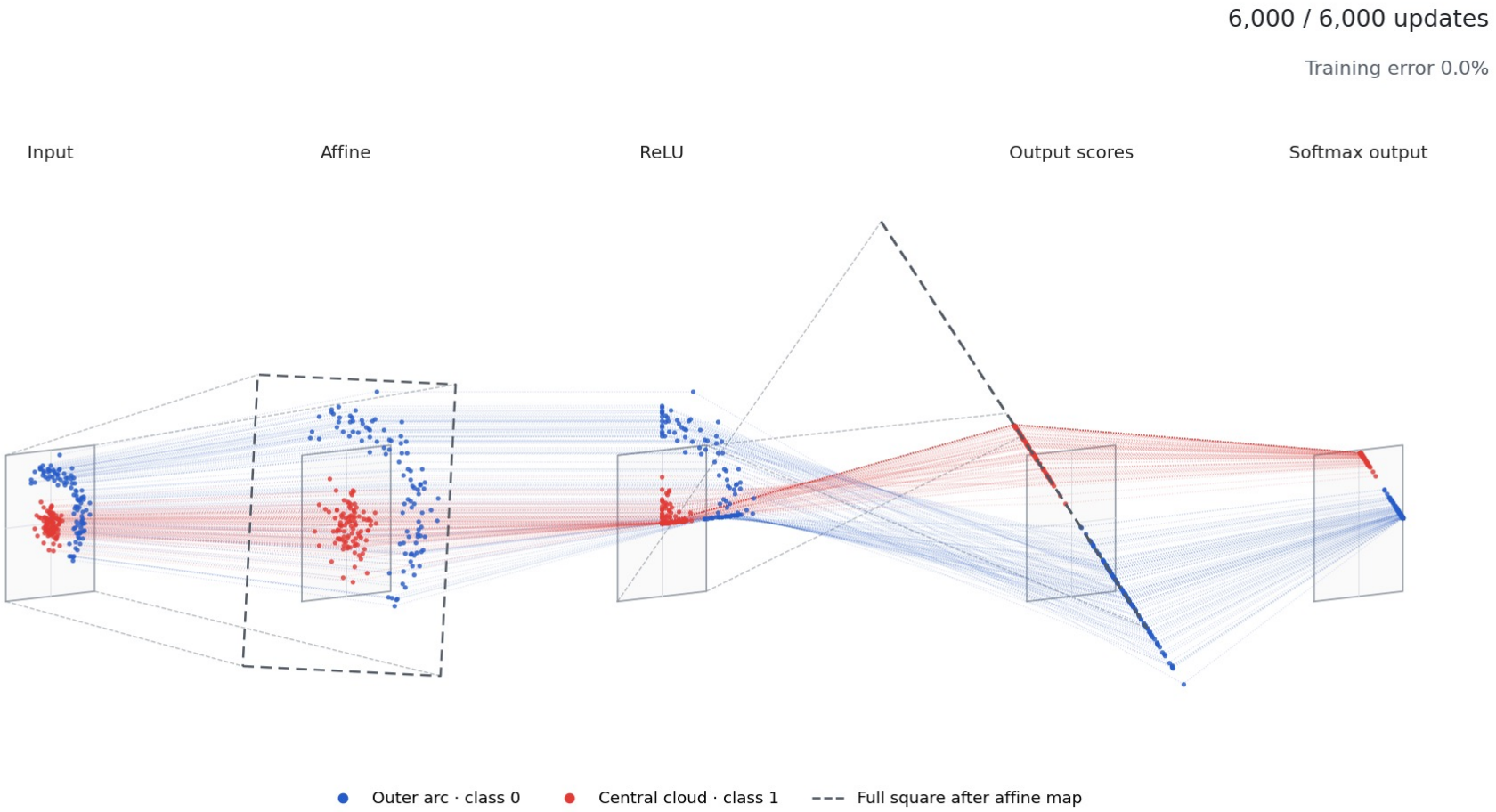
Nonzero input → unit circle

Learning changes the whole transformation

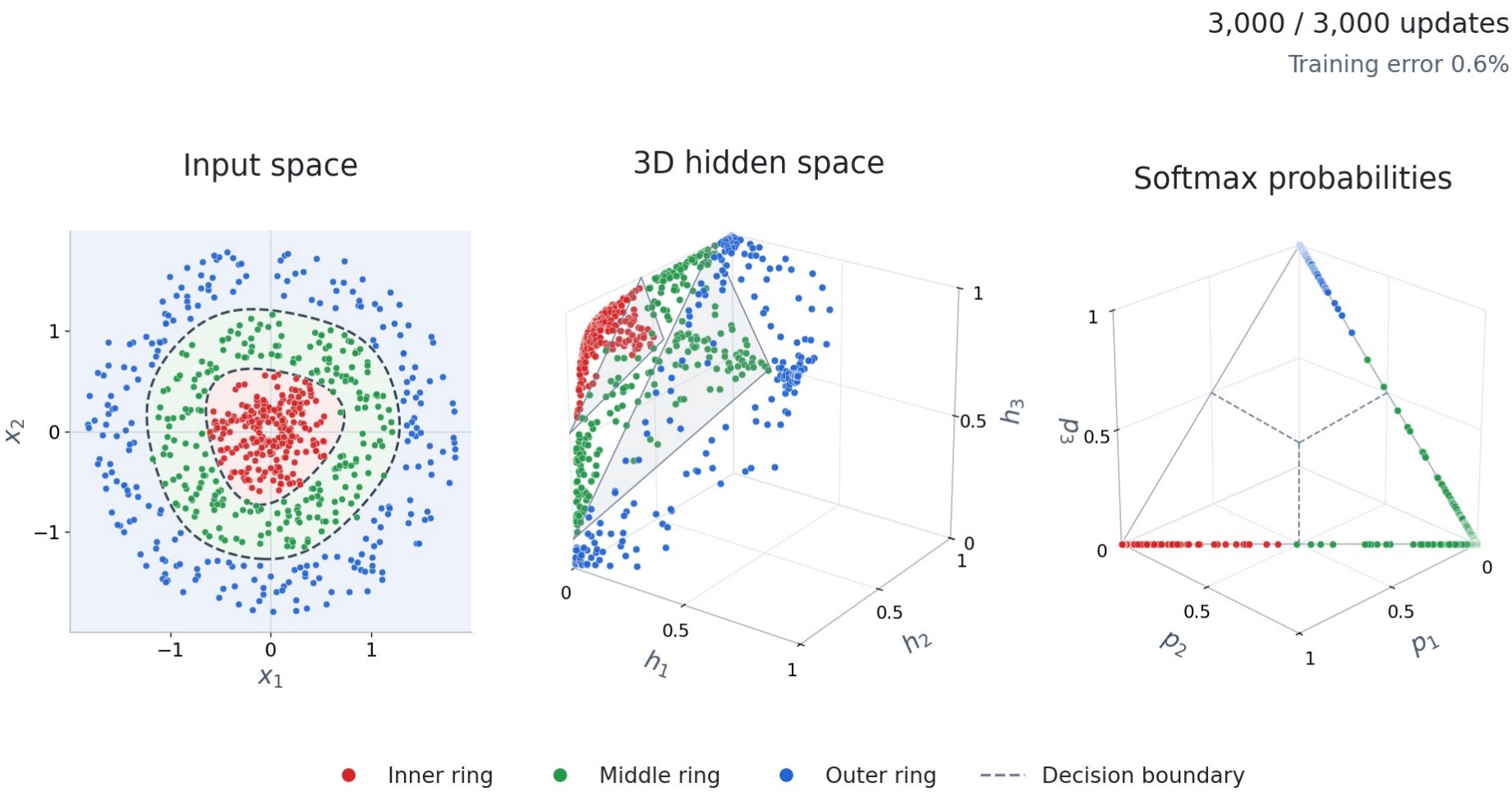


Upwards: layers. Across: training checkpoints. Colours: two classes. Training pulls each output towards its own class target.

Learning a nonlinear classifier

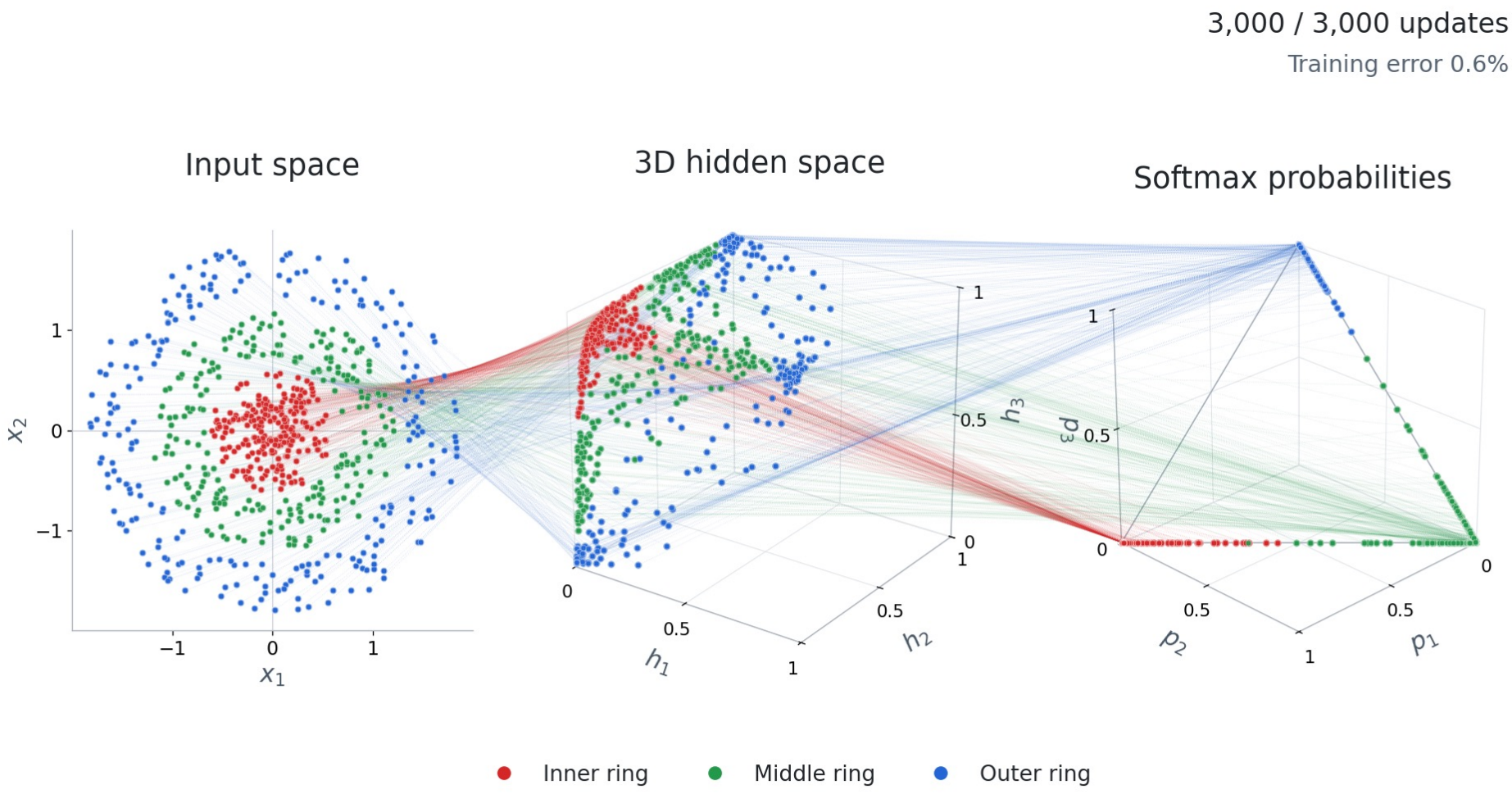


Learning a three-dimensional representation



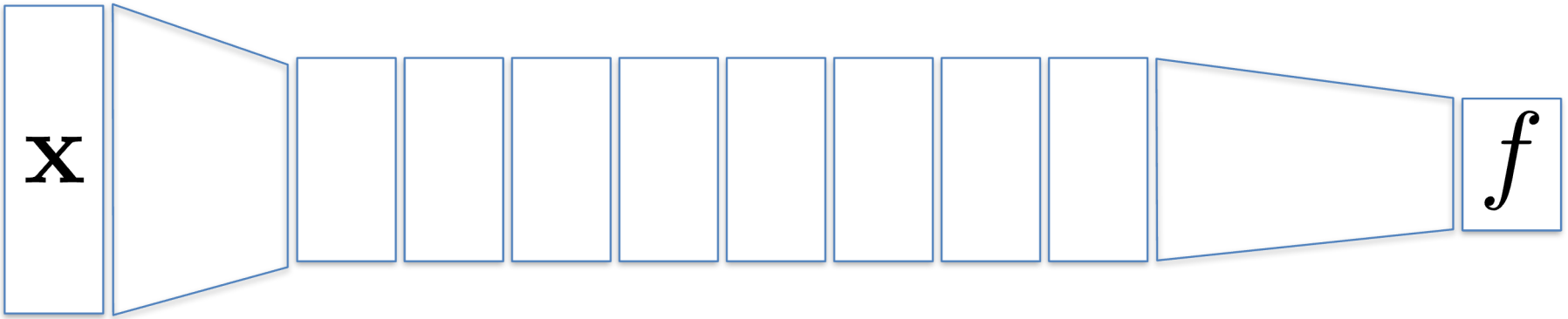
Three hidden activations give 3D features with linear class boundaries.

Following points across representations



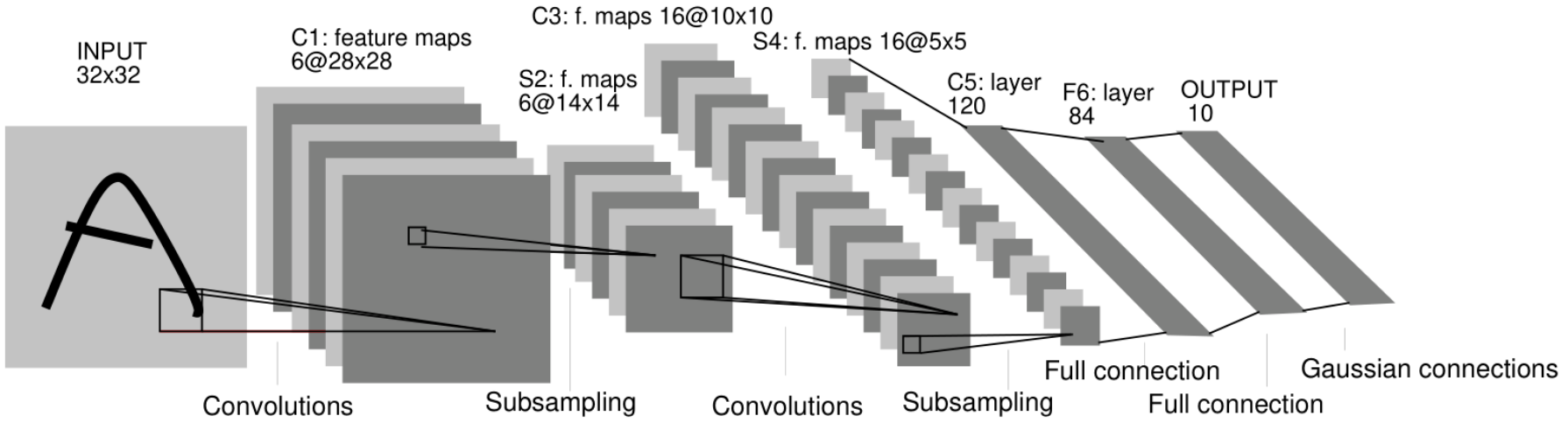
Links track each point. Softmax probabilities sum to one.

Deep networks



$$f = W_i \max(0, W_{i-1} \max(0, \cdots W_0 \mathbf{x})$$

Deep networks: LeNet-5

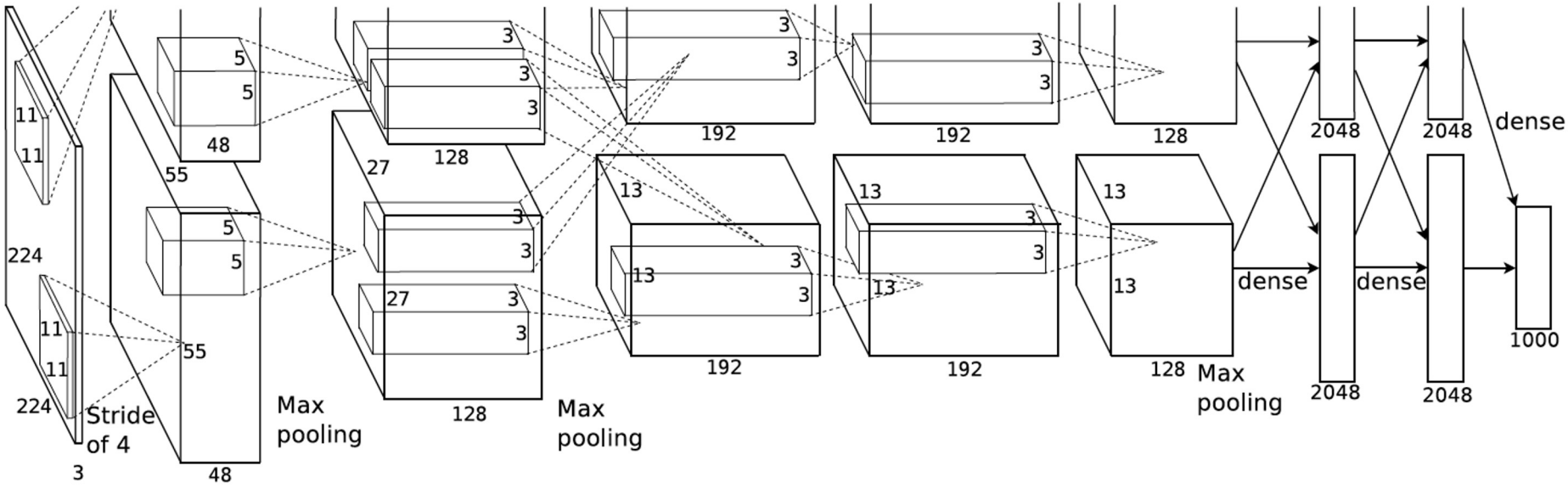


LeNet-5 recognises handwritten digits.

Convolutional networks in more detail: Week 4.

Y. LeCun, L. Bottou, Y. Bengio and P. Haffner (1998), Gradient-Based Learning Applied to Document Recognition, Proceedings of the IEEE 86(11), 2278–2324. Fig. 2.

Deep networks: AlexNet

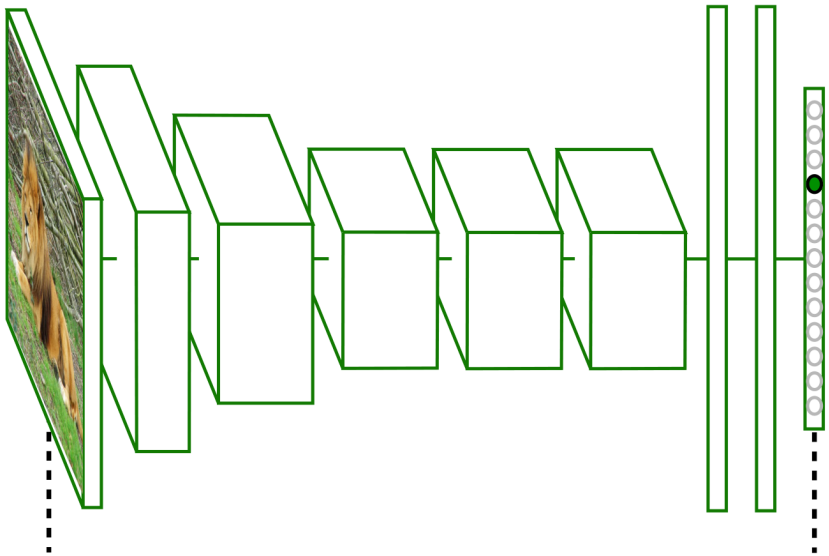


Convolutional networks in more detail: Week 4.

A. Krizhevsky, I. Sutskever and G. E. Hinton (2012), ImageNet Classification with
Deep Convolutional Neural Networks, Advances in Neural Information Processing Systems 25. Fig. 2.

2012: AlexNet

AlexNet used a deep convolutional network for ImageNet classification.



ImageNet top-5 error

AlexNet: 16.4%
Best shallow competitor: 26%

Next week: convolutional networks and ImageNet

Krizhevsky, Sutskever and Hinton (2012). Comparison reused from Lecture 4.

Meeting hours and course feedback

Monday, 6–7 pm

Office 450

Questions, clarifications and feedback



forms.cloud.microsoft/e/De8UK4392r

Backpropagation

Decision trees

Bagging and random forests

Kernels and SVMs

Neural networks

Expressiveness and representations

Backpropagation

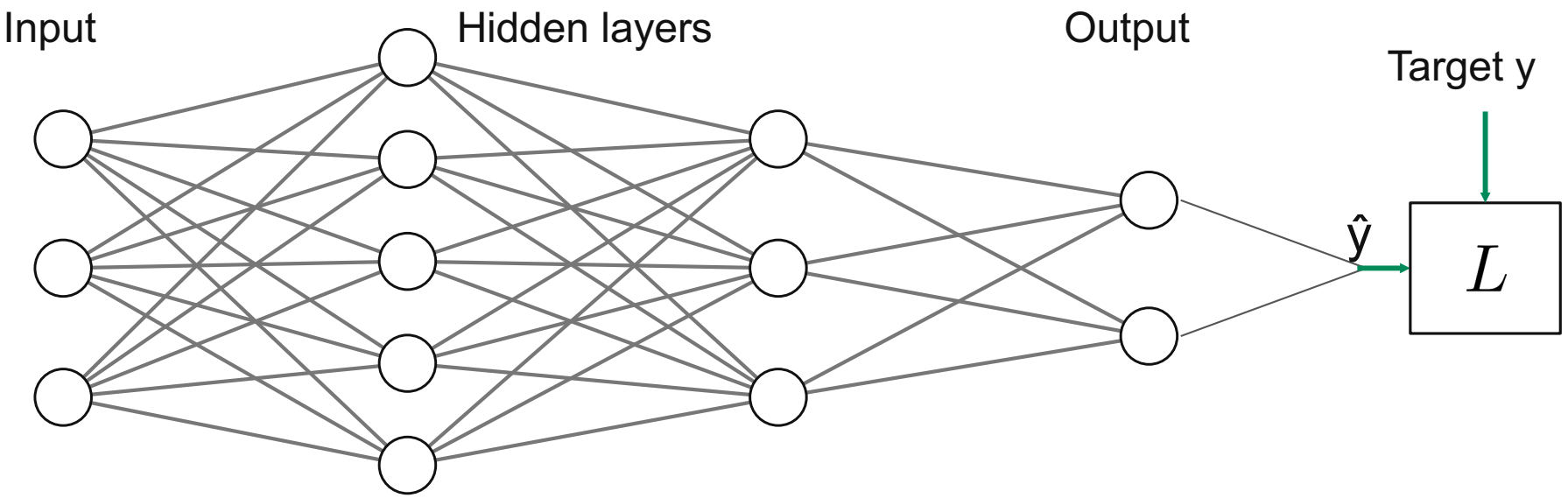
$$x \xrightarrow{g} u \xrightarrow{J} y$$

Reading: Backpropagation

K. Murphy (2022), §§13.3.2–13.3.4: backpropagation and local backward rules

Learning the network's parameters

θ_l collects all weights and biases in layer l. M is the final layer.



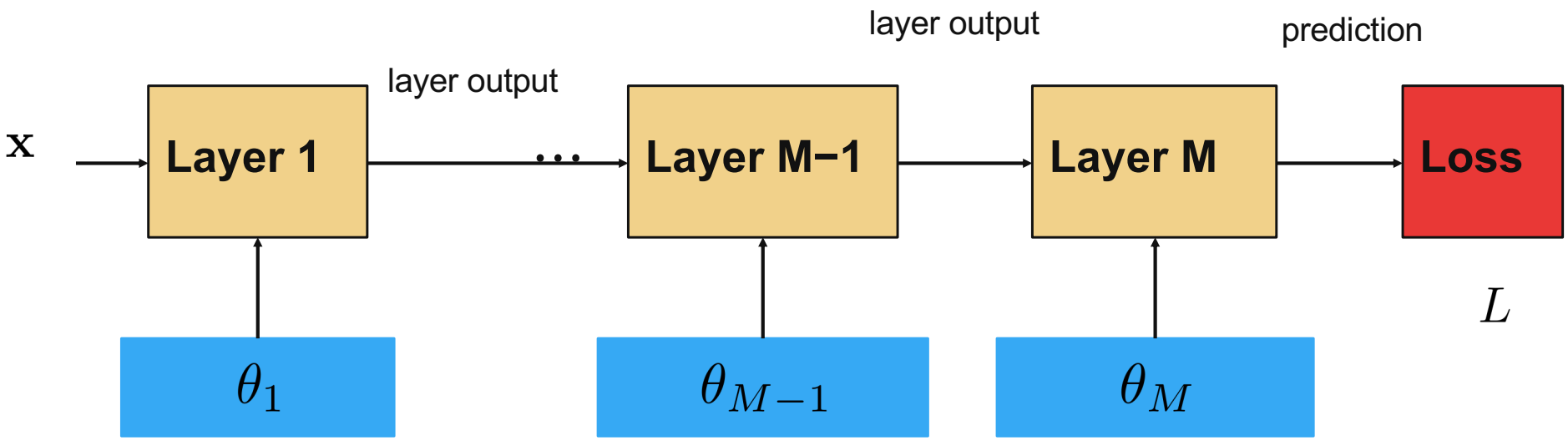
Backpropagation

Derivatives for every weight and bias

$$\frac{\partial L}{\partial \theta_1}, \quad \ldots, \quad \frac{\partial L}{\partial \theta_M}$$

Shared computation in a multilayer network

θ_l collects all weights and biases in layer l. M is the final layer.



Gradients for earlier layers pass through the same later layers.

Calculating every parameter derivative separately would repeat this work.

Backpropagation reuses derivative computation

Three scalar layer outputs. θ_l contains layer l's parameters.

$$x_0 \xrightarrow{f_1(\cdot,\theta_1)} x_1 \xrightarrow{f_2(\cdot,\theta_2)} x_2 \xrightarrow{f_3(\cdot,\theta_3)} x_3 \longrightarrow L$$

$$\frac{\partial L}{\partial \theta_3} = \frac{\partial L}{\partial x_3}\frac{\partial x_3}{\partial \theta_3}$$

$$\frac{\partial L}{\partial \theta_2} = \frac{\partial L}{\partial x_3}\frac{\partial x_3}{\partial x_2}\frac{\partial x_2}{\partial \theta_2}$$

$$\frac{\partial L}{\partial \theta_1} = \frac{\partial L}{\partial x_3}\frac{\partial x_3}{\partial x_2}\frac{\partial x_2}{\partial x_1}\frac{\partial x_1}{\partial \theta_1}$$

Compute shared factors once, then reuse them for earlier layers.

Mathematical notation

Vectors, matrices and scalar entries

$$\mathbf{x} \in \mathbb{R}^D, \quad x_j \in \mathbb{R}, \quad \mathbf{W} \in \mathbb{R}^{C \times H}$$

Transpose, norms and elementwise products

$$\mathbf{u}^T \mathbf{v} = \sum_j u_j v_j, \qquad \|\mathbf{x}\|_2^2 = \sum_j x_j^2$$

$$\|\mathbf{W}\|_F^2 = \sum_{j,k} W_{jk}^2, \qquad \mathbf{u} \odot \mathbf{v} = (u_j v_j)_j$$

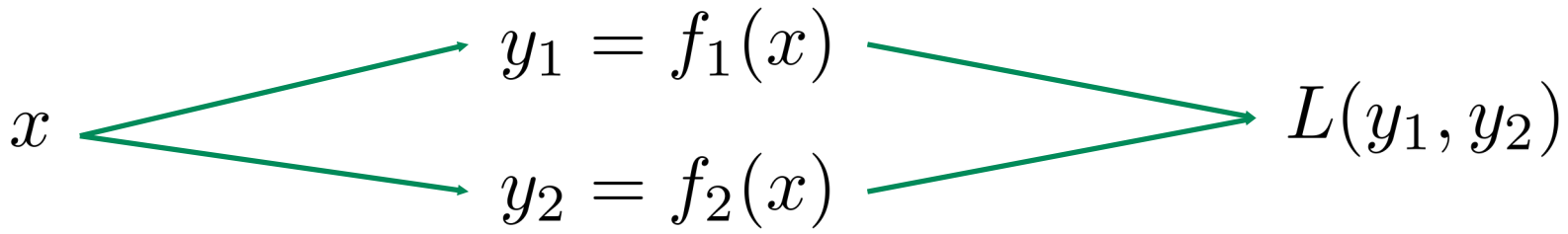
Derivatives of a scalar loss L

$$\mathbf{g_u} \equiv \frac{\partial L}{\partial \mathbf{u}} \in \mathbb{R}^{n \times 1}, \qquad \left[\frac{\partial L}{\partial \mathbf{W}}\right]_{jk} = \frac{\partial L}{\partial W_{jk}}$$

$$J_f = \frac{\partial \mathbf{v}}{\partial \mathbf{u}} \in \mathbb{R}^{m \times n}, \qquad \mathbf{g_u} = J_f^T \mathbf{g_v}$$

Jacobian: output components in rows, input components in columns.

Chain rule: from scalars to vectors



L: partial derivatives. $y_1(x)$, $y_2(x)$: ordinary derivatives.

$$\frac{dL}{dx} = \frac{\partial L}{\partial y_1}\frac{dy_1}{dx} + \frac{\partial L}{\partial y_2}\frac{dy_2}{dx}$$

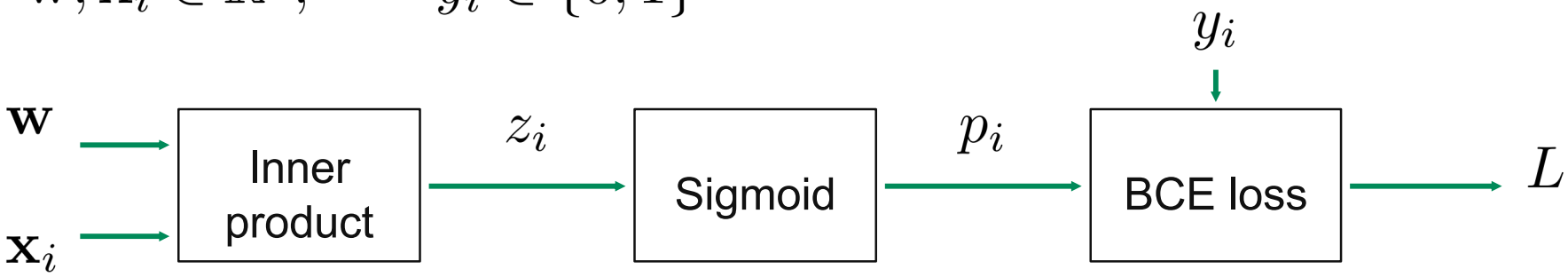
$\mathbf{y} = (y_1, y_2)^T$ Collect the derivatives into column vectors:

$$\frac{\partial L}{\partial \mathbf{y}} := \begin{bmatrix} \partial L/\partial y_1 \\ \partial L/\partial y_2 \end{bmatrix} \qquad \frac{d\mathbf{y}}{dx} := \begin{bmatrix} dy_1/dx \\ dy_2/dx \end{bmatrix}$$

$$\underbrace{\frac{dL}{dx}}_{1\times 1} = \underbrace{\left(\frac{\partial L}{\partial \mathbf{y}}\right)^T}_{1\times 2}\underbrace{\frac{d\mathbf{y}}{dx}}_{2\times 1}$$

Logistic regression: local derivatives

$$\mathbf{w}, \mathbf{x}_i \in \mathbb{R}^d, \qquad y_i \in \{0, 1\}$$



$$z_i = \mathbf{w}^T \mathbf{x}_i \qquad p_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}} \qquad L = -y_i \log p_i - (1 - y_i) \log(1 - p_i)$$

Inner product

$$\nabla_{\mathbf{w}} z_i = \mathbf{x}_i$$

$$\nabla_{\mathbf{x}_i} z_i = \mathbf{w}$$

Sigmoid

$$\frac{dp_i}{dz_i} = p_i(1 - p_i)$$

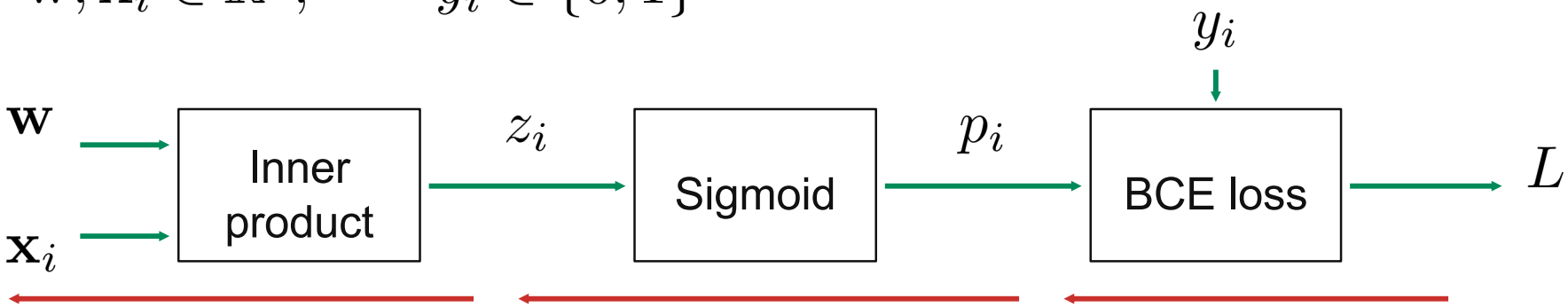
Binary cross-entropy

$$\frac{\partial L}{\partial p_i} = -\frac{y_i}{p_i} + \frac{1 - y_i}{1 - p_i}$$

Each block supplies a local derivative for the backward pass.

Logistic regression: applying the chain rule

$$\mathbf{w}, \mathbf{x}_i \in \mathbb{R}^d, \qquad y_i \in \{0, 1\}$$



Use the local derivatives from the previous slide.

Score

$$\frac{\partial L}{\partial z_i} = \frac{\partial L}{\partial p_i}\frac{\partial p_i}{\partial z_i}$$
$$= \left(-\frac{y_i}{p_i} + \frac{1 - y_i}{1 - p_i}\right) p_i(1 - p_i) = p_i - y_i$$

Weights

$$\nabla_{\mathbf{w}} L = \frac{\partial L}{\partial z_i} \nabla_{\mathbf{w}} z_i$$
$$= (p_i - y_i)\mathbf{x}_i$$

Gradients have the parameter's shape

The loss L is scalar. Differentiate with respect to each parameter entry.

Scalar

$$q \in \mathbb{R}: \quad \nabla_q L = \frac{\partial L}{\partial q} \in \mathbb{R}$$

Vector

$$\mathbf{q} \in \mathbb{R}^{d\times 1}: \quad \nabla_{\mathbf{q}} L = \begin{bmatrix} \partial L/\partial q_1 \\ \vdots \\ \partial L/\partial q_d \end{bmatrix} \in \mathbb{R}^{d\times 1}$$

Matrix

$$\mathbf{Q} \in \mathbb{R}^{r\times c}: \quad (\nabla_{\mathbf{Q}} L)_{ij} = \frac{\partial L}{\partial Q_{ij}}, \quad \nabla_{\mathbf{Q}} L \in \mathbb{R}^{r\times c}$$

$$\mathbf{g}_q := \frac{\partial L}{\partial \mathbf{q}} = \nabla_{\mathbf{q}} L$$

Vectors and their gradients are columns.
Matrix gradients have the parameter's shape.

Network notation

Two-layer worked MLP

$$\mathbf{z} = \mathbf{V}\mathbf{x}, \quad \mathbf{a} = \mathrm{ReLU}(\mathbf{z}), \quad \hat{\mathbf{y}} = \mathbf{W}\mathbf{a}$$

$$\mathbf{V} \in \mathbb{R}^{H \times D}, \quad \mathbf{W} \in \mathbb{R}^{C \times H}, \quad \mathbf{z}, \mathbf{a} \in \mathbb{R}^{H \times 1}$$

x: input z: hidden scores a: hidden activations
ŷ: prediction y: target L: scalar loss

D: inputs H: hidden units C: outputs or classes

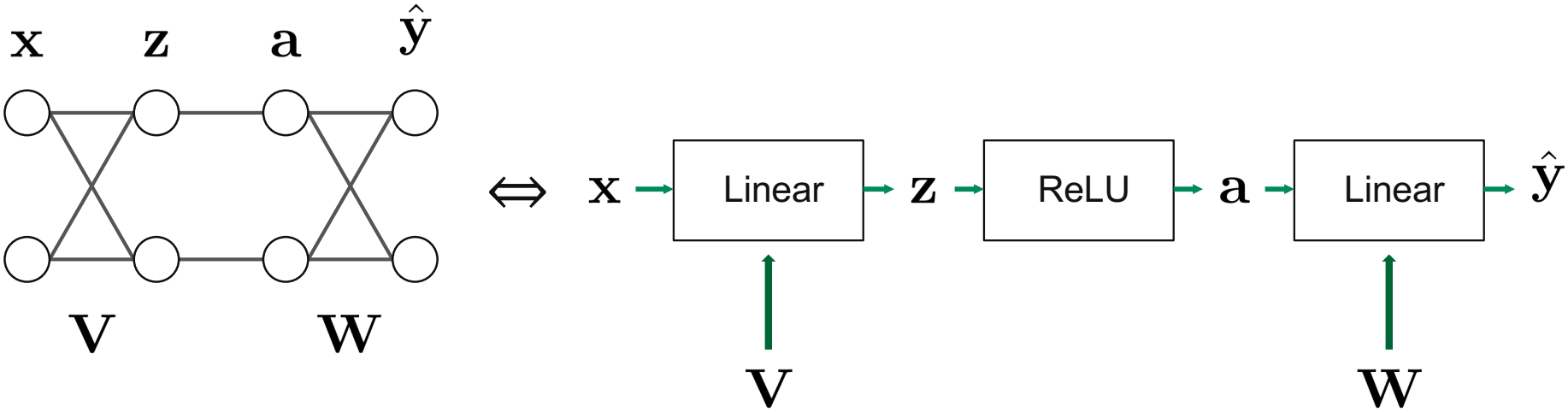
Deeper network

$$\mathbf{a}^{[0]} = \mathbf{x}, \quad \mathbf{z}^{[l]} = \mathbf{W}^{[l]}\mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}, \quad \mathbf{a}^{[l]} = f_l(\mathbf{z}^{[l]})$$

θ_l: all parameters in layer l [l]: layer index M: final layer
g_u: the column loss gradient at u $\oslash$: elementwise division

ℓ: loss for one example (used in the risk appendix).

An MLP as a computation graph



Parameters and activations

Parameters are the weights and biases shared across examples.

Activations are the intermediate values computed for one example.

Learning changes the parameters using a training dataset.

Inference holds those parameters fixed and computes new activations.

Backpropagation: worked MLP

Lecture 3 · Part I

Decision trees

Bagging and random forests

Kernels and SVMs

Neural networks

Expressiveness and representations

Backpropagation

A two-layer regression network

Follow one gradient at a time

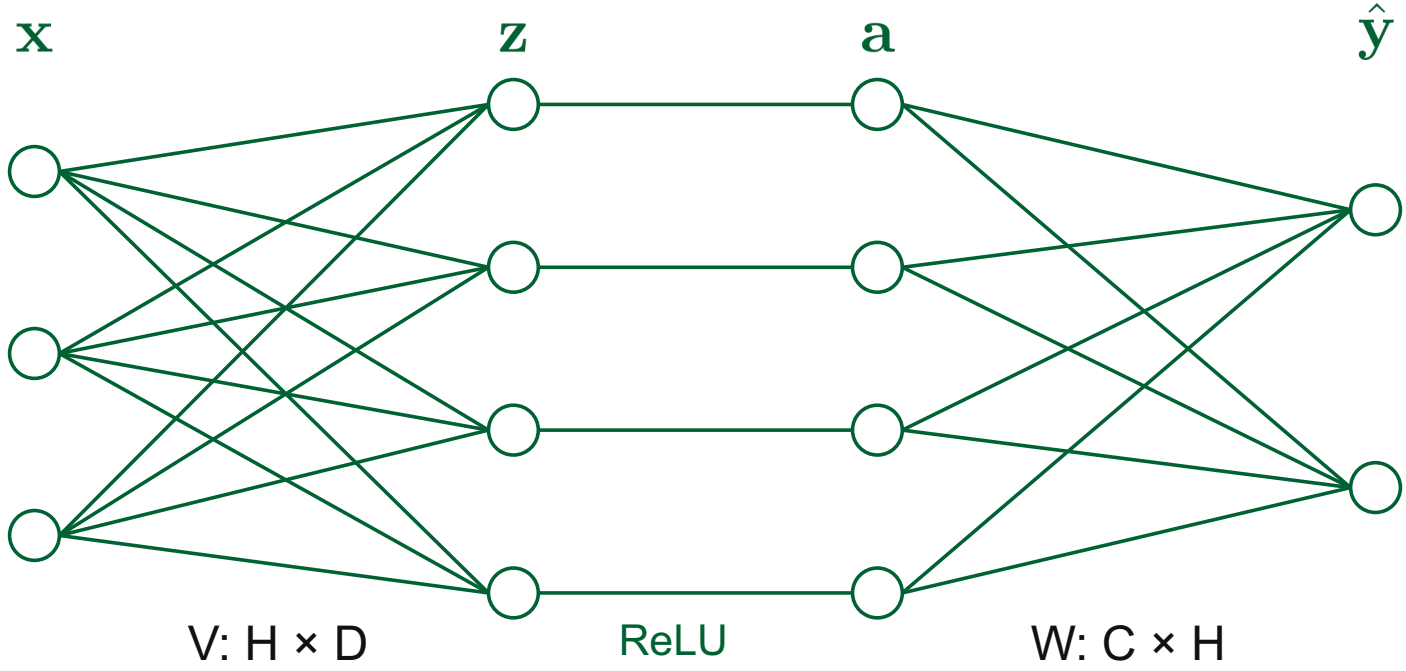
Then generalise to deeper graphs

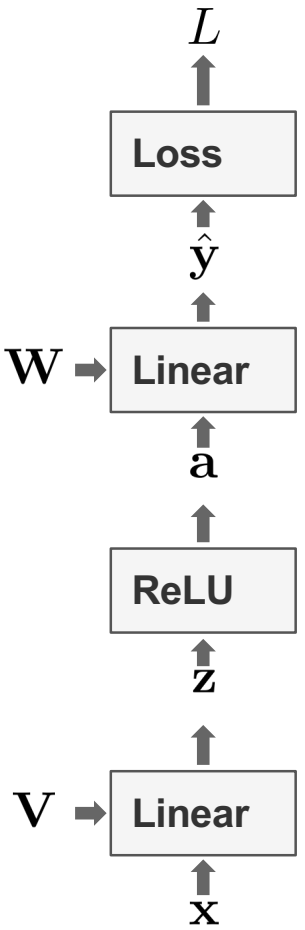
Reading

K. Murphy (2022), §§13.3.2–13.3.4

The worked regression MLP

D inputs, H hidden units, C outputs. Biases are omitted.



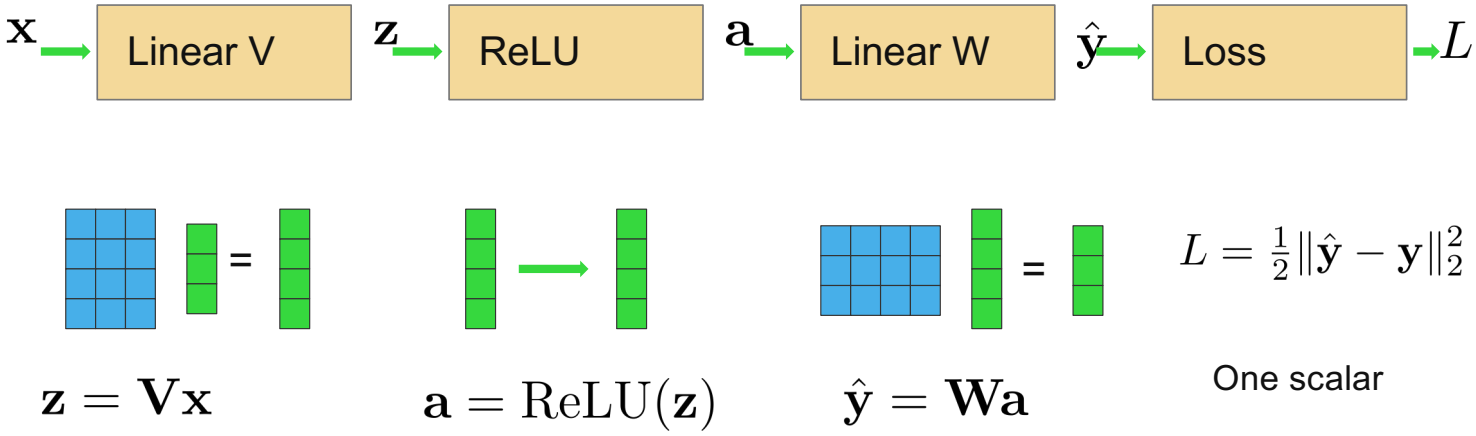


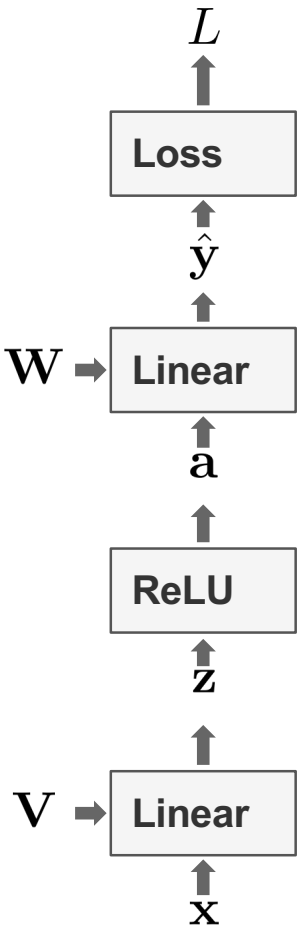
$$\mathbf{z} = \mathbf{V}\mathbf{x} \qquad \mathbf{a} = \mathrm{ReLU}(\mathbf{z}) \qquad \hat{\mathbf{y}} = \mathbf{W}\mathbf{a}$$

$$L = \tfrac{1}{2}\|\hat{\mathbf{y}} - \mathbf{y}\|_2^2$$

Forward pass through a ReLU MLP

The same cached values will be reused in the backward pass.

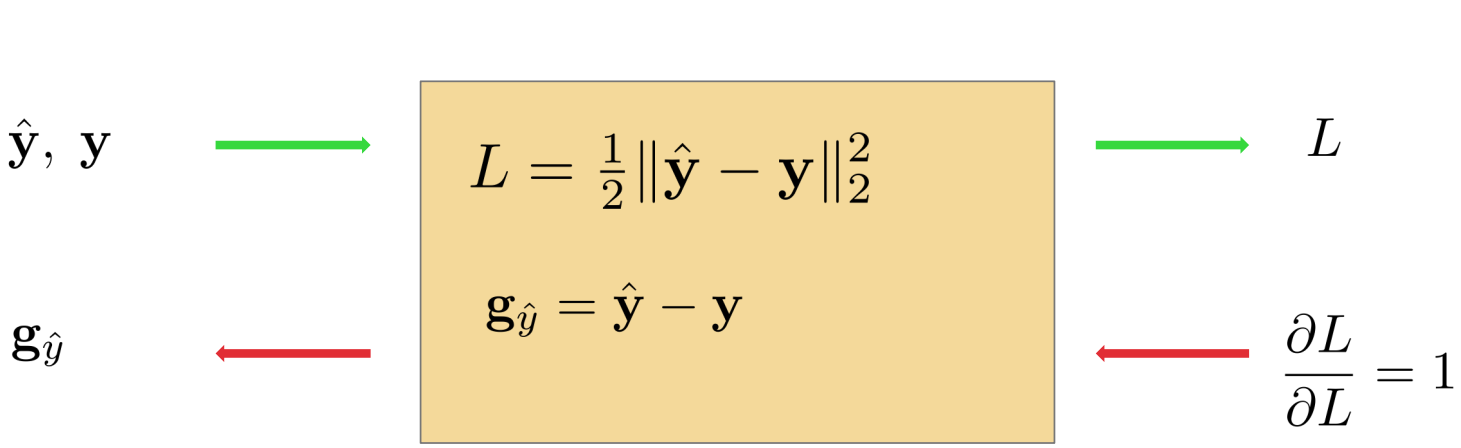




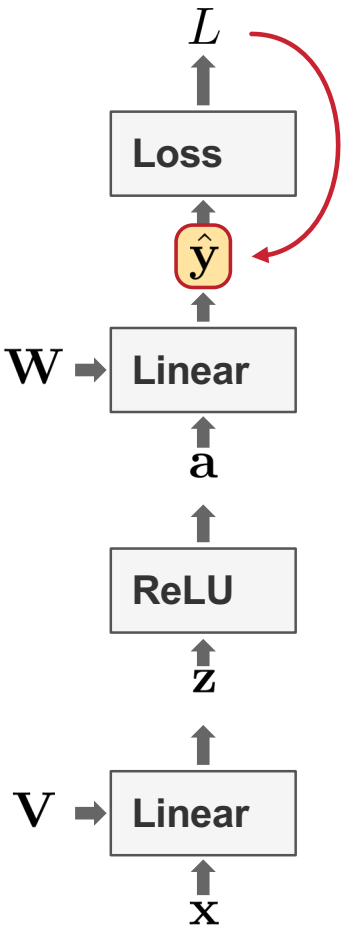
Cache x, z and a; the prediction is ŷ = Wa.

The loss starts the backward pass

Our loss includes ½, so its derivative is the residual.



$$\frac{\partial L}{\partial \hat{y}_c} = \hat{y}_c - y_c$$



The C residual entries form the incoming column gradient.

The local squared-loss derivative

One training point with C real-valued output coordinates.

Loss

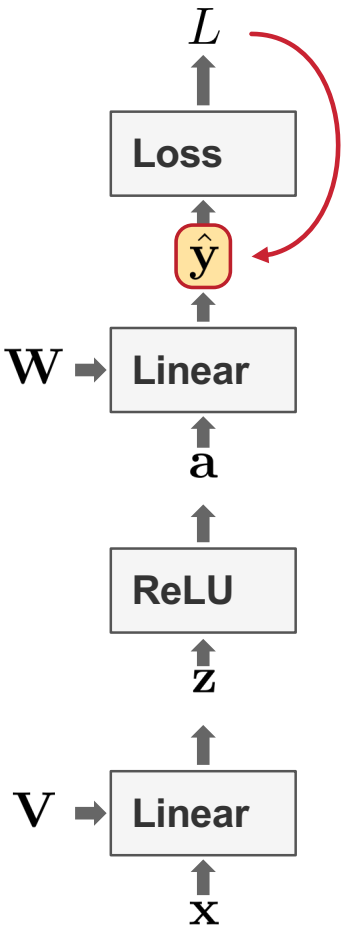
$$L = \frac{1}{2}\sum_{c=1}^{C}(\hat{y}_c - y_c)^2$$

One coordinate

$$\frac{\partial L}{\partial \hat{y}_c} = \frac{1}{2}\,2(\hat{y}_c - y_c) = \hat{y}_c - y_c$$

Column gradient

$$\underbrace{\mathbf{g}_{\hat{y}}}_{C\times 1} := \nabla_{\hat{\mathbf{y}}} L = \hat{\mathbf{y}} - \mathbf{y}$$



The residual is the incoming column gradient for the output linear map.

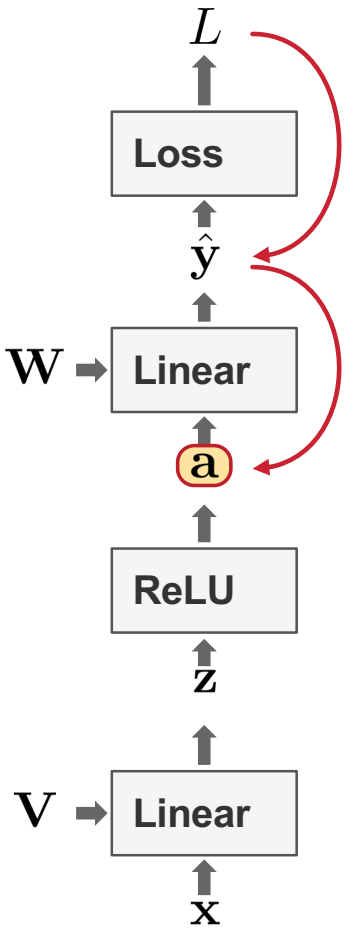
Backward through the output linear map

Each hidden activation influences every output coordinate.

$$\hat{y}_c = \sum_{j=1}^{H} W_{cj} a_j, \qquad \frac{\partial \hat{y}_c}{\partial a_h} = W_{ch}$$

$$g_{a,h} = \frac{\partial L}{\partial a_h} = \sum_{c=1}^{C} \frac{\partial L}{\partial \hat{y}_c} W_{ch} = \sum_{c=1}^{C} W_{ch} g_{\hat{y},c}$$

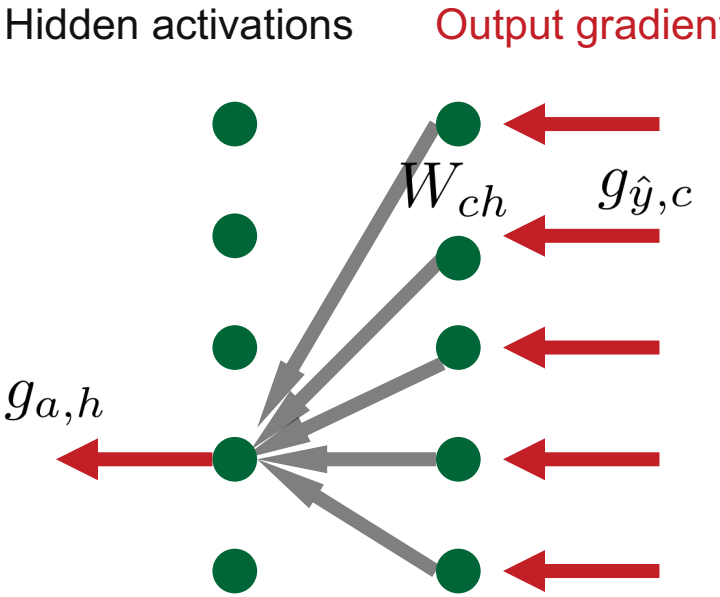
$$\underbrace{\mathbf{g}_a}_{H \times 1} = \underbrace{\mathbf{W}^T}_{H \times C} \underbrace{\mathbf{g}_{\hat{y}}}_{C \times 1}$$



The transpose collects the same weighted sums into one matrix product.

Backward through the output linear map

One hidden activation receives contributions from every output.

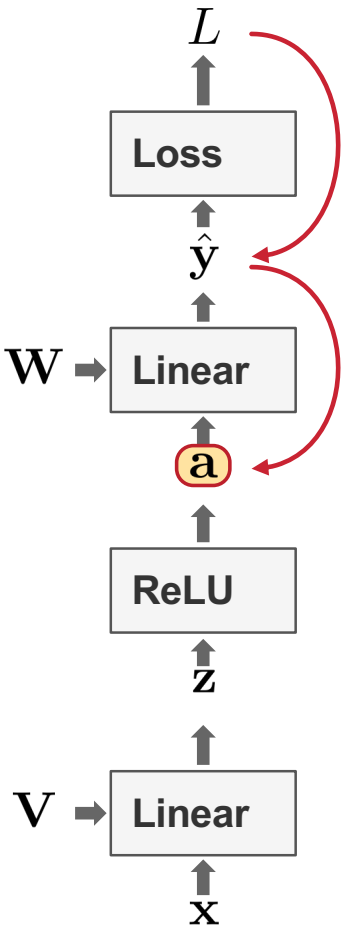


$$\hat{y}_c = \sum_{j=1}^{H} W_{cj} a_j$$

$$\frac{\partial \hat{y}_c}{\partial a_h} = W_{ch}$$

$$g_{a,h} = \sum_{c=1}^{C} W_{ch}\, g_{\hat{y},c}$$

$$\underbrace{\mathbf{g}_a}_{H\times 1} = \underbrace{\mathbf{W}^T}_{H\times C} \underbrace{\mathbf{g}_{\hat{y}}}_{C\times 1}$$



The transpose collects the same weighted sums into one matrix product.

The gradient of one output weight

Each output weight connects hidden activation h to output c.

Local derivative

$$\frac{\partial \hat{y}_j}{\partial W_{ch}} = \begin{cases} a_h & j = c, \\ 0 & j \neq c, \end{cases}$$

Chain rule

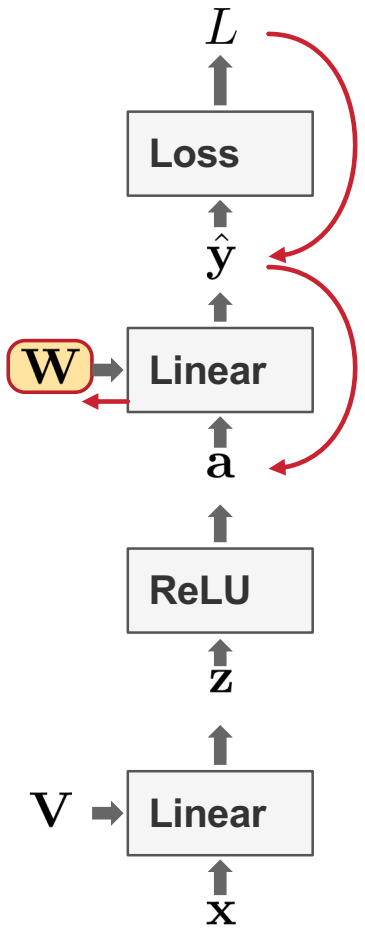
$$\frac{\partial L}{\partial W_{ch}} = \sum_{j=1}^{C} \frac{\partial L}{\partial \hat{y}_j} \frac{\partial \hat{y}_j}{\partial W_{ch}}$$

Substitution

$$\frac{\partial L}{\partial W_{ch}} = \frac{\partial L}{\partial \hat{y}_c} a_h$$

Gradient

$$\frac{\partial L}{\partial W_{ch}} = g_{\hat{y},c}\, a_h$$

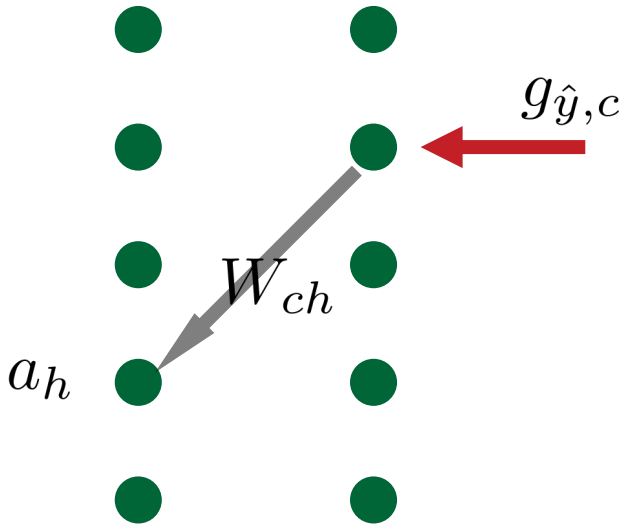


Only one output depends directly on this coefficient.

The gradient of one output weight

Each weight uses its input activation and its output's loss gradient.

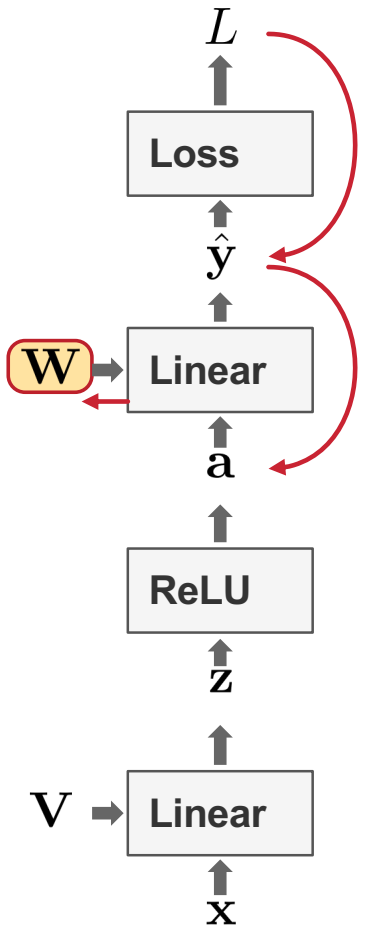
One selected connection



$$\frac{\partial \hat{y}_c}{\partial W_{ch}} = a_h$$

$$\frac{\partial L}{\partial W_{ch}} = g_{\hat{y},c}\, a_h$$

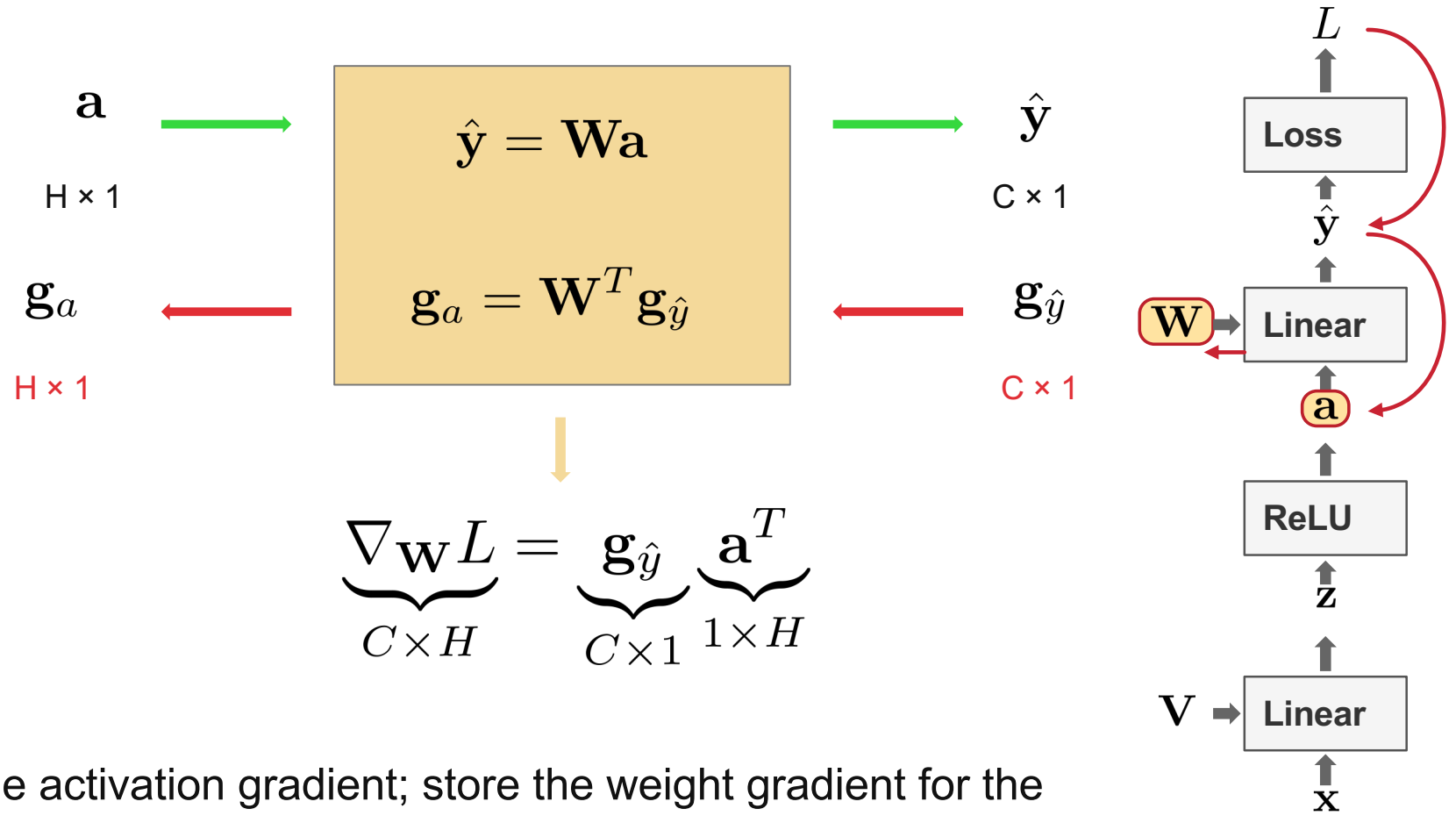
$$\underbrace{\nabla_{\mathbf{W}} L}_{C \times H} = \underbrace{\mathbf{g}_{\hat{y}}}_{C \times 1} \underbrace{\mathbf{a}^T}_{1 \times H}$$



The same two factors give every entry of the weight-gradient matrix.

Output linear layer: the backward block

One incoming column gradient produces both required gradients.

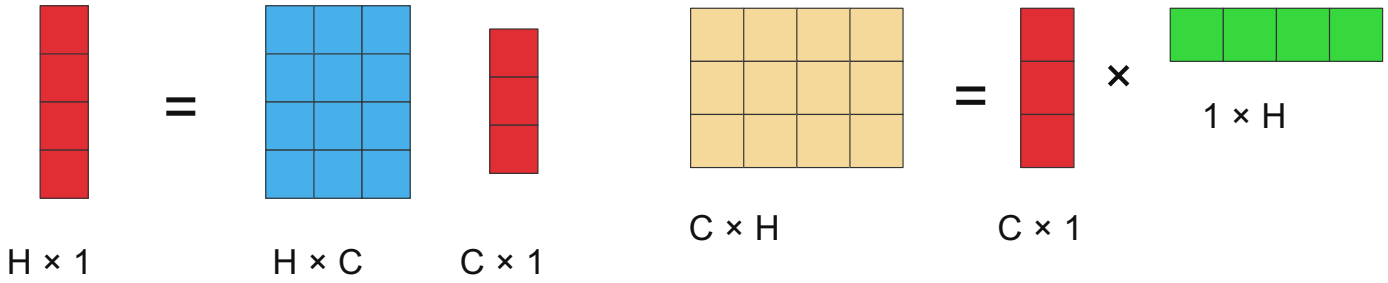


Return the activation gradient; store the weight gradient for the update.

Column-gradient convention: W is C × H, so its gradient is also C × H. Cache the input activation a.

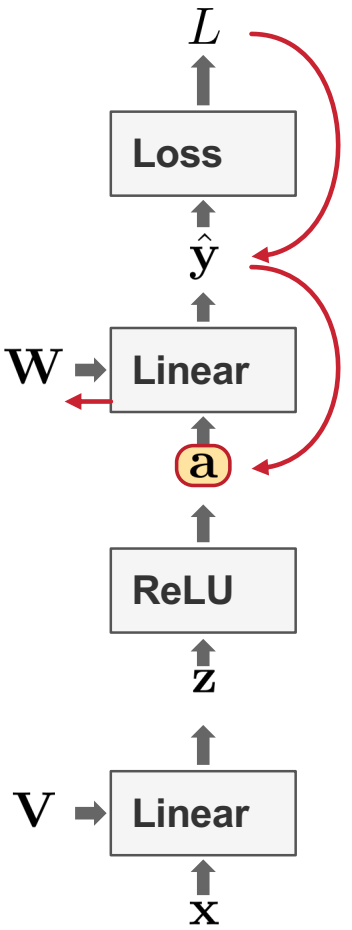
Linear layers send gradients backward

A column gradient is multiplied by the transpose of the weight matrix.



$$\underbrace{\mathbf{g}_a}_{H\times 1} = \underbrace{\mathbf{W}^T}_{H\times C}\underbrace{\mathbf{g}_{\hat{y}}}_{C\times 1}$$

$$\underbrace{\nabla_{\mathbf{W}} L}_{C\times H} = \underbrace{\mathbf{g}_{\hat{y}}}_{C\times 1}\underbrace{\mathbf{a}^T}_{1\times H}$$



Backward through ReLU

The cached score decides whether each gradient passes through.

Gradient returned | Local derivative | Incoming gradient

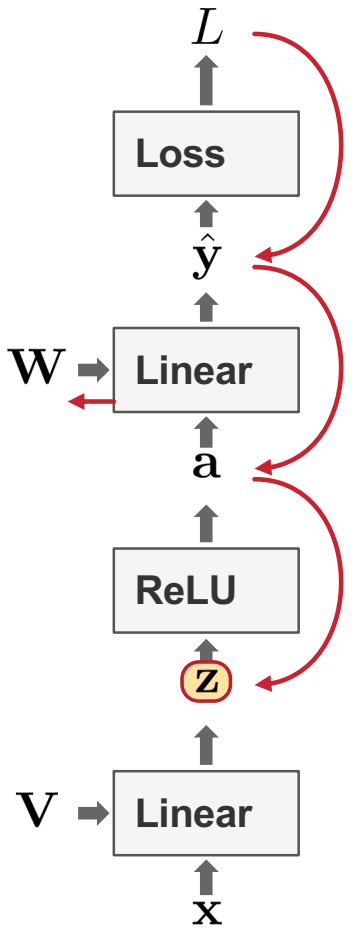
$z_h > 0$

$g_{z,h} = g_{a,h}$ × 1 $g_{a,h}$

$z_h \leq 0$

$g_{z,h} = 0$ × 0 $g_{a,h}$

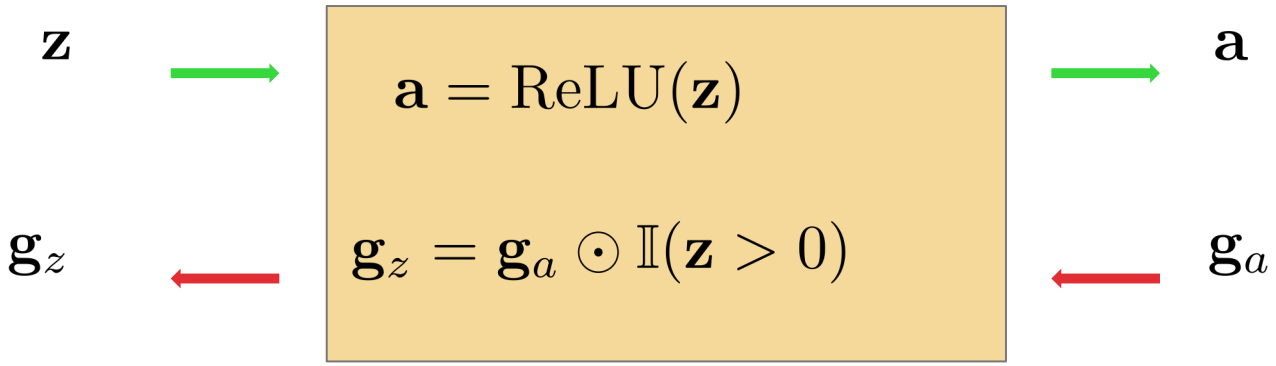
$$\mathbf{g}_z = \mathbf{g}_a \odot \mathbb{I}(\mathbf{z} > 0) \in \mathbb{R}^{H \times 1}$$



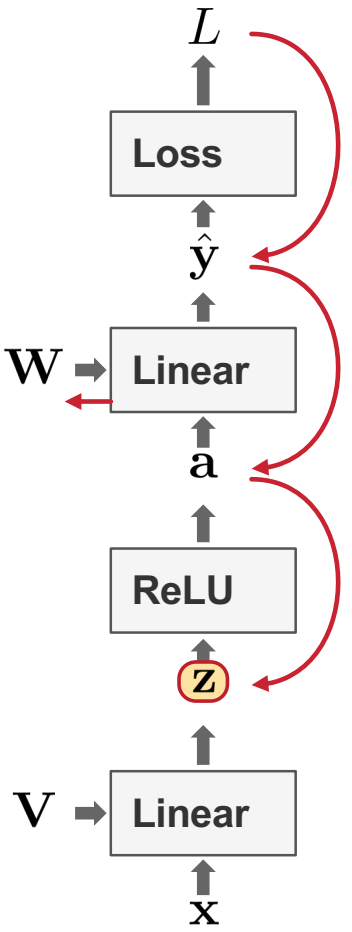
$\odot$ means elementwise multiplication. Use derivative 0 at a score of 0.

Pointwise activation derivatives

ReLU sends each incoming gradient through its cached activation gate.



$$g_{z,h} = g_{a,h}\,\mathbb{I}(z_h > 0)$$



Positive score: multiply by 1. Negative score: multiply by 0.

Use derivative 0 at the kink $z_h = 0$.

The first layer uses the same backward rule

Reuse the affine-layer rule with the hidden-score gradient and the input.

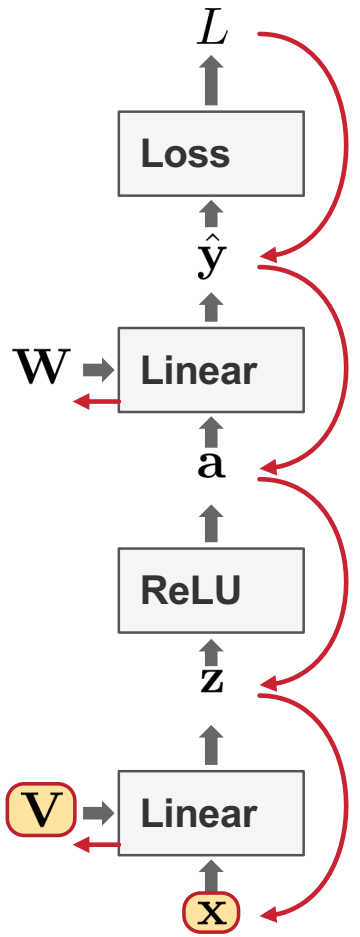
$$\mathbf{z} = \mathbf{V}\mathbf{x}, \qquad \mathbf{V} \in \mathbb{R}^{H \times D}$$

$$\frac{\partial L}{\partial V_{hi}} = g_{z,h} x_i$$

$$\underbrace{\nabla_{\mathbf{V}} L}_{H \times D} = \underbrace{\mathbf{g}_z}_{H \times 1} \underbrace{\mathbf{x}^T}_{1 \times D}$$

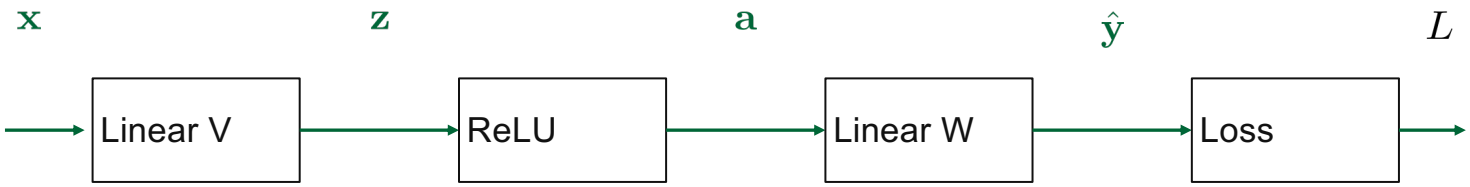
If a bias is added $\nabla_{\mathbf{b}} L = \mathbf{g}_z$

Optional input gradient $\mathbf{g}_x = \mathbf{V}^T \mathbf{g}_z$



The completed backward pass

Cache x, z and a. Compute all gradients before updating the weights.

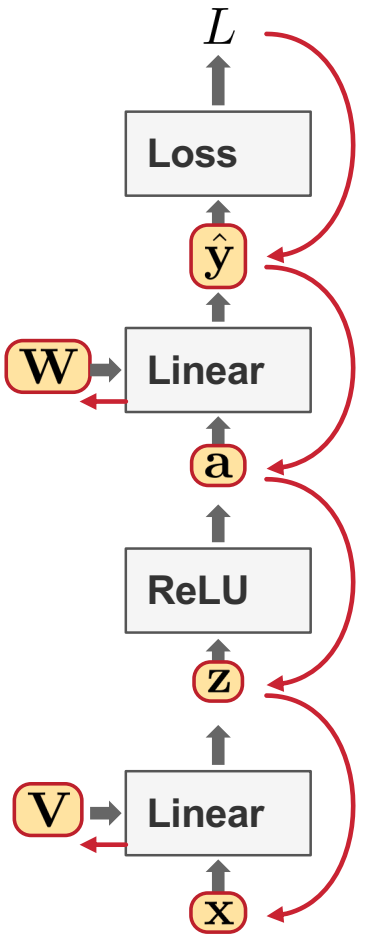


Output $\mathbf{g}_{\hat{y}} = \hat{\mathbf{y}} - \mathbf{y}$

Output linear $\nabla_{\mathbf{W}} L = \mathbf{g}_{\hat{y}} \mathbf{a}^T, \qquad \mathbf{g}_a = \mathbf{W}^T \mathbf{g}_{\hat{y}}$

ReLU $\mathbf{g}_z = \mathbf{g}_a \odot \mathbb{I}(\mathbf{z} > 0)$

First linear $\nabla_{\mathbf{V}} L = \mathbf{g}_z \mathbf{x}^T$



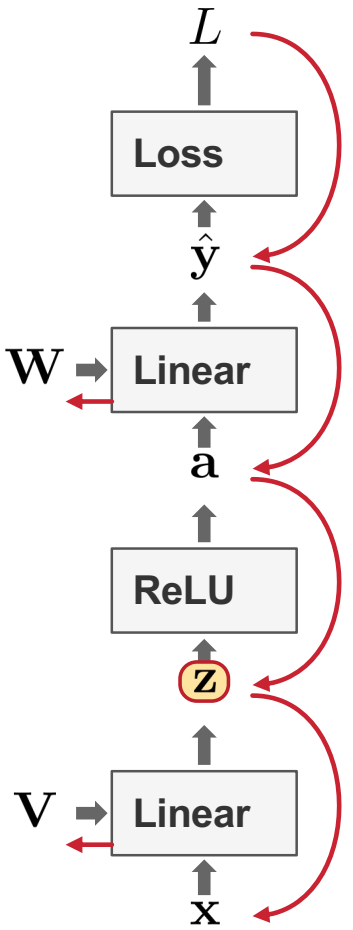
The data gradient is optional for learning the network parameters.

Why the backward pass reuses work

Compute a downstream gradient once, then reuse it for all earlier derivatives.

$$\underbrace{g_{z,h}}_{\text{compute once}} = \mathbb{I}(z_h > 0) \sum_{c=1}^{C} W_{ch} g_{\hat{y},c}$$

$$\frac{\partial L}{\partial V_{h1}} = g_{z,h} x_1, \qquad \frac{\partial L}{\partial V_{h2}} = g_{z,h} x_2, \qquad \cdots$$



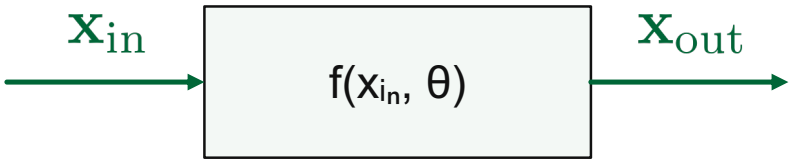
Every weight feeding this hidden unit uses the same hidden-score derivative.

One scalar loss needs one backward traversal for all parameter gradients.

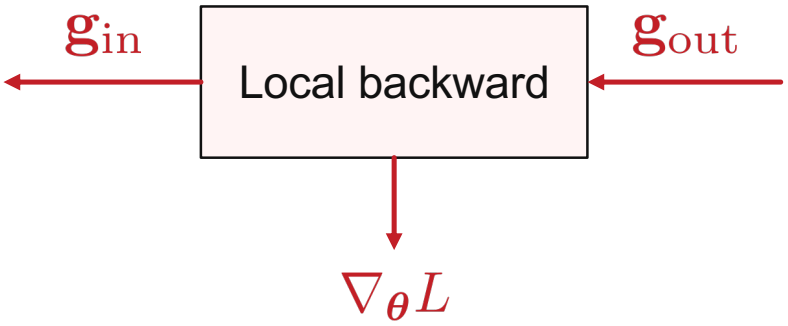
A block's local backward operation

Given the loss gradient at a block's output, return both required gradients.

Forward



Backward



$$\mathbf{J}_{\text{in}} := \frac{\partial \mathbf{x}_{\text{out}}}{\partial \mathbf{x}_{\text{in}}}, \qquad \mathbf{J}_{\theta} := \frac{\partial \mathbf{x}_{\text{out}}}{\partial \boldsymbol{\theta}}$$

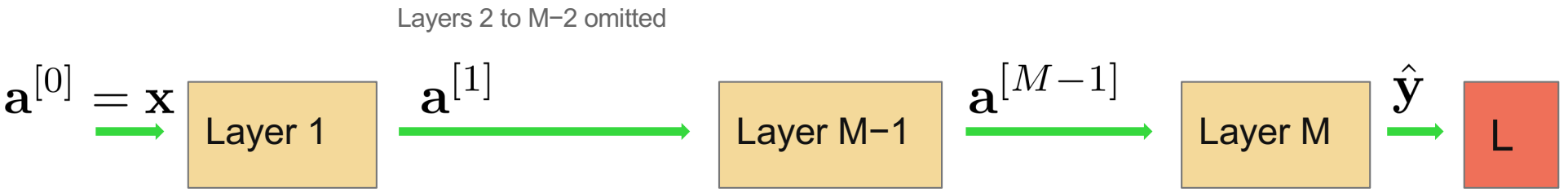
$$\mathbf{g}_{\text{in}} = \mathbf{J}_{\text{in}}^{T} \mathbf{g}_{\text{out}} \qquad \nabla_{\boldsymbol{\theta}} L = \mathbf{J}_{\theta}^{T} \mathbf{g}_{\text{out}}$$

θ stacks the block's parameters. Matrix parameter gradients keep their stored shape.

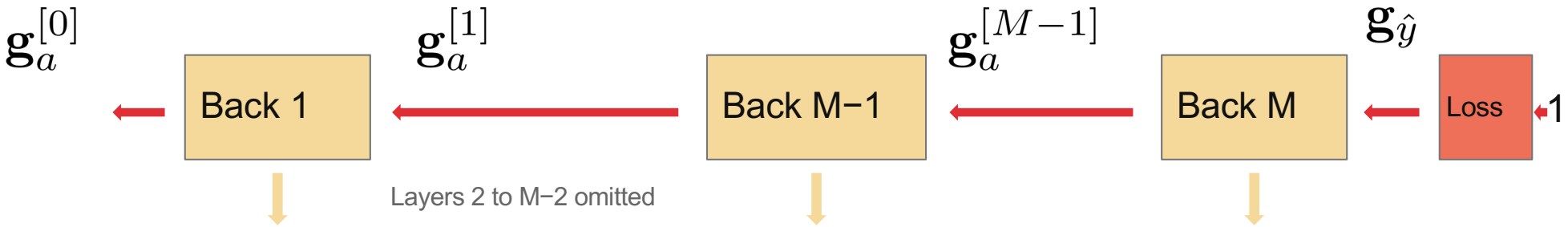
Implement the products directly. Full Jacobian matrices are unnecessary.

The full forward and backward passes

Forward: evaluate each layer and cache its values.



Backward: reuse downstream column gradients.



Each backward block also stores its own parameter gradients.

Update the parameters after the complete backward pass.

Backpropagation in a deeper MLP

M network layers. Half squared regression loss. Hidden ReLUs and a linear output.

$$\mathbf{z}^{[l]} = \mathbf{W}^{[l]}\mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}, \qquad \mathbf{a}^{[l]} = \mathrm{ReLU}(\mathbf{z}^{[l]}) \ (l < M)$$

$$\hat{\mathbf{y}} = \mathbf{z}^{[M]}, \qquad \mathbf{g}_z^{[M]} = \hat{\mathbf{y}} - \mathbf{y}$$

$$\mathbf{g}_z^{[l]} = \left((\mathbf{W}^{[l+1]})^T \mathbf{g}_z^{[l+1]}\right) \odot \mathbb{I}(\mathbf{z}^{[l]} > 0) \quad (l = M-1, \ldots, 1)$$

$$\nabla_{\mathbf{W}^{[l]}} L = \mathbf{g}_z^{[l]} (\mathbf{a}^{[l-1]})^T, \qquad \nabla_{\mathbf{b}^{[l]}} L = \mathbf{g}_z^{[l]}$$

$\mathbf{a}^{[0]} = \mathbf{x}, \qquad \mathbf{V} = \mathbf{W}^{[1]}, \quad \mathbf{W} = \mathbf{W}^{[2]}$ in the worked example

Checking one parameter gradient

Perturb one coefficient and compare the loss change with backpropagation.

$$\frac{L(\boldsymbol{\theta} + \epsilon \mathbf{e}_j) - L(\boldsymbol{\theta} - \epsilon \mathbf{e}_j)}{2\epsilon} \approx [\nabla_{\boldsymbol{\theta}} L]_j$$

θ stacks the parameters. e_j selects coefficient j. ε > 0 is a small perturbation.

Use a fixed small batch, disable randomness, and try several ε values away from activation kinks.

Branching and merging



Merge: concatenate columns

Forward $$\mathbf{x}_{\text{out}} = \begin{bmatrix} \mathbf{x}_{\text{in}}^{a} \\ \mathbf{x}_{\text{in}}^{b} \end{bmatrix}$$

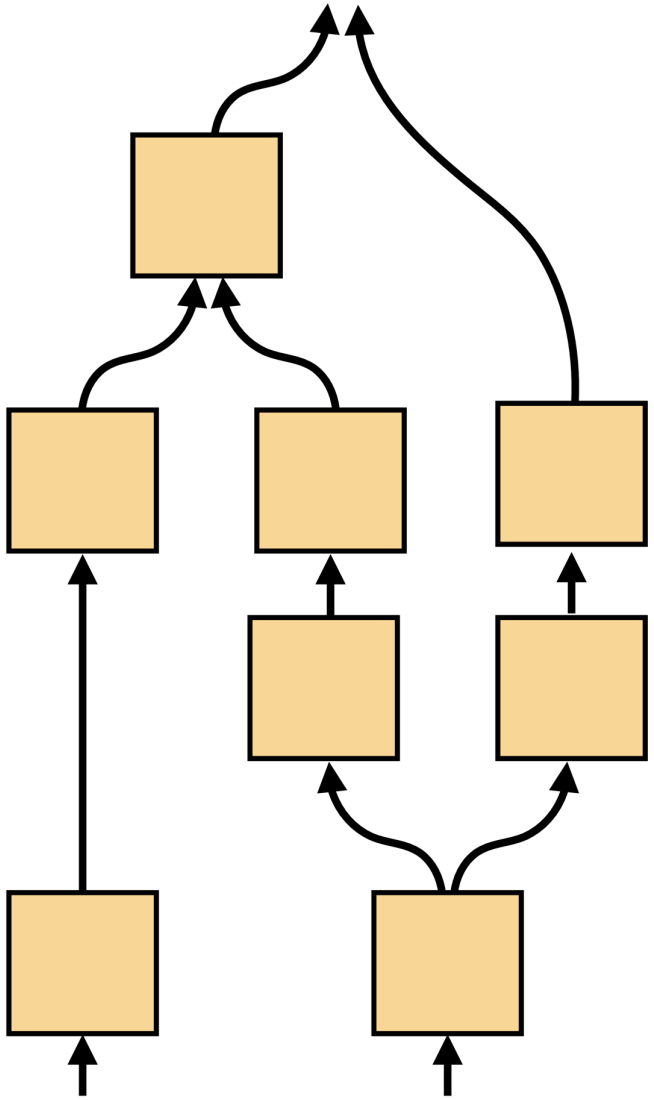
Backward $$\mathbf{g}_{\text{out}} = \begin{bmatrix} \mathbf{g}_{\text{in}}{}^{a} \\ \mathbf{g}_{\text{in}}{}^{b} \end{bmatrix}$$

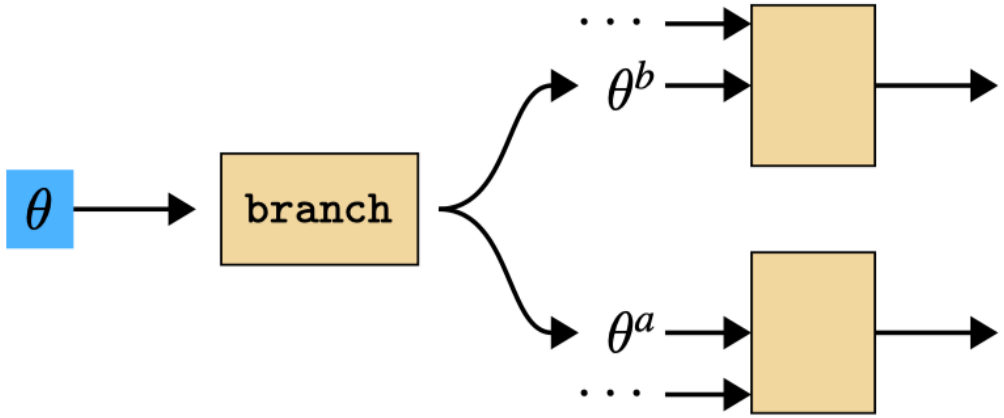
Branch: copy a column

$$\mathbf{x}_{\text{out}}^{a} = \mathbf{x}_{\text{in}}, \quad \mathbf{x}_{\text{out}}^{b} = \mathbf{x}_{\text{in}}$$

$$\mathbf{g}_{\text{in}} = \mathbf{g}_{\text{out}}{}^{a} + \mathbf{g}_{\text{out}}{}^{b}$$

Branched graphs and shared parameters





When a value or parameter feeds several paths, add the gradient contributions from those paths.

Appendix: bias and variance

Optional appendix material

Bias and variance

Forest applications

Historical context

Kernels and margins

Softmax and cross-entropy

Computation graphs

Risk and generalisation

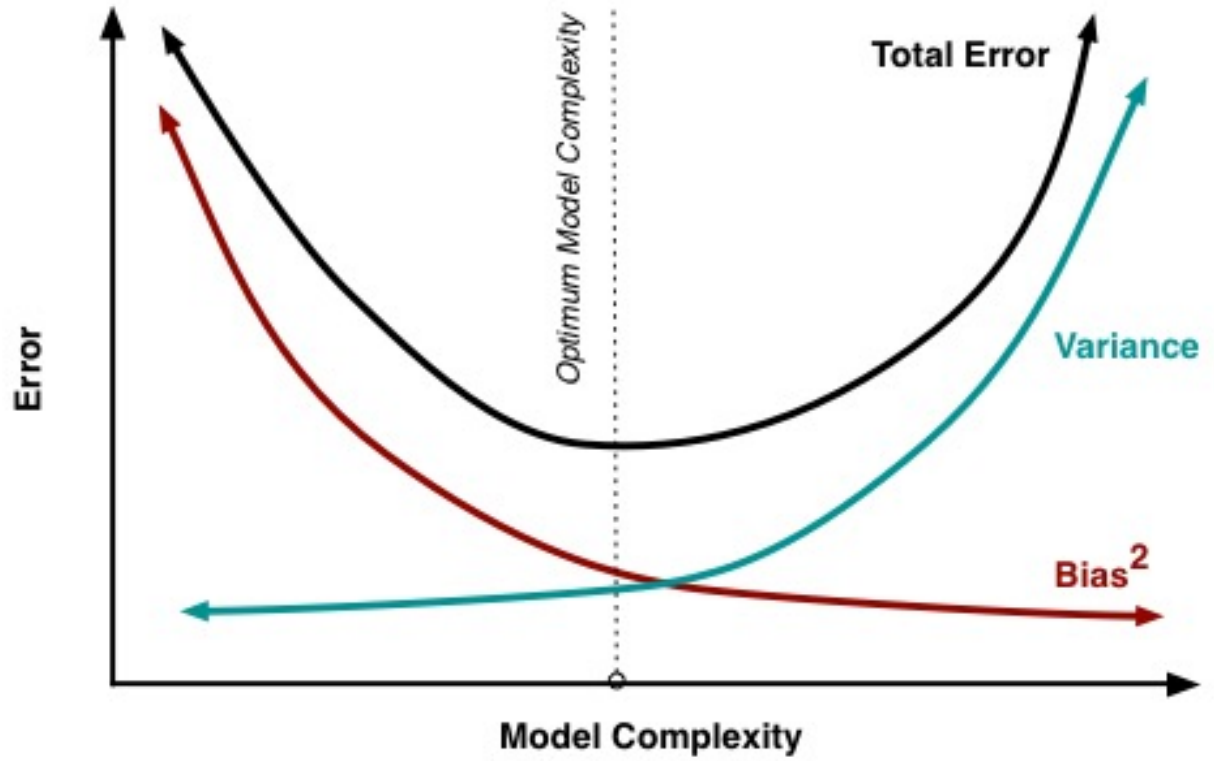
Averaging over training datasets

Bias–variance decomposition

Reading

K. Murphy (2022), §§4.3, 4.7.6 Empirical risk and the bias–variance tradeoff

Bias-variance decomposition



Schematic; noise is not shown.

Empirical risk vs population risk

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N} \qquad \ell(y, \hat{y}) = (y - \hat{y})^2$$

Training risk: mean squared error on the training set:

$$\hat{R}_{\text{train}}(f) = \frac{1}{N} \sum_{i=1}^{N} \ell(y_i, f(\mathbf{x}_i))$$

Empirical risk minimization:

$$\hat{f}_{\mathcal{D}} \in \arg\min_{f \in \mathcal{F}} \hat{R}_{\text{train}}(f)$$

Population risk:

$$R(f) = \mathbb{E}_{(\mathbf{X},Y)\sim P}[\ell(Y, f(\mathbf{X}))] = \iint \ell(y, f(\mathbf{x}))\, p(\mathbf{x}, y)\, \mathrm{d}y\, \mathrm{d}\mathbf{x}$$

F: candidate function class.
Subscript D: the training set used for this fit.
p(x, y): joint density for continuous inputs and targets.

Bias–variance decomposition at one input

$$Y = f^*(x) + \epsilon, \quad \mathbb{E}[\epsilon \mid x] = 0, \quad \mathbb{E}[\epsilon^2 \mid x] = \sigma^2(x)$$

m(x): mean fitted prediction over training datasets.

Add and subtract the true mean and the mean fit

$$Y - \hat{f}_{\mathcal{D}}(x) = \underbrace{f^*(x) - m(x)}_{\text{mean error}} + \underbrace{m(x) - \hat{f}_{\mathcal{D}}(x)}_{\text{fit fluctuation}} + \epsilon$$

Cross-terms vanish: the fluctuations have mean zero, noise is independent of the fitted predictor. $\mathbb{E}_{\mathcal{D}}[m(x) - \hat{f}_{\mathcal{D}}(x)] = 0, \qquad \mathbb{E}[\epsilon \mid x] = 0$

$$\mathbb{E}_{\mathcal{D},Y}[(Y - \hat{f}_{\mathcal{D}}(x))^2 \mid X = x]$$

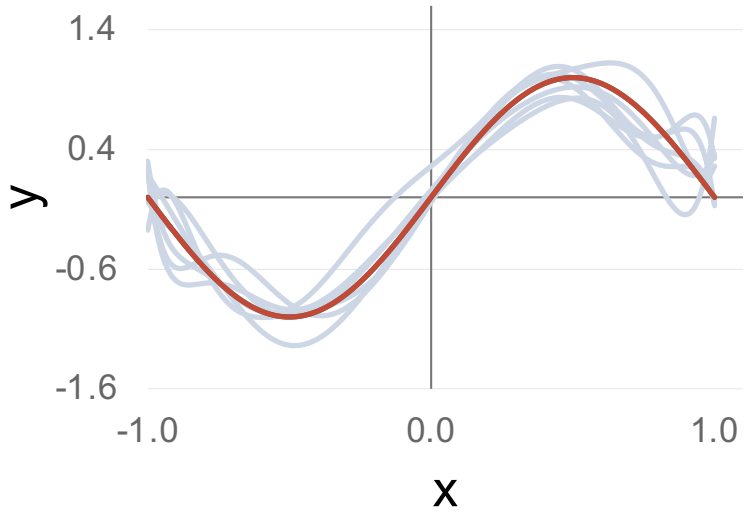
$$= \underbrace{(m(x) - f^*(x))^2}_{\text{bias}^2} + \underbrace{\mathbb{E}_{\mathcal{D}}[(\hat{f}_{\mathcal{D}}(x) - m(x))^2]}_{\text{variance}} + \underbrace{\sigma^2(x)}_{\text{noise}}$$

Target noise ε is independent of the training data. σ²(x) is noise variance at x.

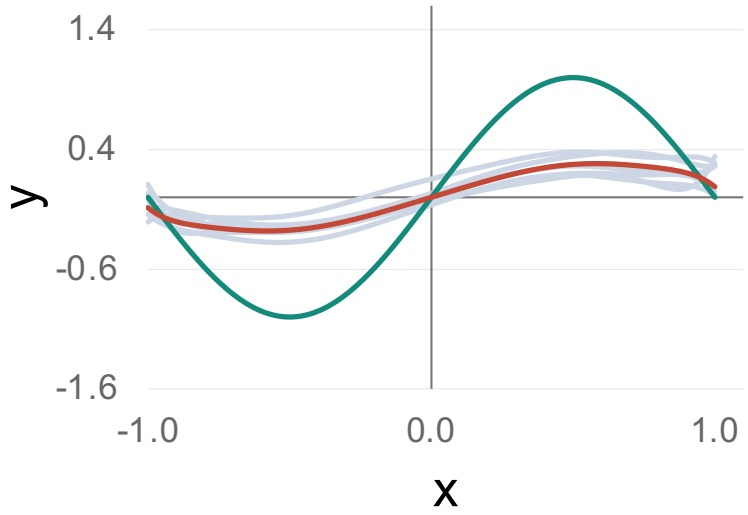
What does “averaging over datasets” mean?

Repeat collecting N examples from the same source and refitting.
Keep the function class and ridge strength λ fixed within each plot.

Weak ridge: λ = 0.0001



Strong ridge: λ = 10



At scalar input x: grey fits, red mean fit m(x), green true mean f*(x).

$$m(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)], \qquad f^*(x) = \mathbb{E}[Y \mid X = x]$$

E with subscript D averages training sets.
Here: fixed inputs, fresh noisy targets. With random inputs, resample those too.

Appendix: forest applications

Bias and variance

Forest applications

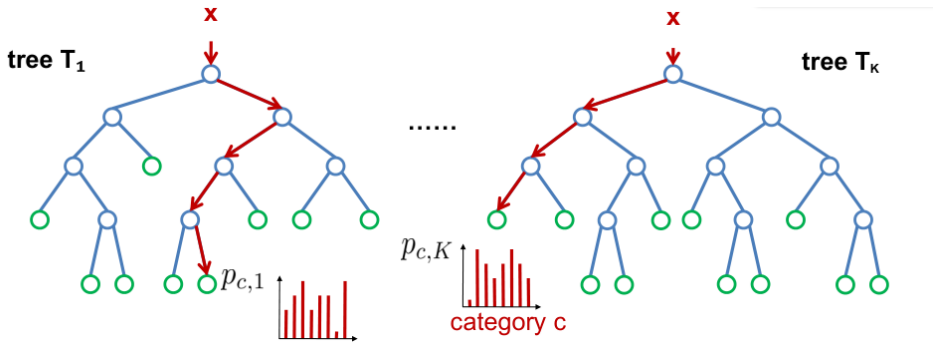
Historical context

Kernels and margins

Softmax and cross-entropy

Computation graphs

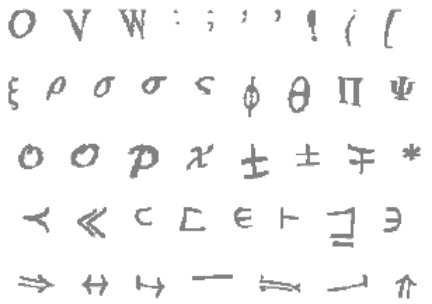
Appendix: optional material



Reading: Forest applications

K. Murphy (2022), §18.4 Random forests

Randomized Forests in Vision



[Amit & Geman, 97]
digit recognition



[Lepetit *et al.*, 06]
keypoint recognition



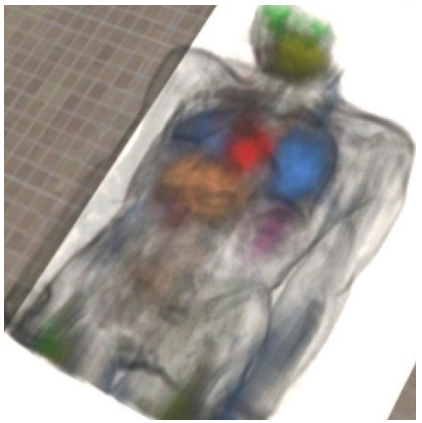
[Moosmann *et al.*, 06]
visual word clustering



[Shotton *et al.*, 08]
object segmentation



[Rogez *et al.*, 08]
pose estimation



[Criminisi *et al.*, 09]
organ detection

Selected applications

Kinect's pipeline







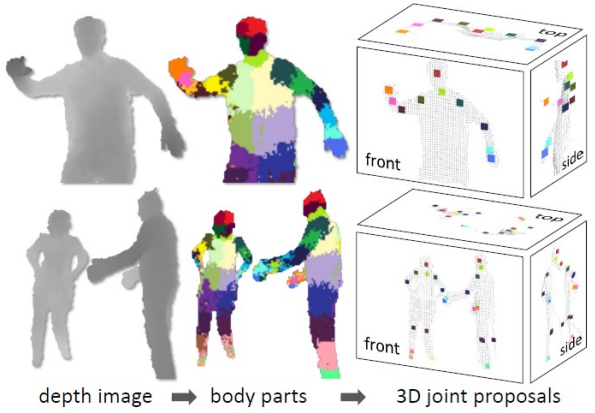


Figure 1. **Overview.** From an single input depth image, a per-pixel body part distribution is inferred. (Colors indicate the most likely part labels at each pixel, and correspond in the joint proposals). Local modes of this signal are estimated to give high-quality proposals for the 3D locations of body joints, even for multiple users.

A single depth image supplies a classification problem at each pixel.

A random forest predicts body-part labels.

The body-part distributions support 3D joint estimates.

Shotton et al., CVPR 2011: Real-Time Human Pose Recognition in Parts from a Single Depth Image

Appendix: historical context

Optional appendix material

Bias and variance

Forest applications

Historical context

Kernels and margins

Softmax and cross-entropy

Computation graphs

Bayesian estimation

Least squares

Logistic and ridge regression

Reading

Wasserman (2004), All of Statistics, Chs. 11, 13 and 22

Statistical foundations: historical context

Bayesian parameter estimation **1763–1774**

Bayes (1763), Laplace (1774): infer unknown parameters from data.

Linear regression **1805–1809**

Legendre (1805), Gauss (1809): least squares.
The term “regression” followed with Galton’s work in the 1880s.

Logistic regression **1944–1958**

Berkson (1944), Cox (1958): model binary-outcome probabilities.

Ridge regression **1962–1970**

Hoerl (1962), Hoerl & Kennard (1970): coefficient shrinkage.
The 1970 paper already gives the Gaussian-prior interpretation.

Modern machine learning builds on these statistical foundations:
learning from data, regularisation and probabilistic prediction.

Wasserman (2004), All of Statistics, Chs. 11, 13 and 22.

Hastie, Tibshirani & Friedman (2009), The Elements of Statistical Learning.

Appendix: kernels and margins

Optional appendix material

Bias and variance

Forest applications

Historical context

Kernels and margins

Softmax and cross-entropy

Computation graphs

Large margins

Kernel evaluations

Gaussian feature expansion

Reading

K. Murphy (2022), §§17.1, 17.3

Why support vector machines mattered

1990s: SVMs challenge multilayer perceptrons (MLPs)

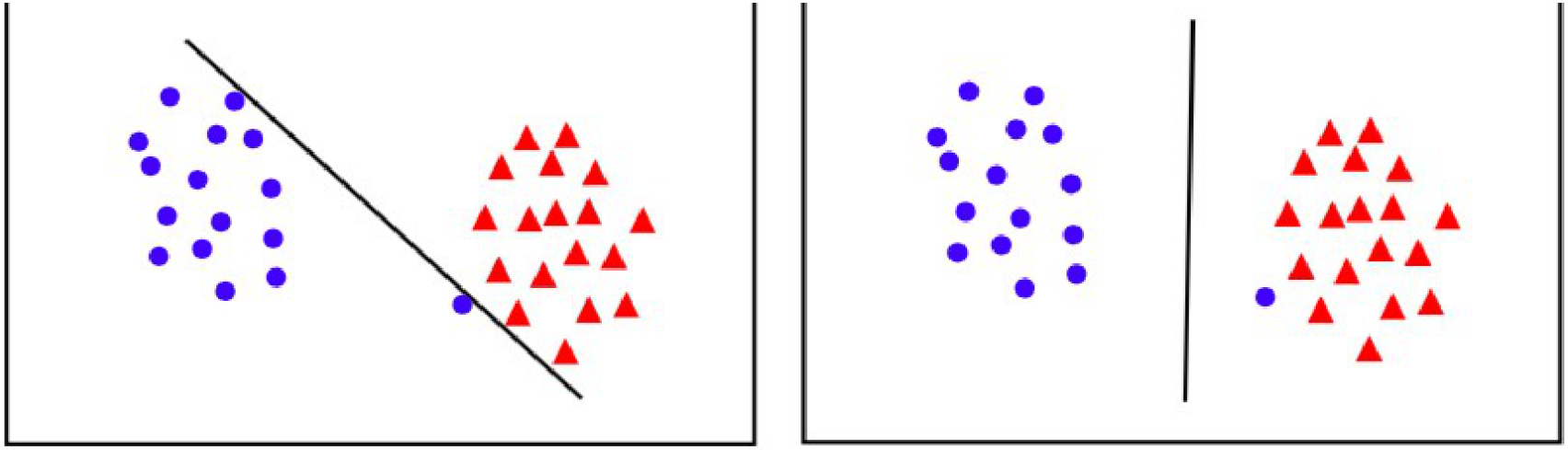
SVMs became a leading alternative to MLPs, with convex training, margin-based theory and flexible kernels.

Handwritten digit recognition was an important comparison between kernel methods and neural networks.

The strongest method depended on the benchmark, preprocessing and training data.

Cortes & Vapnik (1995); LeCun et al. (1998)

Large margins and imperfect separation



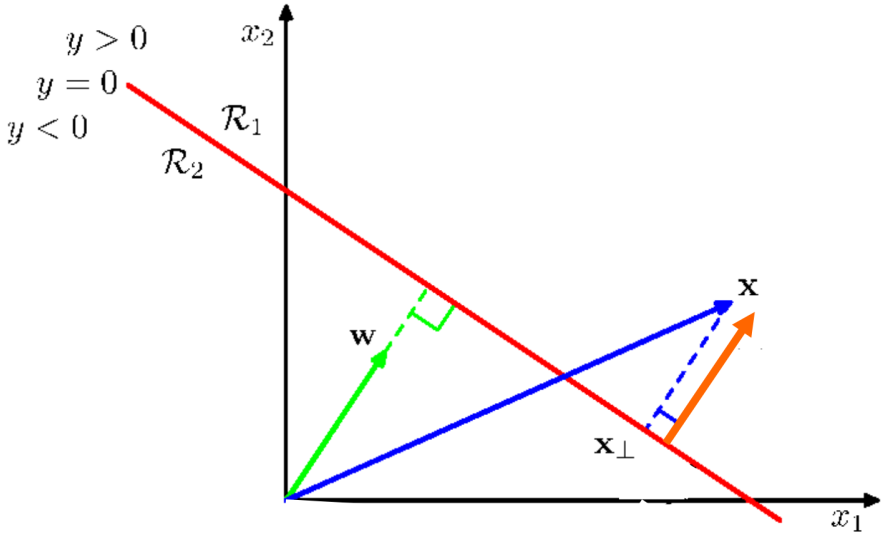
All points on the correct side!

But this looks better overall!

Prefer a separator with the largest minimum geometric margin.

Allow some violations when they improve the overall tradeoff.
Choose the tradeoff using validation data.

Geometric margin



The signed distance to the decision boundary measures how clearly an example is classified.

$$\gamma_i = \frac{y_i(\mathbf{w}^T\mathbf{x}_i + b)}{\|\mathbf{w}\|}$$

Labels $y_i \in \{-1, +1\}$. A positive margin means correct classification.
Positive rescaling of w and b leaves the geometric margin unchanged.

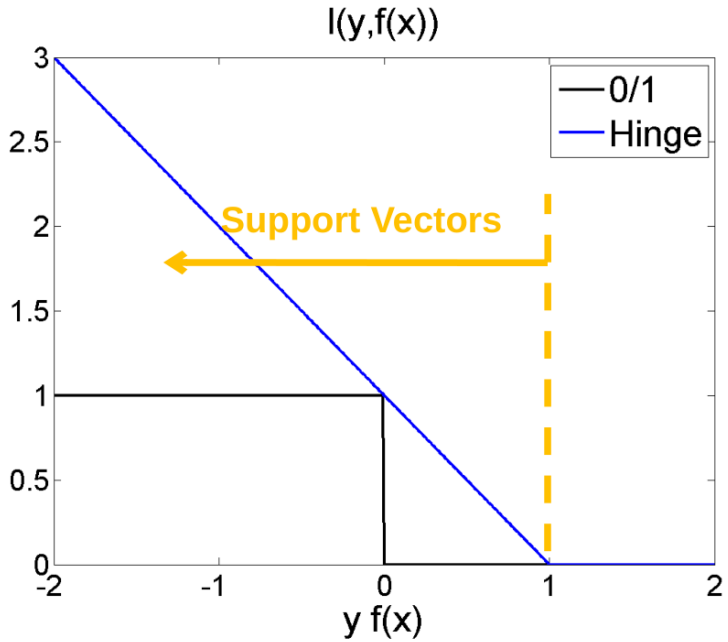
Hinge loss and regularization

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N}\max(0, 1 - y_i f(\mathbf{x}_i))$$

$f(x) = w^T x + b$
Labels $y = \pm 1$; margin $y\, f(x)$.

The weight penalty favours a wide margin.

Hinge loss penalises margins below 1.



Larger C puts more weight on fitting the training examples.

The kernel training problem

Binary labels $y_i = \pm 1$; α_i are learned nonnegative coefficients.

$$k(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^T \phi(\mathbf{z})$$

$$\max_{\boldsymbol{\alpha}} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j k(\mathbf{x}_i, \mathbf{x}_j)$$

$$0 \leq \alpha_i \leq C, \qquad \sum_i \alpha_i y_i = 0$$

Only inner products are needed. A kernel computes them directly, without constructing the feature vector explicitly.

Kernel examples

Linear kernel

$$k(\mathbf{x}, \mathbf{z}) = \mathbf{x}^T \mathbf{z}$$

Polynomial kernel

$$k(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^T \mathbf{z} + 1)^p$$

Radial basis function (Gaussian) kernel

$$k(\mathbf{x}, \mathbf{z}) = \exp\left[-\frac{\|\mathbf{x} - \mathbf{z}\|^2}{2\sigma^2}\right], \qquad \sigma > 0$$

Underlying feature dimension for the Gaussian kernel: infinite

The Gaussian feature expansion

$$\gamma = \frac{1}{2\sigma^2} > 0, \qquad x, z \in \mathbb{R}$$

$$\begin{aligned} k(x,z) &= e^{-\gamma(x-z)^2} \\ &= e^{-\gamma x^2} e^{-\gamma z^2} e^{2\gamma xz} \\ &= e^{-\gamma x^2} e^{-\gamma z^2} \sum_{r=0}^{\infty} \frac{(2\gamma xz)^r}{r!} \\ &= \sum_{r=0}^{\infty} \left[e^{-\gamma x^2} \sqrt{\frac{(2\gamma)^r}{r!}} x^r \right] \left[e^{-\gamma z^2} \sqrt{\frac{(2\gamma)^r}{r!}} z^r \right] \\ &= \phi(x)^T \phi(z). \end{aligned}$$

The feature vector contains one coordinate for every non-negative integer:

$$\phi(x) = e^{-\gamma x^2} \left(1, \sqrt{2\gamma}\, x, \sqrt{\frac{(2\gamma)^2}{2!}} x^2, \sqrt{\frac{(2\gamma)^3}{3!}} x^3, \ldots \right)^T$$

Appendix: multiclass backpropagation

Optional appendix material

Bias and variance

Forest applications

Historical context

Kernels and margins

Softmax and cross-entropy

Computation graphs

Linear scores

Softmax probabilities

Cross-entropy loss

Reading

K. Murphy (2022), §§10.3, 13.3

Multiclass example: forward pass

One example. The target y is one-hot over C classes. Biases are omitted.

Linear scores z $\mathbf{z} = \mathbf{W}\mathbf{x}, \qquad \mathbf{W} \in \mathbb{R}^{C \times D}, \quad \mathbf{z} \in \mathbb{R}^{C \times 1}$

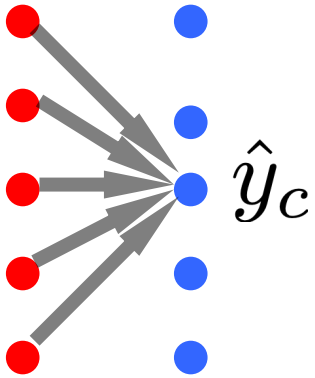
Softmax ŷ $\hat{y}_c = \dfrac{\exp(z_c)}{\sum_{k=1}^{C} \exp(z_k)}, \qquad c = 1, \ldots, C$

Cross-entropy L $L = -\sum_{c=1}^{C} y_c \log \hat{y}_c = -\log \hat{y}_{c_*}$

Cache x, z and ŷ. The correct class is c*.

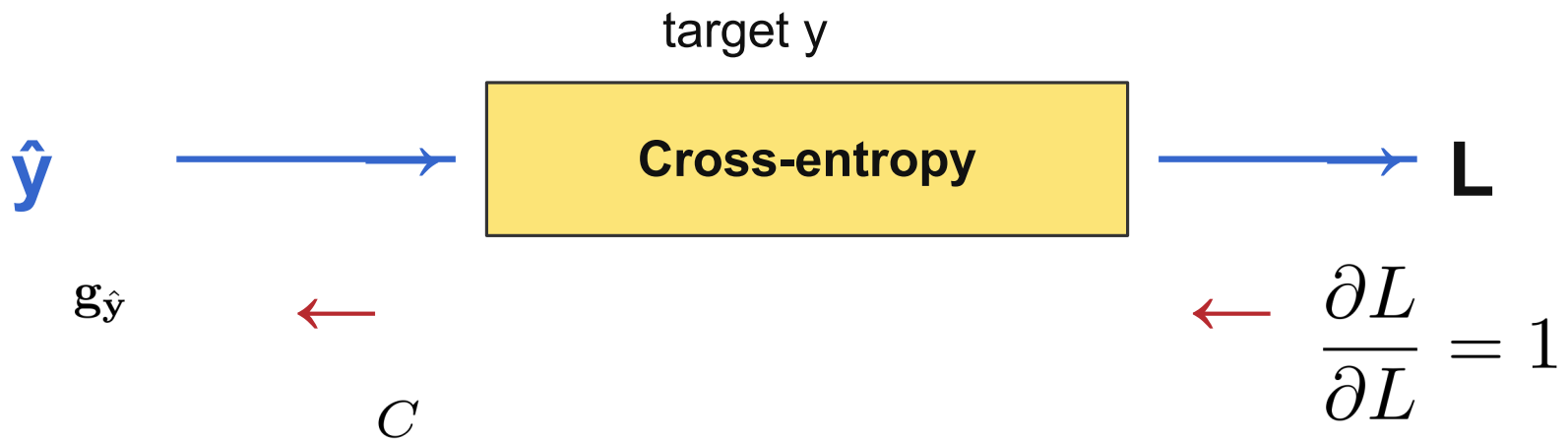
Softmax in forward mode: all for one

$$\hat{y}_c = \frac{\exp(z_c)}{\sum_{c\prime=1}^{C} \exp(z_{c\prime})}$$



Cross-entropy as a computation layer

Forward: one-hot y selects the correct class, c*.



$$L = -\sum_{c=1}^{C} y_c \log \hat{y}_c = -\log \hat{y}_{c_*}$$

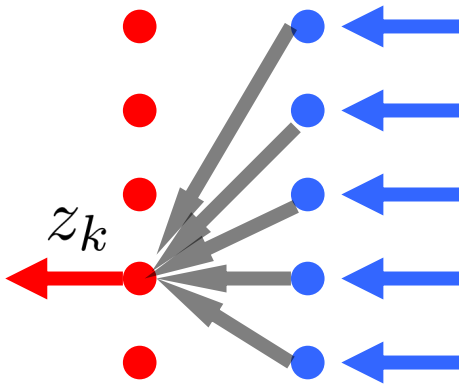
Backward: the loss block sends a gradient into softmax.

$$\frac{\partial L}{\partial \hat{y}_c} = -\frac{y_c}{\hat{y}_c}, \qquad \mathbf{g}_{\hat{\mathbf{y}}} = -\mathbf{y} \oslash \hat{\mathbf{y}}$$

Pass this column gradient to the softmax backward rule.

Softmax in backward mode: one from all

$$\hat{y}_c = \frac{\exp(z_c)}{\sum_{c\prime=1}^{C} \exp(z_{c\prime})}$$



$$\frac{\partial L}{\partial \hat{y}_c} = -\frac{y_c}{\hat{y}_c}$$

$$\frac{\partial L}{\partial z_k} = \sum_c \frac{\partial L}{\partial \hat{y}_c} \frac{\partial \hat{y}_c}{\partial z_k}$$

Softmax and cross-entropy: the chain rule

Softmax local derivative

$$\frac{\partial \hat{y}_c}{\partial z_k} = \hat{y}_c\big([c = k] - \hat{y}_k\big)$$

[c = k] equals 1 when c = k, and 0 otherwise.

Chain with the loss gradient

$$\frac{\partial L}{\partial z_k} = \sum_{c=1}^{C} \frac{\partial L}{\partial \hat{y}_c} \frac{\partial \hat{y}_c}{\partial z_k}$$

$$= \sum_{c=1}^{C} \left(-\frac{y_c}{\hat{y}_c} \right) \hat{y}_c\big([c = k] - \hat{y}_k\big)$$

Use the one-hot target: its entries sum to one.

$$= -y_k + \hat{y}_k \sum_{c=1}^{C} y_c = \hat{y}_k - y_k$$

Multiclass example: backpropagation

The cross-entropy and softmax layers give the score gradient.

Score gradient

$$\mathbf{g_z} = \frac{\partial L}{\partial \mathbf{z}} = \hat{\mathbf{y}} - \mathbf{y} \in \mathbb{R}^{C \times 1}$$

Weight gradient

$$\frac{\partial L}{\partial \mathbf{W}} = \mathbf{g_z} \mathbf{x}^T \in \mathbb{R}^{C \times D}$$

Input gradient

$$\mathbf{g_x} = \mathbf{W}^T \mathbf{g_z} \in \mathbb{R}^{D \times 1}$$

For a mean data loss, average these per-example gradients.

Appendix: computation graphs

Bias and variance

Forest applications

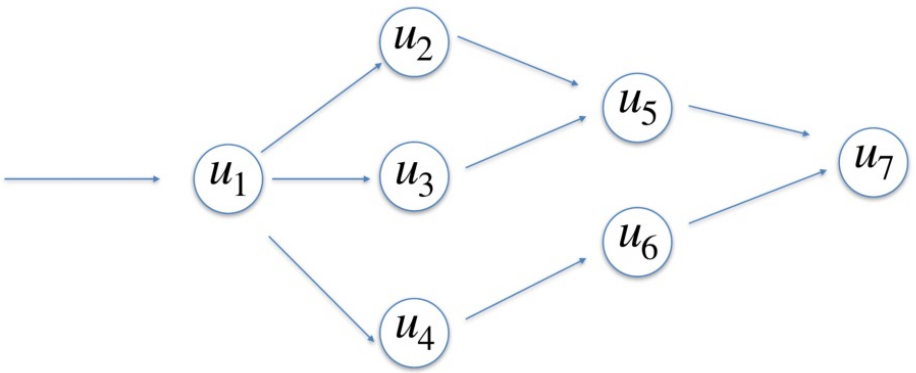
Historical context

Kernels and margins

Softmax and cross-entropy

Computation graphs

Appendix: optional material

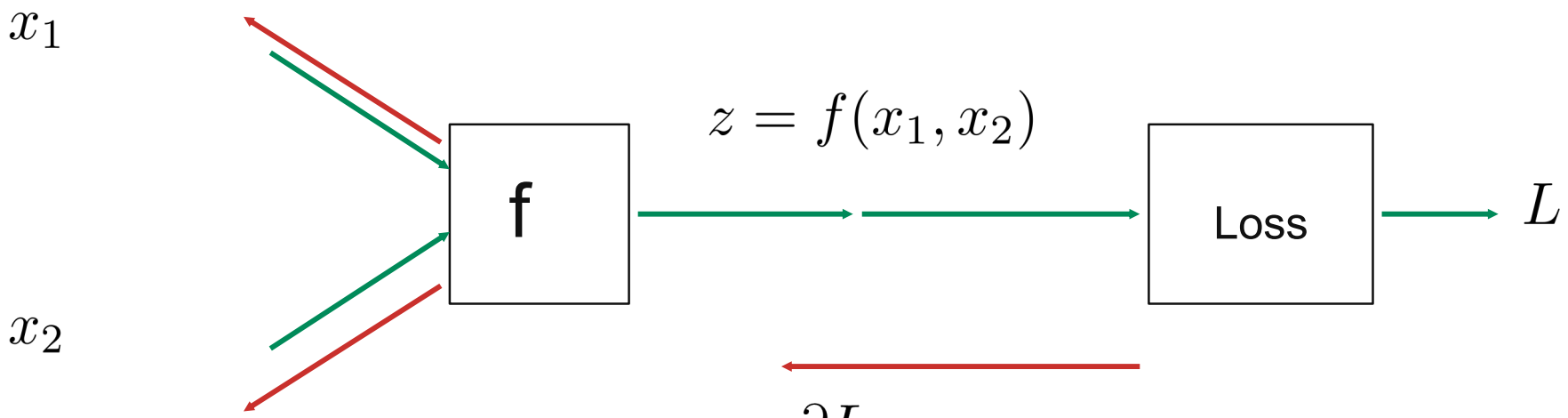


Reading: Computation graphs

K. Murphy (2022), §13.3.4: Computation graphs
COMP0169 lecture slides, pp. 24–32

Basic block

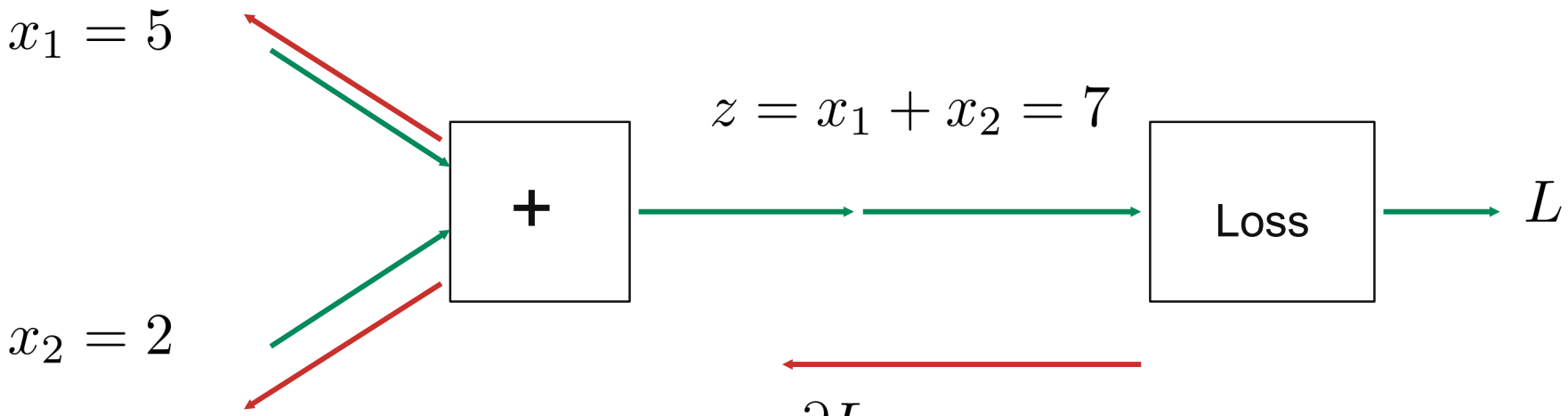
x_1 and x_2 are scalar inputs.



$$\frac{\partial z}{\partial x_1}, \qquad \frac{\partial z}{\partial x_2}$$

$$\frac{\partial L}{\partial x_1} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial x_1} \qquad \frac{\partial L}{\partial x_2} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial x_2}$$

Backpropagation: addition

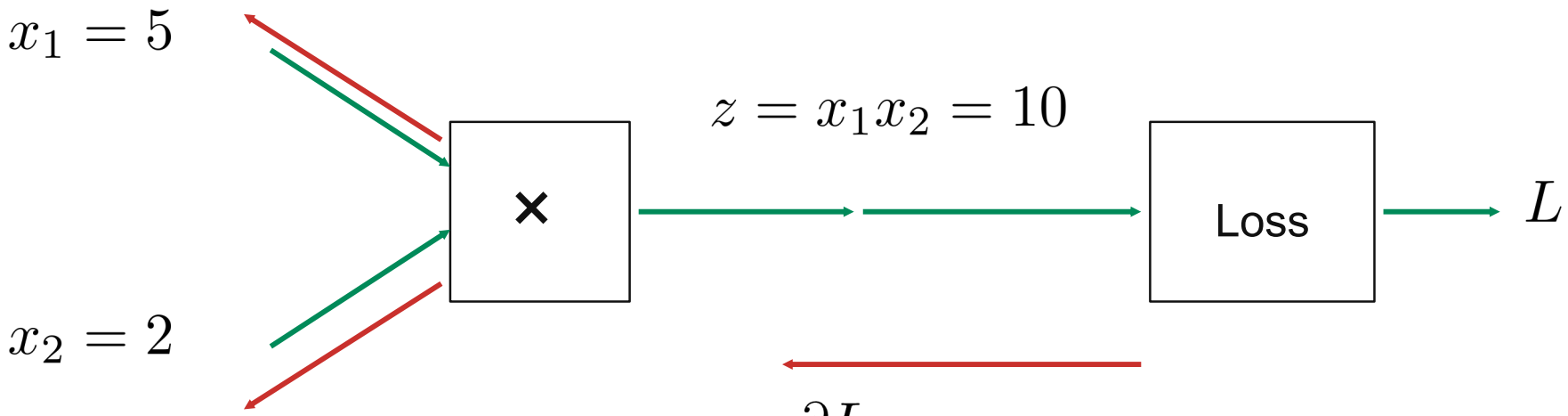


$$\frac{\partial L}{\partial z} = 15$$

$$\frac{\partial z}{\partial x_1} = 1, \qquad \frac{\partial z}{\partial x_2} = 1$$

$$\frac{\partial L}{\partial x_1} = \frac{\partial L}{\partial z} \cdot 1 = 15 \qquad \frac{\partial L}{\partial x_2} = \frac{\partial L}{\partial z} \cdot 1 = 15$$

Backpropagation: multiplication

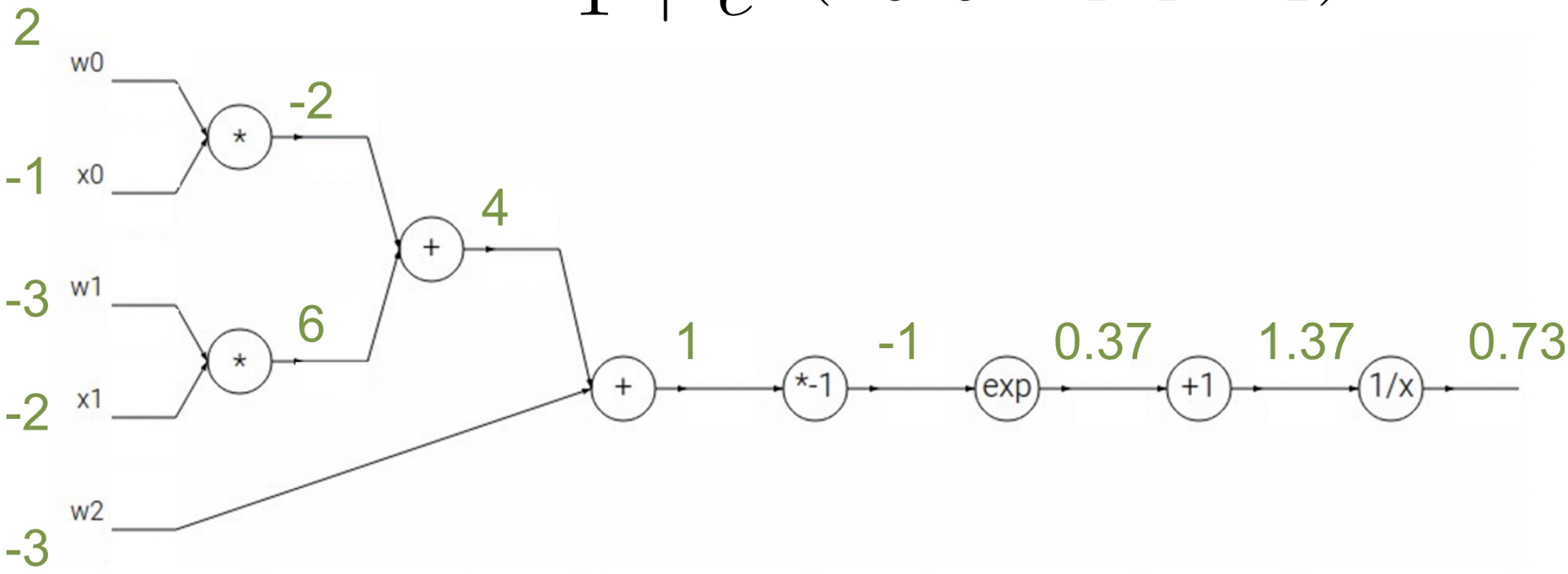


$$\frac{\partial z}{\partial x_1} = x_2 = 2, \qquad \frac{\partial z}{\partial x_2} = x_1 = 5$$

$$\frac{\partial L}{\partial x_1} = \frac{\partial L}{\partial z} x_2 = 7 \cdot 2 = 14 \qquad \frac{\partial L}{\partial x_2} = \frac{\partial L}{\partial z} x_1 = 7 \cdot 5 = 35$$

Example 2

$$f(\mathbf{x}, \mathbf{w}) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



Source notation: w_2 is the bias b; inputs are indexed 0 and 1.

Niloy, COMP0169, pp. 28–30

Backpropagation on a small computation

$$f = (r + s)t, \quad r = 3,\ s = 5,\ t = -2$$

$$q = r + s = 8, \qquad f = qt = -16$$

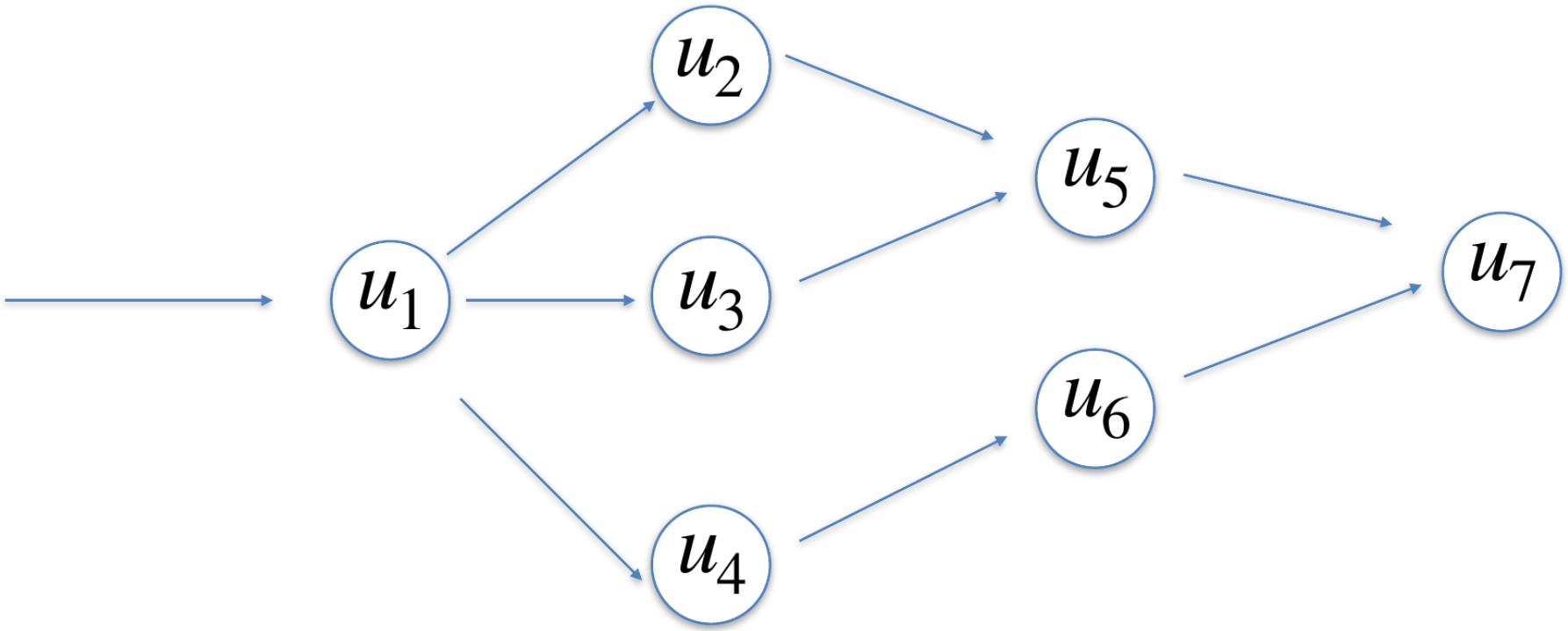
$$\frac{\partial f}{\partial q} = t = -2, \quad \frac{\partial f}{\partial t} = q = 8$$

$$\frac{\partial f}{\partial r} = \frac{\partial f}{\partial q} \cdot 1 = -2, \quad \frac{\partial f}{\partial s} = -2$$

Multiplication uses the other input. Addition sends the gradient to both inputs.

Worked example adapted from supplied COMP0169 slides, PDF pp. 24–27

Ordering a computation graph



A forward pass visits each operation after its inputs.
Several topological orders can be valid.

Source: COMP0169, PDF p. 31

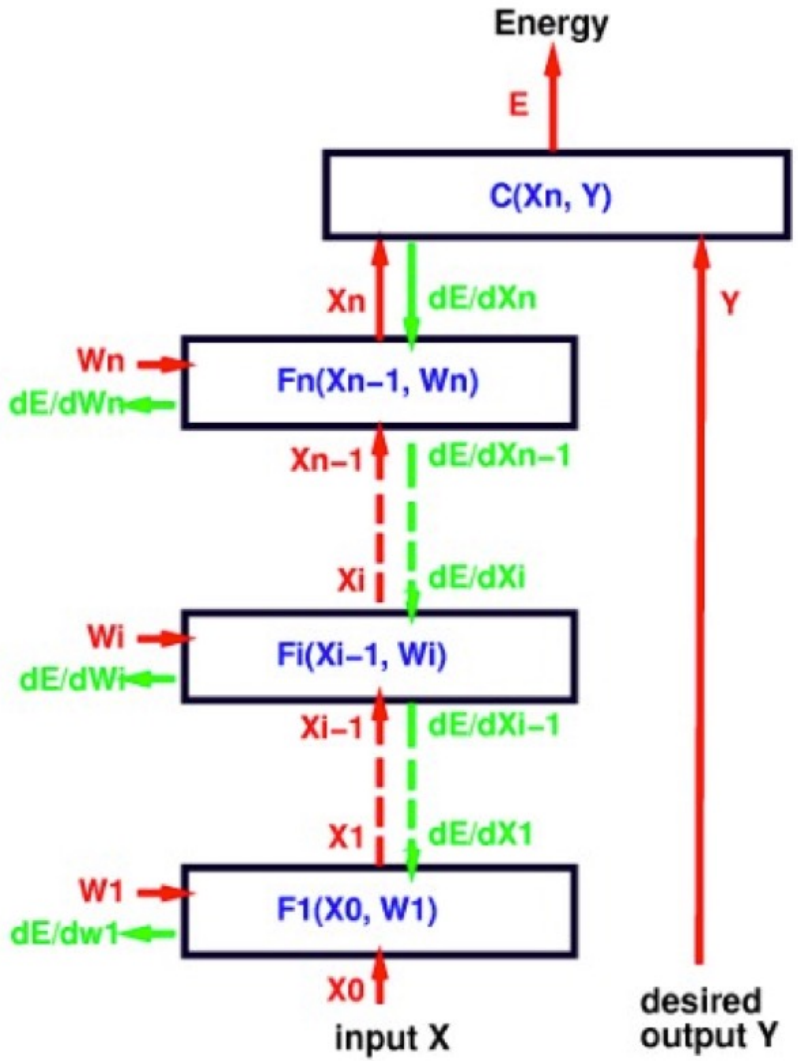
Backpropagation on a graph

```
class ComputationalGraph(object):
    #...
    def forward(inputs):
        # 1. [pass inputs to input gates...]
        # 2. forward the computational graph:
        for gate in self.graph.nodes_topologically_sorted():
            gate.forward()
        return loss # the final gate in the graph outputs the loss
    def backward():
        for gate in reversed(self.graph.nodes_topologically_sorted()):
            gate.backward() # little piece of backprop (chain rule applied)
        return inputs_gradients
```

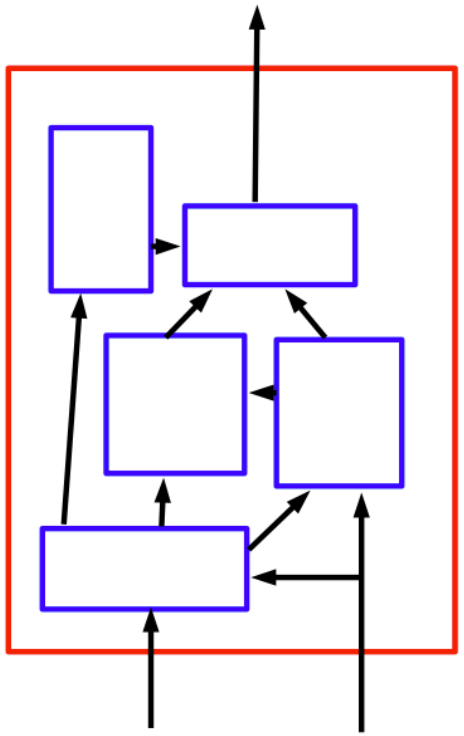
The backward pass reverses the forward order.
Each operation contributes gradients to its inputs.

Source: COMP0169, PDF p. 32

Forward/backward information flow



Backpropagation computes gradients in acyclic graphs with suitable local derivatives.



Source notation: E = L; X_i is a layer activation.

Accompanying reading

Trees and forests

K. Murphy (2022), §§18.1, 18.3–18.4

Kernels and SVMs

K. Murphy (2022), §§17.1, 17.3 (optional)

Neural networks and representations

K. Murphy (2022), §13.2

Backpropagation and computation graphs

K. Murphy (2022), §13.3

Appendix: multiclass loss and softmax

K. Murphy (2022), §10.3 and §13.3

References: the eight ML milestones

[1] Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. Psychological Review, 65(6), 386–408.

[2] Breiman, L., Friedman, J. H., Olshen, R. A. & Stone, C. J. (1984). Classification and Regression Trees. Wadsworth & Brooks/Cole.

[3] Rumelhart, D. E., Hinton, G. E. & Williams, R. J. (1986). Learning representations by back-propagating errors. Nature, 323, 533–536.

[4] Cortes, C. & Vapnik, V. (1995). Support-vector networks. Machine Learning, 20(3), 273–297.

[5] LeCun, Y., Bottou, L., Bengio, Y. & Haffner, P. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), 2278–2324.

[6] Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32.

[7] Krizhevsky, A., Sutskever, I. & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. Advances in Neural Information Processing Systems, 25, 1097–1105.

[8] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. & Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30, 5998–6008.